

Bölüm 12: Kümeler

Veri Yapıları

Küme ADT

Birleşim, kesişim, fark

Hash ve ağaç tabanlı küme

Çoklu küme

Union-Find

Yol kısaltma, rütbe

Kruskal

Sercan KÜLCÜ · Giresun Üniversitesi · Bilgisayar Mühendisliği

Öğrenme Çıktıları

Küme modeli

Küme ADT'sini ve Java `Set` arayüzünü açıklamak.

Çoklu küme

Tekrarlı öge gerektiren problemlerde çoklu küme yaklaşımını kullanmak.

Union-Find

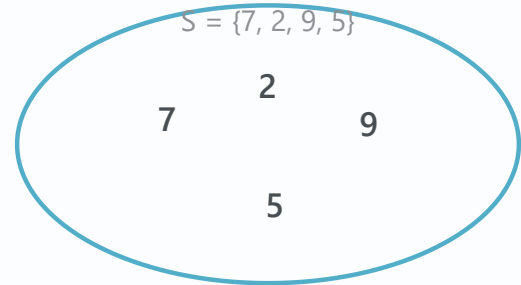
Yol kısaltma ve rütbeye göre birleştirme yöntemlerini uygulamak.

Uygulama

Union-Find yapısını Kruskal algoritması ve bağlılık problemleriyle ilişkilendirmek.

Küme (Set)

- Küme (set), **benzersiz (tekil)** öğelerden oluşan bir koleksiyondur: aynı öğe iki kez bulunmaz.
- Matematikteki küme kavramının veri yapısı karşılığıdır.
- Öğelerin sırası önemsizdir: $\{1, 2, 3\} = \{3, 1, 2\}$.
- Temel soru: **"x kümede var mı?"** (üyelik testi) hızlı yanıtlanmalıdır.



`add(2)` → değişmez (2 zaten var)

Tekrar ayıklama

Bir listedeki farklı kelimeler.

```
new HashSet<>(liste)
```

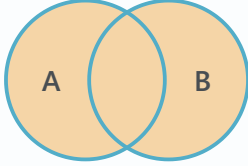
Üyelik

Yasaklı kullanıcılar, ziyaret edilen düğümler (Bölüm 11).

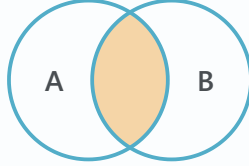
Küme cebri

Ortak arkadaşlar = kesişim;
tüm etiketler = birleşim.

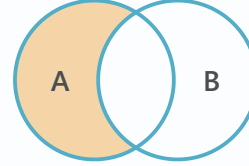
Küme İşlemleri (İşleçler)



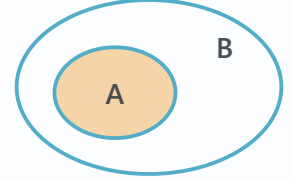
Birleşim $A \cup B$
A'da *veya* B'de olanlar



Kesişim $A \cap B$
Hem A'da *hem* B'de olanlar



Fark $A - B$
A'da olup B'de olmayanlar



Alt küme $A \subseteq B$
A'nın her ögesi B'de

A = {1, 2, 3, 4, 5}, B = {3, 4, 5, 6, 7}	Sonuç
Birleşim $A \cup B$	{1, 2, 3, 4, 5, 6, 7}
Kesişim $A \cap B$	{3, 4, 5}
Fark $A - B$ / $B - A$	{1, 2} / {6, 7}
Alt küme?	$A \not\subseteq B$ ($1 \notin B$) · $\{3, 4\} \subseteq A$ ✓

java.util.Set Arayüzü

Metot	Açıklama
<code>add(e)</code>	ekler; varsa <code>false</code>
<code>remove(o)</code> , <code>contains(o)</code>	çıkartır; üyeliği sorgular
<code>addAll(T)</code>	birleşim: $S \leftarrow S \cup T$
<code>retainAll(T)</code>	kesişim: $S \leftarrow S \cap T$
<code>removeAll(T)</code>	fark: $S \leftarrow S - T$

```
Set<Integer> a = new HashSet<>(List.of(1,2,3,4,5));
Set<Integer> b = new HashSet<>(List.of(3,4,5,6,7));
Set<Integer> bir = new HashSet<>(a);
bir.addAll(b);      // [1..7]
Set<Integer> kes = new HashSet<>(a);
kes.retainAll(b);   // [3, 4, 5]
Set<Integer> fark = new HashSet<>(a);
fark.removeAll(b);  // [1, 2]
a.containsAll(b);   // false
```

`addAll` , `retainAll` , `removeAll` kümeyi **yerinde değiştirir**; a'yı korumak için önce kopyalanır.

Kod İzleme: add dönüşüyle ilk tekrar

a

4	7	2	7	4
0	1	2	3	4

gorulen: HashSet, 16 kova
kova = hash(x) & 15 (Integer: hash = x)

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Boş bir **HashSet** oluşturulur: 16 boş kova. Her öğe **hash & 15** kovasına gider.

Adım 1 / 10

```

1 int ilkTekrar(int[] a) {
2     Set<Integer> gorulen = new HashSet<>();
3     for (int x : a) {
4         if (!gorulen.add(x)) // eklenemedi mi?
5             return x;       // zaten vardı: tekrar
6     }
7     return -1;              // tekrar yok
8 }

```

DEĞİŞKENLER

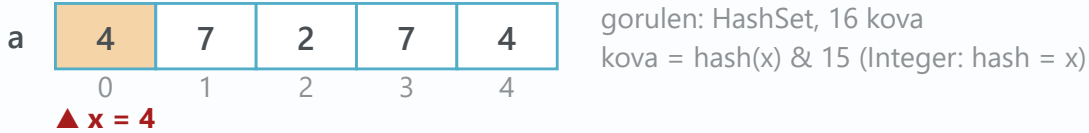
a = [4, 7, 2, 7, 4] gorulen = {}

ÇAĞRI YIĞINI

ilkTekrar(a)

main()

Kod İzleme: add dönüşüyle ilk tekrar



Döngü sıradaki öğeyi alır: **x = 4**.

Adım 2 / 10

```
1 int ilkTekrar(int[] a) {  
2     Set<Integer> gorulen = new HashSet<>();  
3     for (int x : a) {  
4         if (!gorulen.add(x)) // eklenemedi mi?  
5             return x;        // zaten vardı: tekrar  
6     }  
7     return -1;               // tekrar yok  
8 }
```

DEĞİŞKENLER

a = [4, 7, 2, 7, 4]

gorulen = {}

x = 4

ÇAĞRI YIĞINI

ilkTekrar(a)

main()

Kod İzleme: add dönüşüyle ilk tekrar

a

4	7	2	7	4
0	1	2	3	4

▲ x = 4

gorulen: HashSet, 16 kova
kova = hash(x) & 15 (Integer: hash = x)

				4												
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	

add(4) : kova 4 boş, öge eklenir ve **true** döner → **!true** yanlış, döngü sürer. Tek kova bakıldı: O(1).

Adım 3 / 10

```
1 int ilkTekrar(int[] a) {  
2     Set<Integer> gorulen = new HashSet<>();  
3     for (int x : a) {  
4         if (!gorulen.add(x)) // eklenemedi mi?  
5             return x;        // zaten vardı: tekrar  
6     }  
7     return -1;                // tekrar yok  
8 }
```

DEĞİŞKENLER

a = [4, 7, 2, 7, 4]

gorulen = {4}

x = 4

add = true

ÇAĞRI YIĞINI

ilkTekrar(a)

main()

Kod İzleme: add dönüşüyle ilk tekrar



gorulen: HashSet, 16 kova
kova = hash(x) & 15 (Integer: hash = x)



Döngü sıradaki öğeyi alır: x = 7.

Adım 4 / 10

```
1 int ilkTekrar(int[] a) {  
2     Set<Integer> gorulen = new HashSet<>();  
3     for (int x : a) {  
4         if (!gorulen.add(x)) // eklenemedi mi?  
5             return x;        // zaten vardı: tekrar  
6     }  
7     return -1;                // tekrar yok  
8 }
```

DEĞİŞKENLER

a = [4, 7, 2, 7, 4]

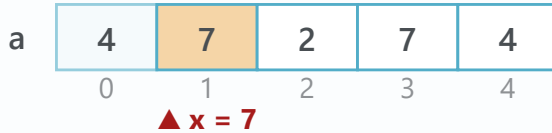
gorulen = {4} x = 7

ÇAĞRI YIĞINI

ilkTekrar(a)

main()

Kod İzleme: add dönüşüyle ilk tekrar



gorulen: HashSet, 16 kova
kova = hash(x) & 15 (Integer: hash = x)



add(7) : kova 7 boş, öge eklenir ve **true** döner → **!true** yanlış, döngü sürer. Tek kova bakıldı: O(1).

Adım 5 / 10

```
1 int ilkTekrar(int[] a) {  
2     Set<Integer> gorulen = new HashSet<>();  
3     for (int x : a) {  
4         if (!gorulen.add(x)) // eklenemedi mi?  
5             return x;        // zaten vardı: tekrar  
6     }  
7     return -1;               // tekrar yok  
8 }
```

DEĞİŞKENLER

a = [4, 7, 2, 7, 4]

gorulen = {4, 7} x = 7

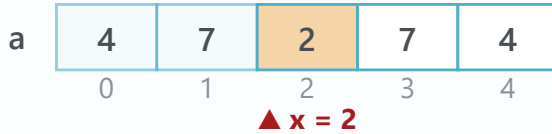
add = true

ÇAĞRI YIĞINI

ilkTekrar(a)

main()

Kod İzleme: add dönüşüyle ilk tekrar



gorulen: HashSet, 16 kova
kova = hash(x) & 15 (Integer: hash = x)



Döngü sıradaki öğeyi alır: **x = 2**.

Adım 6 / 10

```
1 int ilkTekrar(int[] a) {  
2     Set<Integer> gorulen = new HashSet<>();  
3     for (int x : a) {  
4         if (!gorulen.add(x)) // eklenemedi mi?  
5             return x;       // zaten vardı: tekrar  
6     }  
7     return -1;              // tekrar yok  
8 }
```

DEĞİŞKENLER

a = [4, 7, 2, 7, 4]

gorulen = {4, 7} x = 2

ÇAĞRI YIĞINI

ilkTekrar(a)

main()

Kod İzleme: add dönüşüyle ilk tekrar

a

4	7	2	7	4
0	1	2	3	4

▲ x = 2

gorulen: HashSet, 16 kova

kova = hash(x) & 15 (Integer: hash = x)

		2		4			7								
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

add(2) : kova 2 boş, öge eklenir ve **true** döner → **!true** yanlış, döngü sürer. Tek kova bakıldı: O(1).

Adım 7 / 10

```

1 int ilkTekrar(int[] a) {
2     Set<Integer> gorulen = new HashSet<>();
3     for (int x : a) {
4         if (!gorulen.add(x)) // eklenemedi mi?
5             return x;        // zaten vardı: tekrar
6     }
7     return -1;               // tekrar yok
8 }

```

DEĞİŞKENLER

a = [4, 7, 2, 7, 4]

gorulen = {2, 4, 7}

x = 2

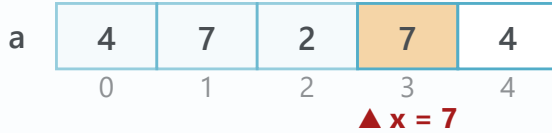
add = true

ÇAĞRI YIĞINI

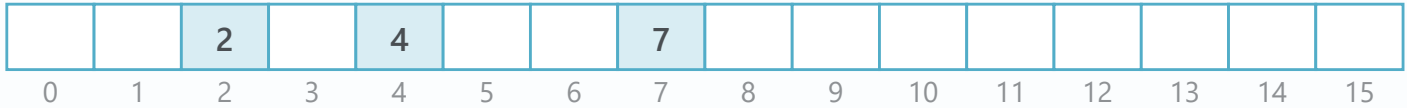
ilkTekrar(a)

main()

Kod İzleme: add dönüşüyle ilk tekrar



gorulen: HashSet, 16 kova
kova = hash(x) & 15 (Integer: hash = x)



Döngü sıradaki öğeyi alır: x = 7.

Adım 8 / 10

```

1 int ilkTekrar(int[] a) {
2     Set<Integer> gorulen = new HashSet<>();
3     for (int x : a) {
4         if (!gorulen.add(x)) // eklenemedi mi?
5             return x;        // zaten vardı: tekrar
6     }
7     return -1;               // tekrar yok
8 }

```

DEĞİŞKENLER

a = [4, 7, 2, 7, 4]

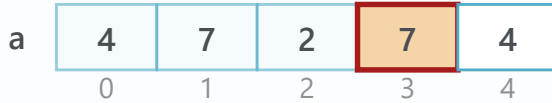
gorulen = {2, 4, 7} x = 7

ÇAĞRI YIĞINI

ilkTekrar(a)

main()

Kod İzleme: add dönüşüyle ilk tekrar



▲ x = 7

gorulen: HashSet, 16 kova
kova = hash(x) & 15 (Integer: hash = x)

zaten var!



add(7) : kova 7'de **7 zaten var**; küme değişmez ve **false** döner → koşul doğru.

Adım 9 / 10

```

1 int ilkTekrar(int[] a) {
2     Set<Integer> gorulen = new HashSet<>();
3     for (int x : a) {
4         if (!gorulen.add(x)) // eklenemedi mi?
5             return x;        // zaten vardı: tekrar
6     }
7     return -1;               // tekrar yok
8 }

```

DEĞİŞKENLER

a = [4, 7, 2, 7, 4]

gorulen = {2, 4, 7} x = 7

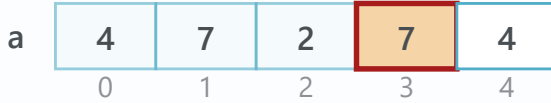
add = false

ÇAĞRI YIĞINI

ilkTekrar(a)

main()

Kod İzleme: add dönüşüyle ilk tekrar



▲ x = 7

gorulen: HashSet, 16 kova
kova = hash(x) & 15 (Integer: hash = x)

zaten var!



İlk tekrar eden öge **7** döndürülür. Toplam iş $O(n)$: her öge için bir `add` ; iç içe döngü yok.

Adım 10 / 10

```

1 int ilkTekrar(int[] a) {
2     Set<Integer> gorulen = new HashSet<>();
3     for (int x : a) {
4         if (!gorulen.add(x)) // eklenemedi mi?
5             return x;       // zaten vardı: tekrar
6     }
7     return -1;              // tekrar yok
8 }

```

DEĞİŞKENLER

a = [4, 7, 2, 7, 4]

gorulen = {2, 4, 7} x = 7

dönüş = 7

ÇAĞRI YIĞINI

ilkTekrar(a)

main()

java.util.Set Uygulamaları

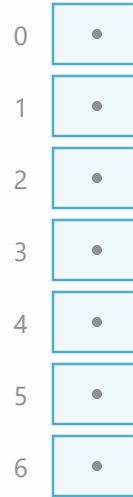
Sınıf	İç yapı	Gezinme sırası	add / contains / remove
HashSet	hash tablosu (Bölüm 9)	hash koduna bağlı, öngörülemez	O(1) ortalama
LinkedHashSet	hash tablosu + çift bağlı liste	ekleme sırası	O(1) ortalama
TreeSet	kırmızı-siyah ağaç (Bölüm 10)	sıralı (küçükten büyüğe, alfabetik)	O(log n)

```
for (String s : List.of("elma", "kiraz", "armut", "elma")) {  
    hash.add(s); linked.add(s); tree.add(s);  
}  
// HashSet      → [elma, armut, kiraz]    (kova sırası: hash'e bağlı)  
// LinkedHashSet → [elma, kiraz, armut]    (eklenme sırası)  
// TreeSet      → [armut, elma, kiraz]    (alfabetik)
```

Tekrar eden "elma" üçünde de **bir kez** bulunur. Sıra önemsizse HashSet, ekleme sırası için LinkedHashSet, sıralı erişim için TreeSet.

Hash Tabanlı Küme

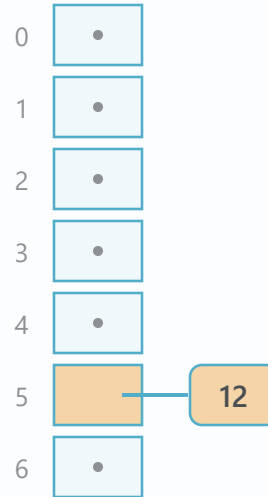
- Küme bir **hash tablosu** olarak temsil edilir (Bölüm 9); yalnızca anahtar saklanır, değer yoktur.
- Her öğenin konumu hash koduna göre hızla hesaplanır.
- Ekleme, silme ve arama ortalama **$O(1)$** ; kötü hash ile en kötü $O(n)$.
- Java'da `HashSet` içeride bir `HashMap` kullanır.



Başlangıç: 7 kovalı boş tablo. Konum = `hashCode(x)`
`mod 7`.

Hash Tabanlı Küme

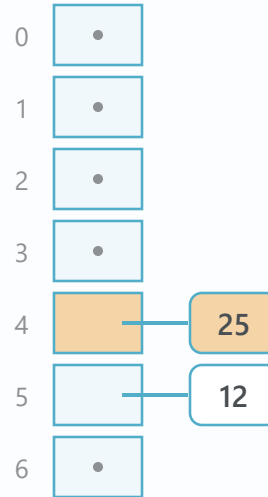
- Küme bir **hash tablosu** olarak temsil edilir (Bölüm 9); yalnızca anahtar saklanır, değer yoktur.
- Her öğenin konumu hash koduna göre hızla hesaplanır.
- Ekleme, silme ve arama ortalama **$O(1)$** ; kötü hash ile en kötü $O(n)$.
- Java'da `HashSet` içeride bir `HashMap` kullanır.



`add(12):` $12 \bmod 7 = 5 \rightarrow$ kova 5 boş, eklenir.

Hash Tabanlı Küme

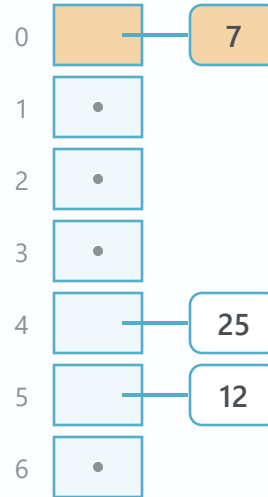
- Küme bir **hash tablosu** olarak temsil edilir (Bölüm 9); yalnızca anahtar saklanır, değer yoktur.
- Her öğenin konumu hash koduna göre hızla hesaplanır.
- Ekleme, silme ve arama ortalama **$O(1)$** ; kötü hash ile en kötü $O(n)$.
- Java'da `HashSet` içeride bir `HashMap` kullanır.



add(25): $25 \bmod 7 = 4 \rightarrow$ kova 4'e eklenir.

Hash Tabanlı Küme

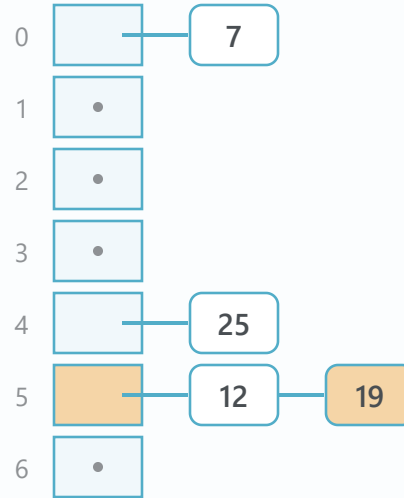
- Küme bir **hash tablosu** olarak temsil edilir (Bölüm 9); yalnızca anahtar saklanır, değer yoktur.
- Her öğenin konumu hash koduna göre hızla hesaplanır.
- Ekleme, silme ve arama ortalama **$O(1)$** ; kötü hash ile en kötü $O(n)$.
- Java'da `HashSet` içeride bir `HashMap` kullanır.



add(7): $7 \bmod 7 = 0 \rightarrow$ kova 0'a eklenir.

Hash Tabanlı Küme

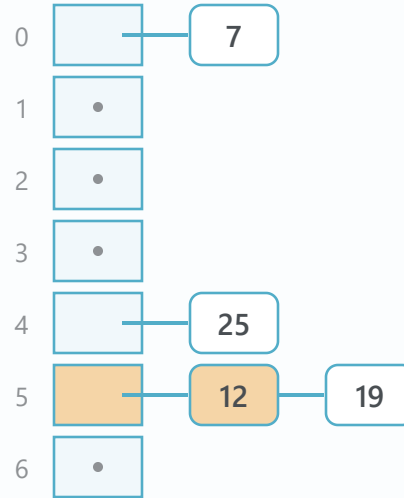
- Küme bir **hash tablosu** olarak temsil edilir (Bölüm 9); yalnızca anahtar saklanır, değer yoktur.
- Her öğenin konumu hash koduna göre hızla hesaplanır.
- Ekleme, silme ve arama ortalama **$O(1)$** ; kötü hash ile en kötü $O(n)$.
- Java'da `HashSet` içeride bir `HashMap` kullanır.



add(19): $19 \bmod 7 = 5 \rightarrow$ çakışma! Kova 5'te 12 var; 19 zincirin sonuna eklenir.

Hash Tabanlı Küme

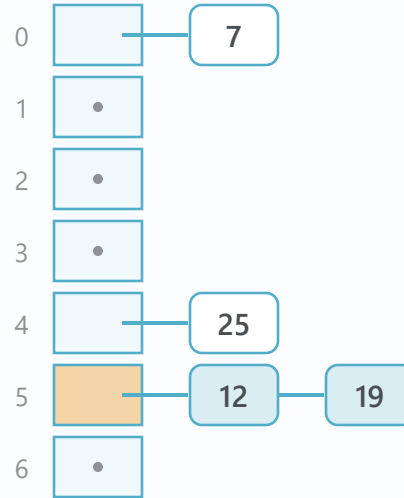
- Küme bir **hash tablosu** olarak temsil edilir (Bölüm 9); yalnızca anahtar saklanır, değer yoktur.
- Her öğenin konumu hash koduna göre hızla hesaplanır.
- Ekleme, silme ve arama ortalama **$O(1)$** ; kötü hash ile en kötü $O(n)$.
- Java'da `HashSet` içeride bir `HashMap` kullanır.



add(12) tekrar: kova 5 taranır, 12 bulunur → **eklenmez**,
`false` döner. Kümeyi küme yapan budur.

Hash Tabanlı Küme

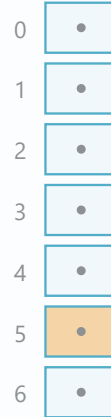
- Küme bir **hash tablosu** olarak temsil edilir (Bölüm 9); yalnızca anahtar saklanır, değer yoktur.
- Her öğenin konumu hash koduna göre hızla hesaplanır.
- Ekleme, silme ve arama ortalama **$O(1)$** ; kötü hash ile en kötü $O(n)$.
- Java'da `HashSet` içeride bir `HashMap` kullanır.



`contains(26)`: $26 \bmod 7 = 5 \rightarrow$ yalnızca kova 5 (12, 19) taranır, bulunamaz $\rightarrow$ `false`. Diğer kovalara hiç bakılmaz.

Kod İzleme: add / contains (m = 7)

```
1 // alanlar: List<Integer>[] kova; n; m = 7
2 boolean add(int x) {           // x ≥ 0
3     ▶ int i = x % m;           // kova no
4     for (int y : kova[i])
5         if (y == x) return false; // zaten var
6     kova[i].add(x);
7     n++;
8     return true;
9 }
10 boolean contains(int x) {
11     int i = x % m;
12     for (int y : kova[i])
13         if (y == x) return true;
14     return false;
15 }
```



add(12) : kova no = $12 \% 7 = 5$. Kova boş;
tarama hemen biter.

Adım 1 / 18

ÇAĞRI YIĞINI

add(12)

main()

DEĞİŞKENLER

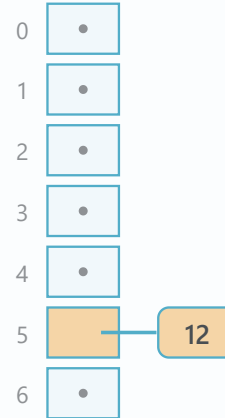
x = 12

i = 5

n = 0

Kod İzleme: add / contains (m = 7)

```
1 // alanlar: List<Integer>[] kova; n; m = 7
2 boolean add(int x) {           // x ≥ 0
3     int i = x % m;             // kova no
4     for (int y : kova[i])
5         if (y == x) return false; // zaten var
6     ▶ kova[i].add(x);
7     ▶ n++;
8     ▶ return true;
9 }
10 boolean contains(int x) {
11     int i = x % m;
12     for (int y : kova[i])
13         if (y == x) return true;
14     return false;
15 }
```



12 kovada yok → zincirin sonuna eklenir, **n = 1**,
return true.

Adım 2 / 18

ÇAĞRI YIĞINI

add(12)

main()

DEĞİŞKENLER

x = 12

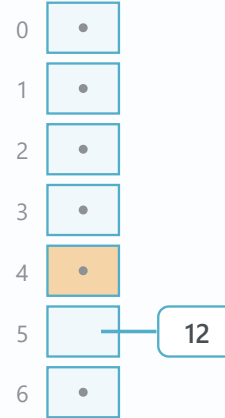
i = 5

n = 1

dönüş = true

Kod İzleme: add / contains (m = 7)

```
1 // alanlar: List<Integer>[] kova; n; m = 7
2 boolean add(int x) {           // x ≥ 0
3     ▶ int i = x % m;           // kova no
4     for (int y : kova[i])
5         if (y == x) return false; // zaten var
6     kova[i].add(x);
7     n++;
8     return true;
9 }
10 boolean contains(int x) {
11     int i = x % m;
12     for (int y : kova[i])
13         if (y == x) return true;
14     return false;
15 }
```



add(25) : kova no = $25 \% 7 = 4$. Kova boş; tarama hemen biter.

Adım 3 / 18

ÇAĞRI YİĞİNİ

add(25)

main()

DEĞİŞKENLER

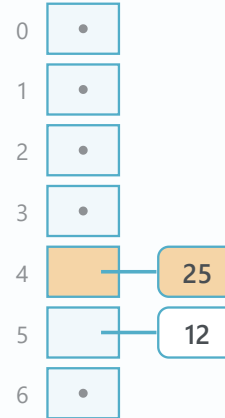
x = 25

i = 4

n = 1

Kod İzleme: add / contains (m = 7)

```
1 // alanlar: List<Integer>[] kova; n; m = 7
2 boolean add(int x) {           // x ≥ 0
3     int i = x % m;             // kova no
4     for (int y : kova[i])
5         if (y == x) return false; // zaten var
6     ▶ kova[i].add(x);
7     ▶ n++;
8     ▶ return true;
9 }
10 boolean contains(int x) {
11     int i = x % m;
12     for (int y : kova[i])
13         if (y == x) return true;
14     return false;
15 }
```



25 kovada yok → zincirin sonuna eklenir, **n = 2**,
return true.

Adım 4 / 18

ÇAĞRI YIĞINI

add(25)

main()

DEĞİŞKENLER

x = 25

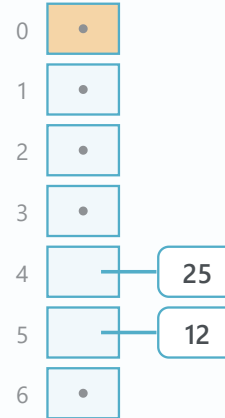
i = 4

n = 2

dönüş = true

Kod İzleme: add / contains (m = 7)

```
1 // alanlar: List<Integer>[] kova; n; m = 7
2 boolean add(int x) {           // x ≥ 0
3     ▶ int i = x % m;           // kova no
4     for (int y : kova[i])
5         if (y == x) return false; // zaten var
6     kova[i].add(x);
7     n++;
8     return true;
9 }
10 boolean contains(int x) {
11     int i = x % m;
12     for (int y : kova[i])
13         if (y == x) return true;
14     return false;
15 }
```



add(7) : $kova\ no = 7 \% 7 = 0$. Kova boş; tarama hemen biter.

Adım 5 / 18

ÇAĞRI YIĞINI

add(7)

main()

DEĞİŞKENLER

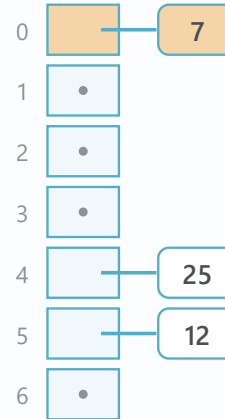
x = 7

i = 0

n = 2

Kod İzleme: add / contains (m = 7)

```
1 // alanlar: List<Integer>[] kova; n; m = 7
2 boolean add(int x) {           // x ≥ 0
3     int i = x % m;             // kova no
4     for (int y : kova[i])
5         if (y == x) return false; // zaten var
6     ▶ kova[i].add(x);
7     ▶ n++;
8     ▶ return true;
9 }
10 boolean contains(int x) {
11     int i = x % m;
12     for (int y : kova[i])
13         if (y == x) return true;
14     return false;
15 }
```



7 kovada yok → zincirin sonuna eklenir, **n = 3**,
return true.

Adım 6 / 18

ÇAĞRI YIĞINI

add(7)

main()

DEĞİŞKENLER

x = 7

i = 0

n = 3

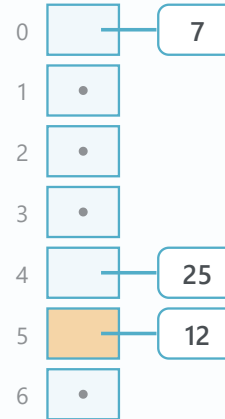
dönüş = true

Kod İzleme: add / contains (m = 7)

```

1 // alanlar: List<Integer>[] kova; n; m = 7
2 boolean add(int x) {           // x ≥ 0
3     ▶ int i = x % m;           // kova no
4     for (int y : kova[i])
5         if (y == x) return false; // zaten var
6     kova[i].add(x);
7     n++;
8     return true;
9 }
10 boolean contains(int x) {
11     int i = x % m;
12     for (int y : kova[i])
13         if (y == x) return true;
14     return false;
15 }

```



add(19) : kova no = $19 \% 7 = 5$. Kovadaki zincir taranmalı: 12.

Adım 7 / 18

ÇAĞRI YİĞİNİ

add(19)

main()

DEĞİŞKENLER

x = 19

i = 5

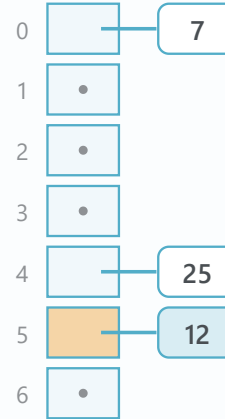
n = 3

Kod İzleme: add / contains (m = 7)

```

1 // alanlar: List<Integer>[] kova; n; m = 7
2 boolean add(int x) {           // x ≥ 0
3     int i = x % m;             // kova no
4     for (int y : kova[i])
5         if (y == x) return false; // zaten var
6     kova[i].add(x);
7     n++;
8     return true;
9 }
10 boolean contains(int x) {
11     int i = x % m;
12     for (int y : kova[i])
13         if (y == x) return true;
14     return false;
15 }

```



$y = 12 \neq 19 \rightarrow$ zincirde ilerle.

Adım 8 / 18

ÇAĞRI YIĞINI

add(19)

main()

DEĞİŞKENLER

x = 19

i = 5

n = 3

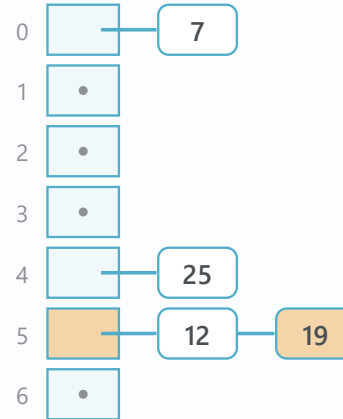
y = 12

Kod İzleme: add / contains (m = 7)

```

1 // alanlar: List<Integer>[] kova; n; m = 7
2 boolean add(int x) {           // x ≥ 0
3     int i = x % m;             // kova no
4     for (int y : kova[i])
5         if (y == x) return false; // zaten var
6     ▶ kova[i].add(x);
7     ▶ n++;
8     ▶ return true;
9 }
10 boolean contains(int x) {
11     int i = x % m;
12     for (int y : kova[i])
13         if (y == x) return true;
14     return false;
15 }

```



19 kovada yok → zincirin sonuna eklenir, **n = 4**,
return true . (Çakışma: kova 5'te artık 2 öge var.)

ÇAĞRI YIĞINI

add(19)

main()

Adım 9 / 18

DEĞİŞKENLER

x = 19

i = 5

n = 4

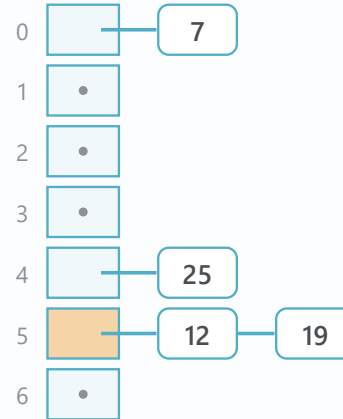
dönüş = true

Kod İzleme: add / contains (m = 7)

```

1 // alanlar: List<Integer>[] kova; n; m = 7
2 boolean add(int x) {           // x ≥ 0
3     ▶ int i = x % m;           // kova no
4     for (int y : kova[i])
5         if (y == x) return false; // zaten var
6     kova[i].add(x);
7     n++;
8     return true;
9 }
10 boolean contains(int x) {
11     int i = x % m;
12     for (int y : kova[i])
13         if (y == x) return true;
14     return false;
15 }

```



add(12) : kova no = $12 \% 7 = 5$. Kovadaki zincir taranmalı: 12, 19.

Adım 10 / 18

ÇAĞRI YIĞINI

add(12)

main()

DEĞİŞKENLER

x = 12

i = 5

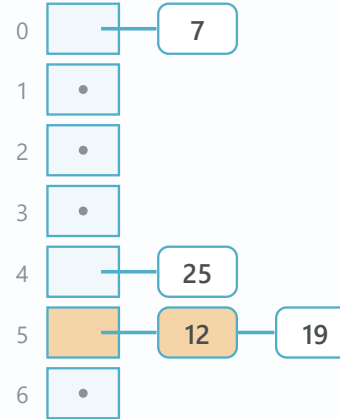
n = 4

Kod İzleme: add / contains (m = 7)

```

1 // alanlar: List<Integer>[] kova; n; m = 7
2 boolean add(int x) {           // x ≥ 0
3     int i = x % m;             // kova no
4     for (int y : kova[i])
5         if (y == x) return false; // zaten var
6     kova[i].add(x);
7     n++;
8     return true;
9 }
10 boolean contains(int x) {
11     int i = x % m;
12     for (int y : kova[i])
13         if (y == x) return true;
14     return false;
15 }

```



$y = 12 == 12 \rightarrow$ öge **zaten kümede**: `return false`, hiçbir şey eklenmez. Kümeyi küme yapan budur.

ÇAĞRI YIĞINI

add(12)

main()

Adım 11 / 18

DEĞİŞKENLER

x = 12

i = 5

n = 4

y = 12

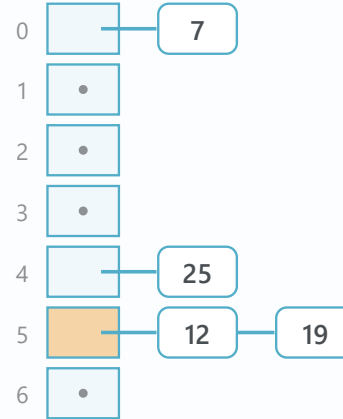
dönüş = false

Kod İzleme: add / contains (m = 7)

```

1 // alanlar: List<Integer>[] kova; n; m = 7
2 boolean add(int x) {           // x ≥ 0
3     int i = x % m;             // kova no
4     for (int y : kova[i])
5         if (y == x) return false; // zaten var
6     kova[i].add(x);
7     n++;
8     return true;
9 }
10 boolean contains(int x) {
11     ▶ int i = x % m;
12     for (int y : kova[i])
13         if (y == x) return true;
14     return false;
15 }

```



contains(26) : $26 \% 7 = 5 \rightarrow$ yalnızca kova 5 taranır; diğer kovalara hiç bakılmaz.

Adım 12 / 18

ÇAĞRI YIĞINI

contains(26)

main()

DEĞİŞKENLER

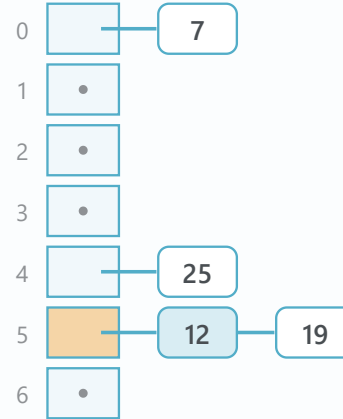
x = 26

i = 5

n = 4

Kod İzleme: add / contains (m = 7)

```
1 // alanlar: List<Integer>[] kova; n; m = 7
2 boolean add(int x) {           // x ≥ 0
3     int i = x % m;             // kova no
4     for (int y : kova[i])
5         if (y == x) return false; // zaten var
6     kova[i].add(x);
7     n++;
8     return true;
9 }
10 boolean contains(int x) {
11     int i = x % m;
12     for (int y : kova[i])
13         if (y == x) return true;
14     return false;
15 }
```



$y = 12 \neq 26 \rightarrow$ devam.

Adım 13 / 18

ÇAĞRI YIĞINI

contains(26)

main()

DEĞİŞKENLER

x = 26

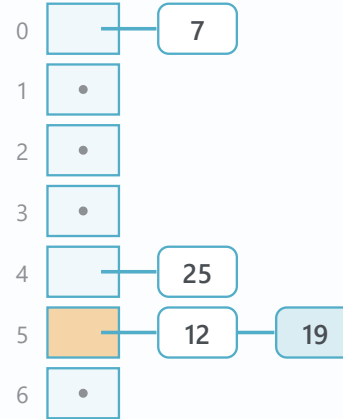
i = 5

n = 4

y = 12

Kod İzleme: add / contains (m = 7)

```
1 // alanlar: List<Integer>[] kova; n; m = 7
2 boolean add(int x) {           // x ≥ 0
3     int i = x % m;             // kova no
4     for (int y : kova[i])
5         if (y == x) return false; // zaten var
6     kova[i].add(x);
7     n++;
8     return true;
9 }
10 boolean contains(int x) {
11     int i = x % m;
12     for (int y : kova[i])
13         if (y == x) return true;
14     return false;
15 }
```



$y = 19 \neq 26 \rightarrow$ devam.

Adım 14 / 18

ÇAĞRI YIĞINI

contains(26)

main()

DEĞİŞKENLER

x = 26

i = 5

n = 4

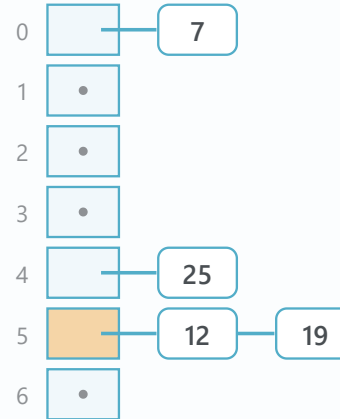
y = 19

Kod İzleme: add / contains (m = 7)

```

1 // alanlar: List<Integer>[] kova; n; m = 7
2 boolean add(int x) {           // x ≥ 0
3     int i = x % m;             // kova no
4     for (int y : kova[i])
5         if (y == x) return false; // zaten var
6     kova[i].add(x);
7     n++;
8     return true;
9 }
10 boolean contains(int x) {
11     int i = x % m;
12     for (int y : kova[i])
13         if (y == x) return true;
14     return false;
15 }

```



Zincir bitti, 26 yok → **return false**. Ortalama zincir boyu n/m sabit → **O(1)**.

Adım 15 / 18

ÇAĞRI YIĞINI

contains(26)

main()

DEĞİŞKENLER

x = 26

i = 5

n = 4

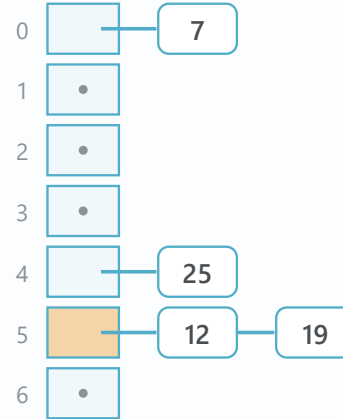
dönüş = false

Kod İzleme: add / contains (m = 7)

```

1 // alanlar: List<Integer>[] kova; n; m = 7
2 boolean add(int x) {           // x ≥ 0
3     int i = x % m;             // kova no
4     for (int y : kova[i])
5         if (y == x) return false; // zaten var
6     kova[i].add(x);
7     n++;
8     return true;
9 }
10 boolean contains(int x) {
11     ▶ int i = x % m;
12     for (int y : kova[i])
13         if (y == x) return true;
14     return false;
15 }

```



contains(19) : $19 \% 7 = 5 \rightarrow$ yalnızca kova 5 taranır; diğer kovalara hiç bakılmaz.

Adım 16 / 18

ÇAĞRI YİĞİNİ

contains(19)

main()

DEĞİŞKENLER

x = 19

i = 5

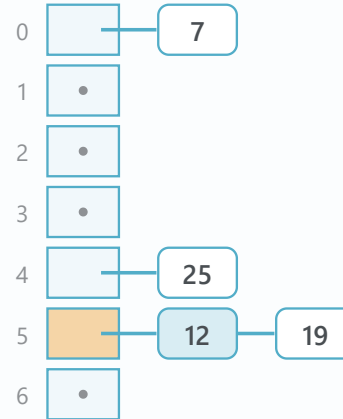
n = 4

Kod İzleme: add / contains (m = 7)

```

1 // alanlar: List<Integer>[] kova; n; m = 7
2 boolean add(int x) {           // x ≥ 0
3     int i = x % m;             // kova no
4     for (int y : kova[i])
5         if (y == x) return false; // zaten var
6     kova[i].add(x);
7     n++;
8     return true;
9 }
10 boolean contains(int x) {
11     int i = x % m;
12     for (int y : kova[i])
13         if (y == x) return true;
14     return false;
15 }

```



$y = 12 \neq 19 \rightarrow$ devam.

Adım 17 / 18

ÇAĞRI YIĞINI

contains(19)

main()

DEĞİŞKENLER

x = 19

i = 5

n = 4

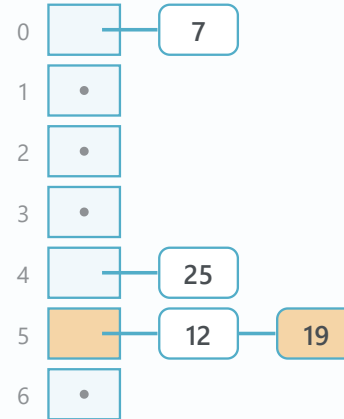
y = 12

Kod İzleme: add / contains (m = 7)

```

1 // alanlar: List<Integer>[] kova; n; m = 7
2 boolean add(int x) {           // x ≥ 0
3     int i = x % m;             // kova no
4     for (int y : kova[i])
5         if (y == x) return false; // zaten var
6     kova[i].add(x);
7     n++;
8     return true;
9 }
10 boolean contains(int x) {
11     int i = x % m;
12     for (int y : kova[i])
13         if (y == x) return true;
14     return false;
15 }

```



$y = 19 == 19 \rightarrow$ bulundu, return true .

Adım 18 / 18

ÇAĞRI YIĞINI

contains(19)

main()

DEĞİŞKENLER

x = 19

i = 5

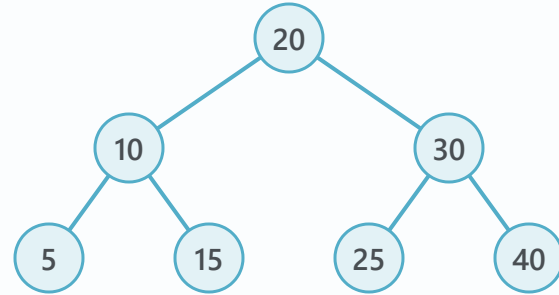
n = 4

y = 19

dönüş = true

Ağaç Tabanlı (Sıralı) Küme

- Küme bir **dengeli ikili arama ağacı** olarak temsil edilir (Java `TreeSet` : kırmızı-siyah ağaç).
- Öğeler **sıralı** saklanır; kök ortada (inorder) gezinme sıralı çıktı verir.
- Arama, ekleme, silme **$O(\log n)$** .
- Öğeler karşılaştırılabilir olmalı (`Comparable` veya `Comparator`).
- `compare(a, b) == 0` ise `TreeSet` iki öğeyi aynı kabul eder; bu nedenle sıralama `equals` ile tutarlı olmalıdır.



Sıralı yapının getirdiği ek işlemler ($S = \{5, 10, 15, 20, 25, 30, 40\}$):

first / last

5 / 40

floor(17)

≤ 17 olan en büyük
15

ceiling(17)

≥ 17 olan en küçük
20

subSet(10, 30)

aralık sorgusu
[10, 15, 20, 25]

Sıralı Dizilerle Birleşim ve Kesişim (iki imleç)

```

1 void birlesimKesisim(int[] a, int[] b,
2   List<Integer> bir, List<Integer> kes) {
3   ▶ int i = 0, j = 0;
4   while (i < a.length && j < b.length) {
5     if (a[i] < b[j]) bir.add(a[i++]);
6     else if (a[i] > b[j]) bir.add(b[j++]);
7     else {                               // ortak öge
8       bir.add(a[i]); kes.add(a[i]);
9       i++; j++;
10    }
11  }
12  while (i < a.length) bir.add(a[i++]);
13  while (j < b.length) bir.add(b[j++]);
14 }

```

a

1	3	5	7	9
---	---	---	---	---

▲ i = 0

b

3	4	5	8
---	---	---	---

▲ j = 0

bir

--	--	--	--	--	--	--	--

kes

--	--	--	--	--	--	--	--

Kümeler **sıralı** dizilerde. Birleştirme sıralamasındaki *merge* gibi iki imleç tek geçişte ilerler (Goodrich: *generic merge*).

Adım 1 / 10

DEĞİŞKENLER

i = 0 j = 0 a[i] = 1 b[j] = 3

Sıralı Dizilerle Birleşim ve Kesişim (iki imleç)

```
1 void birlesimKesisim(int[] a, int[] b,  
2     List<Integer> bir, List<Integer> kes) {  
3     int i = 0, j = 0;  
4     while (i < a.length && j < b.length) {  
5         ▶ if (a[i] < b[j]) bir.add(a[i++]);  
6         else if (a[i] > b[j]) bir.add(b[j++]);  
7         else {                               // ortak öge  
8             bir.add(a[i]); kes.add(a[i]);  
9             i++; j++;  
10        }  
11    }  
12    while (i < a.length) bir.add(a[i++]);  
13    while (j < b.length) bir.add(b[j++]);  
14 }
```

a

1	3	5	7	9
---	---	---	---	---

▲ i = 1

b

3	4	5	8
---	---	---	---

▲ j = 0

bir

1							
---	--	--	--	--	--	--	--

kes

--	--	--	--	--	--	--	--

$a[i] = 1 < b[j] = 3 \rightarrow 1$ yalnızca a'da: **birleşime** eklenir, $i++$.

Adım 2 / 10

DEĞİŞKENLER

i = 0

j = 0

a[i] = 1

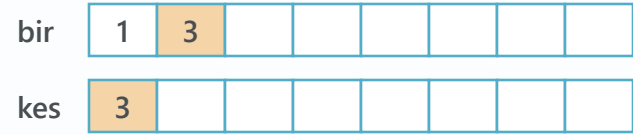
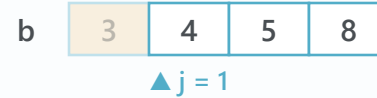
b[j] = 3

Sıralı Dizilerle Birleşim ve Kesişim (iki imleç)

```

1 void birlesimKesisim(int[] a, int[] b,
2   List<Integer> bir, List<Integer> kes) {
3   int i = 0, j = 0;
4   while (i < a.length && j < b.length) {
5     if (a[i] < b[j]) bir.add(a[i++]);
6     else if (a[i] > b[j]) bir.add(b[j++]);
7     else {                               // ortak öge
8       ▶   bir.add(a[i]); kes.add(a[i]);
9       ▶   i++; j++;
10    }
11  }
12  while (i < a.length) bir.add(a[i++]);
13  while (j < b.length) bir.add(b[j++]);
14 }

```



$a[i] = b[j] = 3 \rightarrow$ **ortak öge**: birleşime bir kez ve kesişime eklenir; iki imleç de ilerler.

Adım 3 / 10

DEĞİŞKENLER

i = 1

j = 0

a[i] = 3

b[j] = 3

Sıralı Dizilerle Birleşim ve Kesişim (iki imleç)

```
1 void birlesimKesisim(int[] a, int[] b,  
2     List<Integer> bir, List<Integer> kes) {  
3     int i = 0, j = 0;  
4     while (i < a.length && j < b.length) {  
5         if (a[i] < b[j]) bir.add(a[i++]);  
6         ▶ else if (a[i] > b[j]) bir.add(b[j++]);  
7         else { // ortak öğe  
8             bir.add(a[i]); kes.add(a[i]);  
9             i++; j++;  
10        }  
11    }  
12    while (i < a.length) bir.add(a[i++]);  
13    while (j < b.length) bir.add(b[j++]);  
14 }
```

a

1	3	5	7	9
---	---	---	---	---

▲ i = 2

b

3	4	5	8
---	---	---	---

▲ j = 2

bir

1	3	4					
---	---	---	--	--	--	--	--

kes

3							
---	--	--	--	--	--	--	--

$a[i] = 5 > b[j] = 4 \rightarrow 4$ yalnızca b'de: **birleşime** eklenir, j++.

Adım 4 / 10

DEĞİŞKENLER

i = 2

j = 1

a[i] = 5

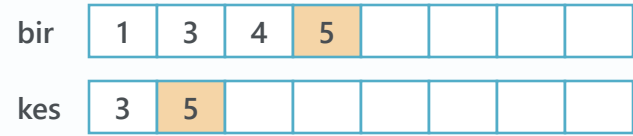
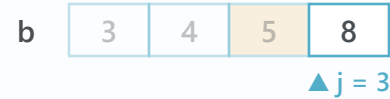
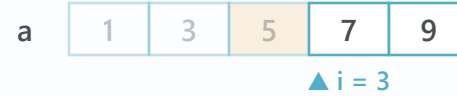
b[j] = 4

Sıralı Dizilerle Birleşim ve Kesişim (iki imleç)

```

1 void birlesimKesisim(int[] a, int[] b,
2   List<Integer> bir, List<Integer> kes) {
3   int i = 0, j = 0;
4   while (i < a.length && j < b.length) {
5     if (a[i] < b[j]) bir.add(a[i++]);
6     else if (a[i] > b[j]) bir.add(b[j++]);
7     else {                               // ortak öge
8       ▶   bir.add(a[i]); kes.add(a[i]);
9       ▶   i++; j++;
10    }
11  }
12  while (i < a.length) bir.add(a[i++]);
13  while (j < b.length) bir.add(b[j++]);
14 }

```



$a[i] = b[j] = 5 \rightarrow$ **ortak öge**: birleşime bir kez ve kesişime eklenir; iki imleç de ilerler.

Adım 5 / 10

DEĞİŞKENLER

i = 2

j = 2

a[i] = 5

b[j] = 5

Sıralı Dizilerle Birleşim ve Kesişim (iki imleç)

```

1 void birlesimKesisim(int[] a, int[] b,
2   List<Integer> bir, List<Integer> kes) {
3   int i = 0, j = 0;
4   while (i < a.length && j < b.length) {
5       ► if (a[i] < b[j]) bir.add(a[i++]);
6       else if (a[i] > b[j]) bir.add(b[j++]);
7       else {                               // ortak öge
8           bir.add(a[i]); kes.add(a[i]);
9           i++; j++;
10      }
11  }
12  while (i < a.length) bir.add(a[i++]);
13  while (j < b.length) bir.add(b[j++]);
14 }

```

a

1	3	5	7	9
---	---	---	---	---

▲ i = 4

b

3	4	5	8
---	---	---	---

▲ j = 3

bir

1	3	4	5	7			
---	---	---	---	---	--	--	--

kes

3	5						
---	---	--	--	--	--	--	--

$a[i] = 7 < b[j] = 8 \rightarrow 7$ yalnızca a'da: **birleşime** eklenir, $i++$.

Adım 6 / 10

DEĞİŞKENLER

i = 3 j = 3 a[i] = 7 b[j] = 8

Sıralı Dizilerle Birleşim ve Kesişim (iki imleç)

```

1 void birlesimKesisim(int[] a, int[] b,
2   List<Integer> bir, List<Integer> kes) {
3   int i = 0, j = 0;
4   while (i < a.length && j < b.length) {
5     if (a[i] < b[j]) bir.add(a[i++]);
6     else if (a[i] > b[j]) bir.add(b[j++]);
7     else { // ortak öge
8       bir.add(a[i]); kes.add(a[i]);
9       i++; j++;
10    }
11  }
12  while (i < a.length) bir.add(a[i++]);
13  while (j < b.length) bir.add(b[j++]);
14 }

```

a

1	3	5	7	9
---	---	---	---	---

▲ i = 4

b

3	4	5	8
---	---	---	---

▲ j = 4 (bitti)

bir

1	3	4	5	7	8		
---	---	---	---	---	---	--	--

kes

3	5						
---	---	--	--	--	--	--	--

$a[i] = 9 > b[j] = 8 \rightarrow 8$ yalnızca b'de: **birleşime** eklenir, j++.

Adım 7 / 10

DEĞİŞKENLER

i = 4

j = 3

a[i] = 9

b[j] = 8

Sıralı Dizilerle Birleşim ve Kesişim (iki imleç)

```

1 void birlesimKesisim(int[] a, int[] b,
2   List<Integer> bir, List<Integer> kes) {
3   int i = 0, j = 0;
4   ► while (i < a.length && j < b.length) {
5       if (a[i] < b[j]) bir.add(a[i++]);
6       else if (a[i] > b[j]) bir.add(b[j++]);
7       else {                               // ortak öge
8           bir.add(a[i]); kes.add(a[i]);
9           i++; j++;
10      }
11  }
12  while (i < a.length) bir.add(a[i++]);
13  while (j < b.length) bir.add(b[j++]);
14 }

```

a

1	3	5	7	9
---	---	---	---	---

▲ i = 4

b

3	4	5	8
---	---	---	---

▲ j = 4 (bitti)

bir

1	3	4	5	7	8		
---	---	---	---	---	---	--	--

kes

3	5						
---	---	--	--	--	--	--	--

j = 4 = b.length → b bitti, ana döngüden çıkılır.
Kesişim tamam: artık ortak öge olamaz.

Adım 8 / 10

DEĞİŞKENLER

i = 4

j = 4

a[i] = 9

Sıralı Dizilerle Birleşim ve Kesişim (iki imleç)

```

1 void birlesimKesisim(int[] a, int[] b,
2   List<Integer> bir, List<Integer> kes) {
3   int i = 0, j = 0;
4   while (i < a.length && j < b.length) {
5       if (a[i] < b[j]) bir.add(a[i++]);
6       else if (a[i] > b[j]) bir.add(b[j++]);
7       else {                               // ortak öge
8           bir.add(a[i]); kes.add(a[i]);
9           i++; j++;
10      }
11  }
12  ► while (i < a.length) bir.add(a[i++]);
13  while (j < b.length) bir.add(b[j++]);
14 }

```

a

1	3	5	7	9
---	---	---	---	---

▲ i = 5 (bitti)

b

3	4	5	8
---	---	---	---

▲ j = 4 (bitti)

bir

1	3	4	5	7	8	9	
---	---	---	---	---	---	---	--

kes

3	5						
---	---	--	--	--	--	--	--

a'da kalan 9 doğrudan birleşime eklenir, i++.

Adım 9 / 10

DEĞİŞKENLER

i = 4

j = 4

a[i] = 9

Sıralı Dizilerle Birleşim ve Kesişim (iki imleç)

```

1 void birlesimKesisim(int[] a, int[] b,
2   List<Integer> bir, List<Integer> kes) {
3   int i = 0, j = 0;
4   while (i < a.length && j < b.length) {
5       if (a[i] < b[j]) bir.add(a[i++]);
6       else if (a[i] > b[j]) bir.add(b[j++]);
7       else {                               // ortak öge
8           bir.add(a[i]); kes.add(a[i]);
9           i++; j++;
10      }
11  }
12  while (i < a.length) bir.add(a[i++]);
13  while (j < b.length) bir.add(b[j++]);
14 }

```

DEĞİŞKENLER

i = 5

j = 4

a

1	3	5	7	9
---	---	---	---	---

 ▲ i = 5 (bitti)

b

3	4	5	8
---	---	---	---

 ▲ j = 4 (bitti)

bir

1	3	4	5	7	8	9	
---	---	---	---	---	---	---	--

kes

3	5						
---	---	--	--	--	--	--	--

b'de kalan yok. Birleşim [1, 3, 4, 5, 7, 8, 9], kesişim [3, 5]. Her adımda bir imleç ilerler → en fazla $n + m$ adım: **$O(n + m)$** .

Adım 10 / 10

Küme Uygulamalarının Karşılaştırması

İşlem ($ A = n, B = m$)	Sırasız liste	Sıralı dizi	Hash tablosu	Dengeli BST
<code>contains</code>	$O(n)$	$O(\log n)$	$O(1)^*$	$O(\log n)$
<code>add</code> / <code>remove</code>	$O(n)^+$	$O(n)$	$O(1)^*$	$O(\log n)$
$A \cup B, A \cap B, A - B$	$O(n \cdot m)$	$O(n + m)$	$O(n + m)^*$	$O(m \log(n+m))$
Sıralı gezinme	$O(n \log n)$	$O(n)$	$O(n \log n)$	$O(n)$
En küçük / floor / aralık	$O(n)$	$O(\log n)$	$O(n)$	$O(\log n)$

* ortalama durum (en kötü $O(n)$). + tekrar kontrolü için önce arama gerekir. Hash ile birleşim: B'nin her ögesi için `contains` + `add`.

Varsayılan seçim: hash

Sıra gerekmiyorsa en hızlı üyelik testi.

Sıra gerekiyorsa: ağaç

Her işlem $O(\log n)$ ve sıralı sorgular bedava.

Bit Vektörü ile Küme (Bitset)

	7	6	5	4	3	2	1	0	
A	0	0	0	1	1	0	1	0	= 26 → {1, 3, 4}
B	0	0	1	0	1	0	0	0	= 40 → {3, 5}
A B	0	0	1	1	1	0	1	0	= 58 → {1, 3, 4, 5}

Fikir

Evren $0..U-1$ ise i . bit $1 \Leftrightarrow i$ kümede.

Tek öge: $O(1)$

ekle, sil, üyelik = tek bit işlemi.

Küme cebri: $O(U / 64)$

$U \cap$ – kelime kelime | &
&~.

İşlem	int maskesi ($U \leq 32$)	java.util.BitSet
ekle x / var mı x?	<code>a = 1 << x · (a >> x & 1) == 1</code>	<code>set(x) · get(x)</code>
birleşim / kesişim	<code>a b · a & b</code>	<code>or(b) · and(b)</code>
fark / öge sayısı	<code>a & ~b · Integer.bitCount(a)</code>	<code>andNot(b) · cardinality()</code>

Bellek U bit: 10^6 ögelik evren ≈ 122 KB. Evren büyük ama küme seyrekse (ör. 3 öge, $U = 10^9$) hash tabanlı küme seçilir.

Kod İzleme: bit işlemleriyle küme cebri

	7	6	5	4	3	2	1	0	
a	0	0	0	0	0	0	0	0	= {}
b	0	0	0	0	0	0	0	0	= {}

İki boş küme: tüm bitler 0.

Adım 1 / 10

```
► int a = 0, b = 0;
2 for (int x : new int[]{1, 3, 4})
3   a |= 1 << x;           // x. biti yak
4 for (int x : new int[]{3, 5})
5   b |= 1 << x;
6 int bir = a | b;          // birleşim
7 int kes = a & b;          // kesişim
8 int fark = a & ~b;        // fark
9 boolean var3 = (a >> 3 & 1) == 1;
```

DEĞİŞKENLER

a = 0 (00000000)

b = 0 (00000000)

Kod İzleme: bit işlemleriyle küme cebri

	7	6	5	4	3	2	1	0	
a	0	0	0	0	0	0	1	0	= {1}
b	0	0	0	0	0	0	0	0	= {}
1 << x	0	0	0	0	0	0	1	0	maske = 2

`1 << 1` = 2 maskesi; `a |= maske` → 1. bit 1 olur: 1 kümeye eklendi.

Adım 2 / 10

```

1 int a = 0, b = 0;
2 for (int x : new int[]{1, 3, 4})
3     a |= 1 << x;          // x. biti yak
4 for (int x : new int[]{3, 5})
5     b |= 1 << x;
6 int bir = a | b;          // birleşim
7 int kes = a & b;          // kesişim
8 int fark = a & ~b;        // fark
9 boolean var3 = (a >> 3 & 1) == 1;
```

DEĞİŞKENLER

a = 2 (00000010)

b = 0 (00000000)

x = 1

Kod İzleme: bit işlemleriyle küme cebri

	7	6	5	4	3	2	1	0	
a	0	0	0	0	1	0	1	0	= {1, 3}
b	0	0	0	0	0	0	0	0	= {}
$1 \ll x$	0	0	0	0	1	0	0	0	maske = 8

$1 \ll 3$ = 8 maskesi; $a \mid \text{maske}$ → 3. bit 1 olur: 3 kümeye eklendi.

Adım 3 / 10

```

1 int a = 0, b = 0;
2 for (int x : new int[]{1, 3, 4})
3     a |= 1 << x;      // x. biti yak
4 for (int x : new int[]{3, 5})
5     b |= 1 << x;
6 int bir = a | b;      // birleşim
7 int kes = a & b;      // kesişim
8 int fark = a & ~b;    // fark
9 boolean var3 = (a >> 3 & 1) == 1;
```

DEĞİŞKENLER

a = 10 (00001010)

b = 0 (00000000)

x = 3

Kod İzleme: bit işlemleriyle küme cebri

	7	6	5	4	3	2	1	0	
a	0	0	0	1	1	0	1	0	= {1, 3, 4}
b	0	0	0	0	0	0	0	0	= {}
$1 \ll x$	0	0	0	1	0	0	0	0	maske = 16

$1 \ll 4$ = 16 maskesi; $a \mid= \text{maske}$ → 4. bit 1 olur: 4 kümeye eklendi.

Adım 4 / 10

```

1 int a = 0, b = 0;
2 for (int x : new int[]{1, 3, 4})
3     a |= 1 << x;          // x. biti yak
4 for (int x : new int[]{3, 5})
5     b |= 1 << x;
6 int bir = a | b;          // birleşim
7 int kes = a & b;          // kesişim
8 int fark = a & ~b;        // fark
9 boolean var3 = (a >> 3 & 1) == 1;
```

DEĞİŞKENLER

a = 26 (00011010)

b = 0 (00000000)

x = 4

Kod İzleme: bit işlemleriyle küme cebri

	7	6	5	4	3	2	1	0	
a	0	0	0	1	1	0	1	0	= {1, 3, 4}
b	0	0	0	0	1	0	0	0	= {3}
$1 \ll x$	0	0	0	0	1	0	0	0	maske = 8

b kümesine 3 eklenir: 3. bit yanar.

Adım 5 / 10

```

1 int a = 0, b = 0;
2 for (int x : new int[]{1, 3, 4})
3   a |= 1 << x;          // x. biti yak
4 for (int x : new int[]{3, 5})
5   b |= 1 << x;
6 int bir = a | b;        // birleşim
7 int kes = a & b;        // kesişim
8 int fark = a & ~b;      // fark
9 boolean var3 = (a >> 3 & 1) == 1;

```

DEĞİŞKENLER

a = 26 (00011010)

b = 8 (00001000)

x = 3

Kod İzleme: bit işlemleriyle küme cebri

	7	6	5	4	3	2	1	0	
a	0	0	0	1	1	0	1	0	= {1, 3, 4}
b	0	0	1	0	1	0	0	0	= {3, 5}
$1 \ll x$	0	0	1	0	0	0	0	0	maske = 32

b kümesine 5 eklenir: 5. bit yanar.

Adım 6 / 10

```

1 int a = 0, b = 0;
2 for (int x : new int[]{1, 3, 4})
3   a |= 1 << x;           // x. biti yak
4 for (int x : new int[]{3, 5})
5   b |= 1 << x;
6 int bir = a | b;         // birleşim
7 int kes = a & b;         // kesişim
8 int fark = a & ~b;       // fark
9 boolean var3 = (a >> 3 & 1) == 1;
```

DEĞİŞKENLER

a = 26 (00011010)

b = 40 (00101000) x = 5

Kod İzleme: bit işlemleriyle küme cebri

	7	6	5	4	3	2	1	0	
a	0	0	0	1	1	0	1	0	= {1, 3, 4}
b	0	0	1	0	1	0	0	0	= {3, 5}
a b	0	0	1	1	1	0	1	0	= {1, 3, 4, 5}

VEYA: bitlerden biri 1 ise 1 → birleşim {1, 3, 4, 5}. Tek makine komutu.

Adım 7 / 10

```

1 int a = 0, b = 0;
2 for (int x : new int[]{1, 3, 4})
3   a |= 1 << x;          // x. biti yak
4 for (int x : new int[]{3, 5})
5   b |= 1 << x;
6
7 ▶ int bir = a | b;       // birleşim
8 int kes = a & b;         // kesişim
9 int fark = a & ~b;       // fark
10 boolean var3 = (a >> 3 & 1) == 1;
```

DEĞİŞKENLER

a = 26 (00011010)

b = 40 (00101000)

bir = 58 (00111010)

Kod İzleme: bit işlemleriyle küme cebri

	7	6	5	4	3	2	1	0	
a	0	0	0	1	1	0	1	0	= {1, 3, 4}
b	0	0	1	0	1	0	0	0	= {3, 5}
a & b	0	0	0	0	1	0	0	0	= {3}

VE: iki bit de 1 ise 1 → kesişim {3}.

Adım 8 / 10

```

1 int a = 0, b = 0;
2 for (int x : new int[]{1, 3, 4})
3   a |= 1 << x;          // x. biti yak
4 for (int x : new int[]{3, 5})
5   b |= 1 << x;
6 int bir = a | b;        // birleşim
7 int kes = a & b;        // kesişim
8 int fark = a & ~b;      // fark
9 boolean var3 = (a >> 3 & 1) == 1;
```

DEĞİŞKENLER

a = 26 (00011010)

b = 40 (00101000)

bir = 58 (00111010)

kes = 8 (00001000)

Kod İzleme: bit işlemleriyle küme cebri

	7	6	5	4	3	2	1	0	
a	0	0	0	1	1	0	1	0	= {1, 3, 4}
b	0	0	1	0	1	0	0	0	= {3, 5}
a & ~b	0	0	0	1	0	0	1	0	= {1, 4}

`~b` b'nin bitlerini ters çevirir; `a & ~b` : a'da olup b'de olmayanlar → fark {1, 4}.

Adım 9 / 10

```

1 int a = 0, b = 0;
2 for (int x : new int[]{1, 3, 4})
3   a |= 1 << x;          // x. biti yak
4 for (int x : new int[]{3, 5})
5   b |= 1 << x;
6 int bir = a | b;        // birleşim
7 int kes = a & b;        // kesişim
8 int fark = a & ~b;      // fark
9 boolean var3 = (a >> 3 & 1) == 1;
```

DEĞİŞKENLER

a = 26 (00011010)

b = 40 (00101000)

bir = 58 (00111010)

kes = 8 (00001000)

fark = 18 (00010010)

Kod İzleme: bit işlemleriyle küme cebri

	7	6	5	4	3	2	1	0	
a	0	0	0	1	1	0	1	0	= {1, 3, 4}
b	0	0	1	0	1	0	0	0	= {3, 5}
a >> 3	0	0	0	0	0	0	1	1	& 1 → 1

Üyelik testi: `a >> 3` 3. biti en sağa kaydırır, `& 1` onu ayırır → 1, yani **3** ∈ a. O(1).

Adım 10 / 10

```

1 int a = 0, b = 0;
2 for (int x : new int[]{1, 3, 4})
3   a |= 1 << x;           // x. biti yak
4 for (int x : new int[]{3, 5})
5   b |= 1 << x;
6 int bir = a | b;         // birleşim
7 int kes = a & b;         // kesişim
8 int fark = a & ~b;       // fark
▶ boolean var3 = (a >> 3 & 1) == 1;

```

DEĞİŞKENLER

a = 26 (00011010)

b = 40 (00101000)

bir = 58 (00111010)

kes = 8 (00001000)

fark = 18 (00010010)

var3 = true

Çoklu Küme (Multiset / Bag)

- **Çoklu küme** (multiset, *bag*): aynı öğenin **birden çok kez** bulunabildiği küme.
- Sıra yine önemsizdir; önemli olan her öğenin **kaç kez** geçtiğidir (çokluk, multiplicity).
- Doğal uygulama: **öge** → **sayaç** eşlemesi (Bölüm 9).
- Örnek: bir metindeki kelime frekansları, alışveriş sepeti, oy sayımı.

Öge	Sayaç
a	2
b	1
c	3

$M = \{a, a, b, c, c, c\} \rightarrow \text{eşleme } \{a:2, b:1, c:3\}$

Multiset ADT (Goodrich)	Anlamı	Hash ile
<code>add(e)</code> / <code>add(e, n)</code>	e'yi 1 / n kez ekle (sayaç += n)	$O(1)^*$
<code>count(e)</code>	e kaç kez var? (yoksa 0)	$O(1)^*$
<code>remove(e)</code> / <code>remove(e, n)</code>	sayaçı azalt; 0 olursa girdiyi sil	$O(1)^*$
<code>size()</code>	tekrarlar dahil öge sayısı (örnekte 6)	$O(1)$

Java ile Çoklu Küme

```
class CokluKume<E> {  
    private final Map<E, Integer> say = new HashMap<>();  
    private int n = 0; // tekrarlar dahil öge sayısı  
    void add(E e) { say.merge(e, 1, Integer::sum); n++; } // yoksa 1, varsa +1  
    int count(E e) { return say.getOrDefault(e, 0); }  
    boolean remove(E e) {  
        Integer c = say.get(e);  
        if (c == null) return false; // e yok  
        if (c == 1) say.remove(e); // sayaç 0 olacak: girdiyi sil  
        else say.put(e, c - 1);  
        n--;  
        return true;  
    }  
    int size() { return n; }  
}
```

Kullanım

```
add("a"); add("a");  
count("a") → 2
```

Küme ile farkı

Set'te ikinci `add("a")` etkisiz; burada sayaç artar.

Sıralı çoklu küme

`TreeMap<E, Integer>` ile sıralı gezinme.

Kod İzleme: ÇokluKume sayacıları

```

1 void add(E e) {
2     say.merge(e, 1, Integer::sum); n++;
3 }
4 int count(E e) {
5     return say.getDefault(e, 0);
6 }
7 boolean remove(E e) {
8     Integer c = say.get(e);
9     if (c == null) return false;
10    if (c == 1) say.remove(e);
11    else say.put(e, c - 1);
12    n--;
13    return true;
14 }

```

ÇAĞRI YIĞINI

add("a")

main()

say (öge → sayaç)

öge	sayaç
"a"	1

n

1

(tekrar dahil)

M = {a}

merge : "a" yoktu → sayaç 1 ile eklenir; n = 1.

Adım 1 / 13

DEĞİŞKENLER

e = "a"

n = 1

Kod İzleme: ÇokluKume sayacıları

```

1 void add(E e) {
2     say.merge(e, 1, Integer::sum); n++;
3 }
4 int count(E e) {
5     return say.getDefault(e, 0);
6 }
7 boolean remove(E e) {
8     Integer c = say.get(e);
9     if (c == null) return false;
10    if (c == 1) say.remove(e);
11    else say.put(e, c - 1);
12    n--;
13    return true;
14 }

```

ÇAĞRI YIĞINI

add("b")

main()

say (öge → sayaç)

öge	sayaç
"a"	1
"b"	1

n

2

(tekrar dahil)

$M = \{a, b\}$

merge : "b" yoktu → sayaç 1 ile eklenir; $n = 2$.

Adım 2 / 13

DEĞİŞKENLER

e = "b"

n = 2

Kod İzleme: ÇokluKume sayacıları

```

1 void add(E e) {
2     say.merge(e, 1, Integer::sum); n++;
3 }
4 int count(E e) {
5     return say.getDefault(e, 0);
6 }
7 boolean remove(E e) {
8     Integer c = say.get(e);
9     if (c == null) return false;
10    if (c == 1) say.remove(e);
11    else say.put(e, c - 1);
12    n--;
13    return true;
14 }

```

ÇAĞRI YIĞINI

add("a")

main()

say (öge → sayacı)

öge	sayacı
"a"	2
"b"	1

n

3

(tekrar dahil)

M = {a, a, b}

merge : "a" vardı → Integer::sum ile 1 + 1 = 2;
n = 3. Set olsaydı değişmezdi.

Adım 3 / 13

DEĞİŞKENLER

e = "a"

n = 3

Kod İzleme: ÇokluKume sayacıları

```

1 void add(E e) {
2     say.merge(e, 1, Integer::sum); n++;
3 }
4 int count(E e) {
5     return say.getDefault(e, 0);
6 }
7 boolean remove(E e) {
8     Integer c = say.get(e);
9     if (c == null) return false;
10    if (c == 1) say.remove(e);
11    else say.put(e, c - 1);
12    n--;
13    return true;
14 }

```

ÇAĞRI YIĞINI

add("c")

main()

say (öge → sayaç)

öge	sayaç
"a"	2
"b"	1
"c"	1

$M = \{a, a, b, c\}$

merge : "c" yoktu → sayaç 1 ile eklenir; $n = 4$.

n

4

(tekrar dahil)

Adım 4 / 13

DEĞİŞKENLER

$e = \text{"c"}$

$n = 4$

Kod İzleme: ÇokluKume sayacıları

```

1 void add(E e) {
2   say.merge(e, 1, Integer::sum); n++;
3 }
4 int count(E e) {
5   ▶ return say.getDefault(e, 0);
6 }
7 boolean remove(E e) {
8   Integer c = say.get(e);
9   if (c == null) return false;
10  if (c == 1) say.remove(e);
11  else say.put(e, c - 1);
12  n--;
13  return true;
14 }

```

ÇAĞRI YIĞINI

count("a")

main()

say (öge → sayaç)

öge	sayaç
"a"	2
"b"	1
"c"	1

$M = \{a, a, b, c\}$

count("a"): `getDefault` → 2. Tek hash araması: $O(1)$.

Adım 5 / 13

DEĞİŞKENLER

e = "a"

dönüş = 2

n

4

(tekrar dahil)

Kod İzleme: ÇokluKume sayacıları

```

1 void add(E e) {
2     say.merge(e, 1, Integer::sum);  n++;
3 }
4 int count(E e) {
5     return say.getDefault(e, 0);
6 }
7 boolean remove(E e) {
8     ▶ Integer c = say.get(e);
9     if (c == null) return false;
10    if (c == 1) say.remove(e);
11    else say.put(e, c - 1);
12    n--;
13    return true;
14 }

```

ÇAĞRI YIĞINI

```
remove("a")
```

```
main()
```

say (öge → sayacı)

öge	sayacı
"a"	2
"b"	1
"c"	1

n

4

(tekrar dahil)

M = {a, a, b, c}

remove("a"): `c = say.get(e)` = 2.

Adım 6 / 13

DEĞİŞKENLER

e = "a"

c = 2

n = 4

Kod İzleme: ÇokluKume sayacıları

```

1 void add(E e) {
2     say.merge(e, 1, Integer::sum); n++;
3 }
4 int count(E e) {
5     return say.getDefault(e, 0);
6 }
7 boolean remove(E e) {
8     Integer c = say.get(e);
9     if (c == null) return false;
10    if (c == 1) say.remove(e);
11    ▶ else say.put(e, c - 1);
12    ▶ n--;
13    ▶ return true;
14 }

```

ÇAĞRI YIĞINI

```
remove("a")
```

```
main()
```

say (öge → sayaç)

öge	sayaç
"a"	1
"b"	1
"c"	1

$M = \{a, b, c\}$

$c = 2 > 1 \rightarrow$ sayaç 1'e iner (girdi kalır); $n = 3$.

n

3

(tekrar dahil)

Adım 7 / 13

DEĞİŞKENLER

e = "a"

c = 2

n = 3

dönüş = true

Kod İzleme: ÇokluKume sayacıları

```

1 void add(E e) {
2     say.merge(e, 1, Integer::sum); n++;
3 }
4 int count(E e) {
5     return say.getDefault(e, 0);
6 }
7 boolean remove(E e) {
8     Integer c = say.get(e);
9     if (c == null) return false;
10    if (c == 1) say.remove(e);
11    else say.put(e, c - 1);
12    n--;
13    return true;
14 }

```

ÇAĞRI YIĞINI

```
remove("b")
```

```
main()
```

say (öge → sayacı)

öge	sayacı
"a"	1
"b"	1
"c"	1

$M = \{a, b, c\}$

remove("b"): $c = \text{say.get}(e) = 1.$

Adım 8 / 13

DEĞİŞKENLER

$e = \text{"b"}$

$c = 1$

$n = 3$

n

3

(tekrar dahil)

Kod İzleme: ÇokluKume sayacıları

```

1 void add(E e) {
2     say.merge(e, 1, Integer::sum); n++;
3 }
4 int count(E e) {
5     return say.getDefault(e, 0);
6 }
7 boolean remove(E e) {
8     Integer c = say.get(e);
9     if (c == null) return false;
10    ▶ if (c == 1) say.remove(e);
11    else say.put(e, c - 1);
12    ▶ n--;
13    ▶ return true;
14 }

```

ÇAĞRI YIĞINI

```
remove("b")
```

```
main()
```

say (öge → sayaç)

öge	sayaç
"a"	1
"c"	1
"b"	0 (silindi)

n

2

(tekrar dahil)

M = {a, c}

c == 1 → sayaç 0 olacaktı: girdi **tamamen silinir** (0 sayaç saklanmaz); n = 2.

Adım 9 / 13

DEĞİŞKENLER

e = "b"

c = 1

n = 2

dönüş = true

Kod İzleme: ÇokluKume sayacıları

```

1 void add(E e) {
2     say.merge(e, 1, Integer::sum);  n++;
3 }
4 int count(E e) {
5     return say.getDefault(e, 0);
6 }
7 boolean remove(E e) {
8     ▶ Integer c = say.get(e);
9     if (c == null) return false;
10    if (c == 1) say.remove(e);
11    else say.put(e, c - 1);
12    n--;
13    return true;
14 }

```

ÇAĞRI YIĞINI

```
remove("x")
```

```
main()
```

say (öge → sayacı)

öge	sayacı
"a"	1
"c"	1

"x" eşlemede yok

n
2
(tekrar dahil)

M = {a, c}

remove("x"): c = say.get(e) = null.

Adım 10 / 13

DEĞİŞKENLER

e = "x"

c = null

n = 2

Kod İzleme: ÇokluKume sayacıları

```

1 void add(E e) {
2   say.merge(e, 1, Integer::sum);  n++;
3 }
4 int count(E e) {
5   return say.getDefault(e, 0);
6 }
7 boolean remove(E e) {
8   Integer c = say.get(e);
9   ► if (c == null) return false;
10  if (c == 1) say.remove(e);
11  else say.put(e, c - 1);
12  n--;
13  return true;
14 }

```

say (öge → sayacı)

öge	sayacı
"a"	1
"c"	1

"x" eşlemede yok

n
2
(tekrar dahil)

M = {a, c}

c == null → öge yok: **return false**, n değişmez.

Adım 11 / 13

ÇAĞRI YIĞINI

remove("x")

main()

DEĞİŞKENLER

e = "x"

c = null

n = 2

dönüş = false

Kod İzleme: ÇokluKume sayacıları

```

1 void add(E e) {
2     say.merge(e, 1, Integer::sum); n++;
3 }
4 int count(E e) {
5     ▶ return say.getDefault(e, 0);
6 }
7 boolean remove(E e) {
8     Integer c = say.get(e);
9     if (c == null) return false;
10    if (c == 1) say.remove(e);
11    else say.put(e, c - 1);
12    n--;
13    return true;
14 }

```

ÇAĞRI YIĞINI

count("b")

main()

say (öge → sayacı)

öge	sayacı
"a"	1
"c"	1

"b" eşlemede yok

n
2
(tekrar dahil)

M = {a, c}

count("b"): `getDefault` → 0. Tek hash araması: O(1).

Adım 12 / 13

DEĞİŞKENLER

e = "b"

dönüş = 0

Kod İzleme: ÇokluKume sayacıları

```
1 void add(E e) {  
2     say.merge(e, 1, Integer::sum); n++;  
3 }  
4 int count(E e) {  
5     return say.getDefault(e, 0);  
6 }  
7 boolean remove(E e) {  
8     Integer c = say.get(e);  
9     if (c == null) return false;  
10    if (c == 1) say.remove(e);  
11    else say.put(e, c - 1);  
12    n--;  
13    return true;  
14 }
```

ÇAĞRI YIĞINI

main()

say (öge → sayacı)

öge	sayacı
"a"	1
"c"	1

n

2

(tekrar dahil)

 $M = \{a, c\}$

Sonuç $M = \{a, c\}$, $n = 2$. Tüm işlemler tek hash erişimi
→ ortalama **$O(1)$** .

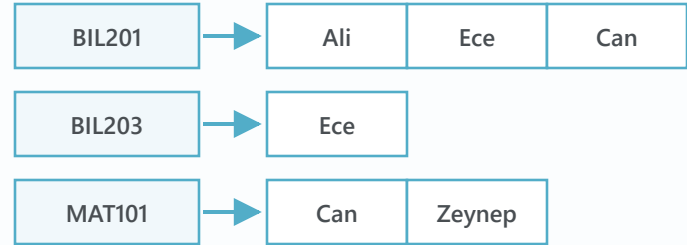
Adım 13 / 13

DEĞİŞKENLER

n = 2

Çoklu Eşleme (Multimap)

- **Çoklu eşleme** (multimap): bir anahtar **birden çok değere** eşlenebilir (ders → öğrenciler).
- Goodrich ADT: `get(k)` tüm değerleri, `put(k, v)`, `remove(k, v)`, `removeAll(k)`, `size()` tüm (k, v) çiftleri.
- Java'da hazır sınıf yok: `Map<K, List<V>>` ile kurulur.



```

Map<String, List<String>> m = new HashMap<>();
void put(String k, String v) { // yoksa boş liste aç
    m.computeIfAbsent(k, x -> new ArrayList<>()).add(v);
}
List<String> get(String k) {
    return m.getOrDefault(k, List.of());
}
  
```

Çoklu kümeden farkı: yalnız kaç kez değil, değerlerin kendisi tutulur. Hash ile `put` / `get` $O(1)^*$, `remove(k, v)` $O(1 + \text{liste boyu})$.

Ayrık Kümeler ve Union-Find

- **Ayrık kümeler** (disjoint sets): hiçbir ögesi ortak olmayan kümeler topluluğu. Her öge tam olarak **bir** kümeye aittir.
- **Union-Find** (Disjoint Set Union, DSU) bu kümeleri tutan veri yapısıdır. İki ana işlemi vardır:

find(x) · bul

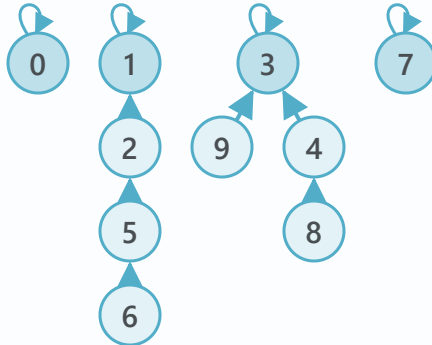
x'in ait olduğu kümenin *temsilsisini* (kökünü) döndürür.

$\text{find}(x) == \text{find}(y) \Leftrightarrow \text{aynı küme}$

union(x, y) · birleştir

x'in ve y'nin kümelerini tek kümede birleştirir.

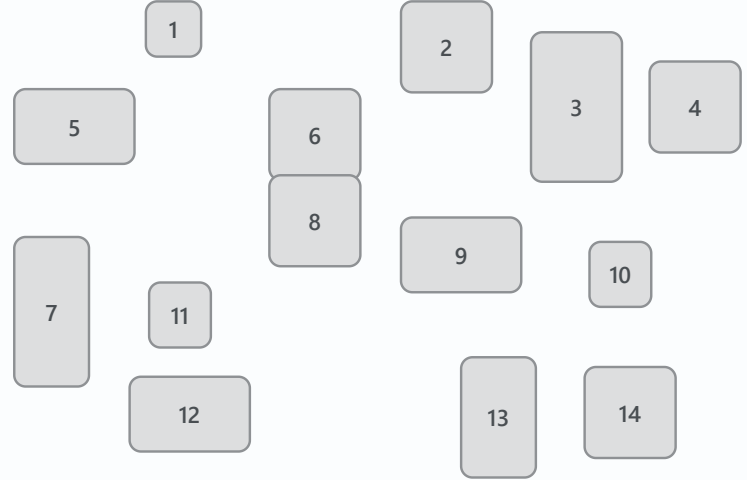
$\{1, 2\} \cup \{3\} \rightarrow \{1, 2, 3\}$



Her küme bir ağaç; her düğüm ebeveynini (parent) gösterir; kök kendini gösterir ve kümenin temsilsisidir.

Mıknatıs Örneği

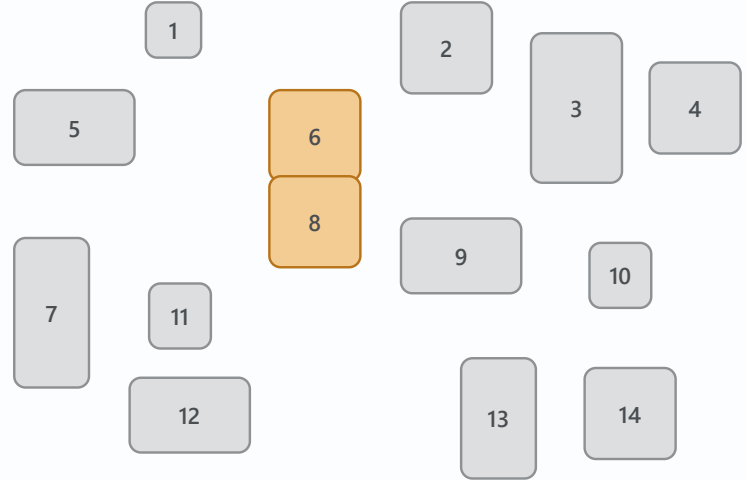
- Birbirini güçlü şekilde çeken mıknatıslar düşünelim: iki mıknatıs değince **ayrılmaz**.
- Birleşmeler sürerken "hangi mıknatıs hangi grupta?" sorusunu izlemek istiyoruz.
- Önce her mıknatısa bir **numara** verilir.
- Birleşme = **union**, "aynı grupta mı?" = iki **find** karşılaştırması.



14 mıknatıs, her biri kendi grubunda: 14 ayrı küme.
 $\text{find}(x) = x$.

Mıknatıs Örneği

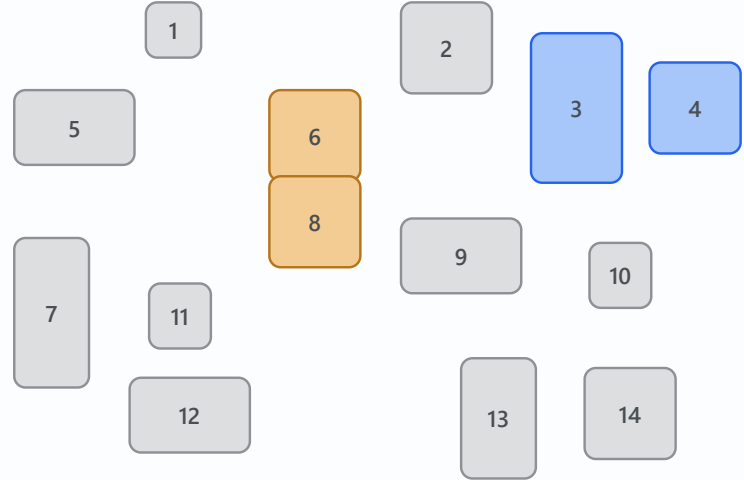
- Birbirini güçlü şekilde çeken mıknatıslar düşünelim: iki mıknatıs değince **ayrılmaz**.
- Birleşmeler sürerken "hangi mıknatıs hangi grupta?" sorusunu izlemek istiyoruz.
- Önce her mıknatısa bir **numara** verilir.
- Birleşme = `union`, "aynı grupta mı?" = iki `find` karşılaştırması.



union(6, 8): en yakın olan 6 ile 8 birleşir; artık `find(6) = find(8)`.

Mıknatıs Örneği

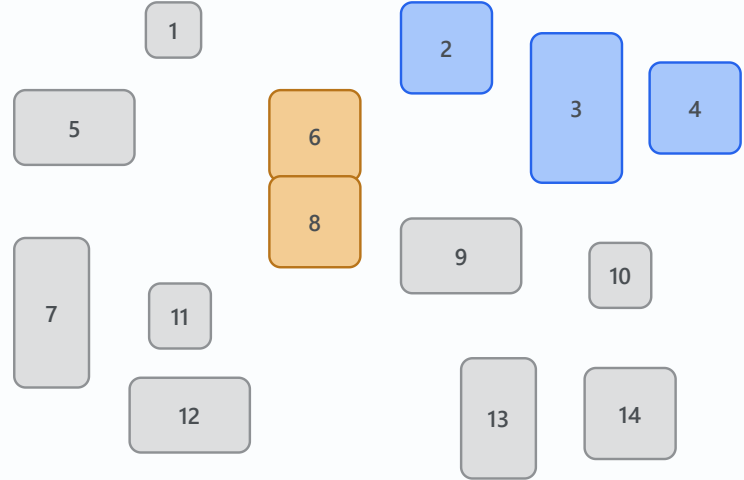
- Birbirini güçlü şekilde çeken mıknatıslar düşünelim: iki mıknatıs değince **ayrılmaz**.
- Birleşmeler sürerken "hangi mıknatıs hangi grupta?" sorusunu izlemek istiyoruz.
- Önce her mıknatısa bir **numara** verilir.
- Birleşme = `union`, "aynı grupta mı?" = iki `find` karşılaştırması.



union(3, 4): 3 ile 4 birbirini çeker, yeni grup {3, 4}.

Mıknatıs Örneği

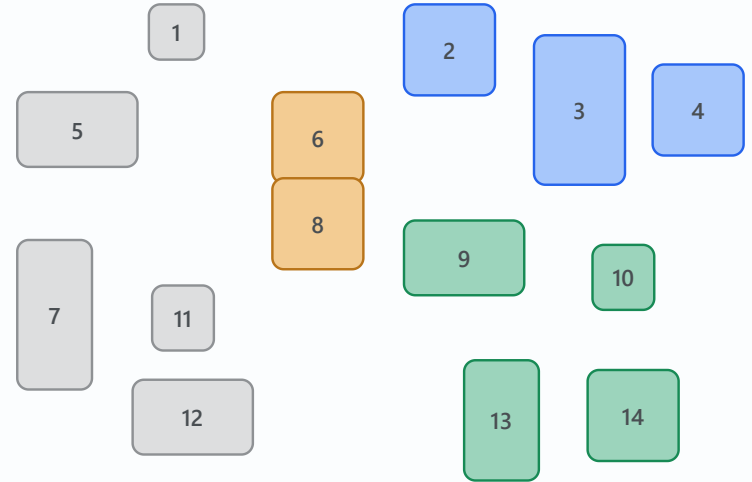
- Birbirini güçlü şekilde çeken mıknatıslar düşünelim: iki mıknatıs değince **ayrılmaz**.
- Birleşmeler sürerken "hangi mıknatıs hangi grupta?" sorusunu izlemek istiyoruz.
- Önce her mıknatısa bir **numara** verilir.
- Birleşme = `union`, "aynı grupta mı?" = iki `find` karşılaştırması.



union(2, 3): 2, {3, 4} grubuna katılır → {2, 3, 4}.

Mıknatıs Örneği

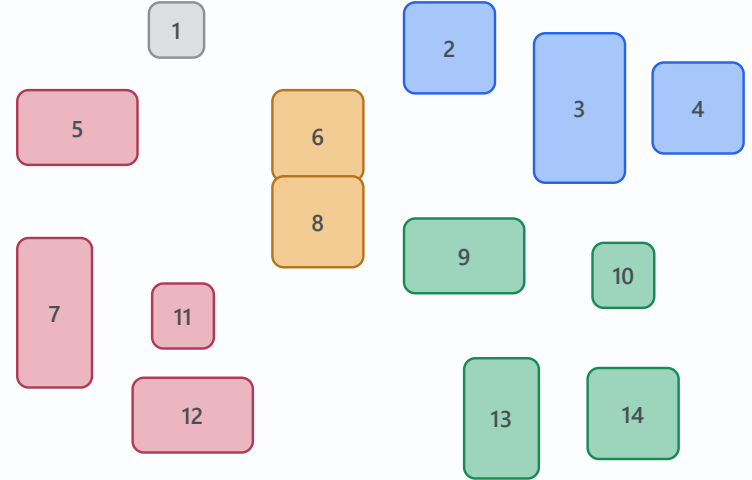
- Birbirini güçlü şekilde çeken mıknatıslar düşünelim: iki mıknatıs değince **ayrılmaz**.
- Birleşmeler sürerken "hangi mıknatıs hangi grupta?" sorusunu izlemek istiyoruz.
- Önce her mıknatısa bir **numara** verilir.
- Birleşme = `union`, "aynı grupta mı?" = iki `find` karşılaştırması.



`union(9, 10), union(13, 14), union(9, 13) → {9, 10, 13, 14}.`

Mıknatıs Örneği

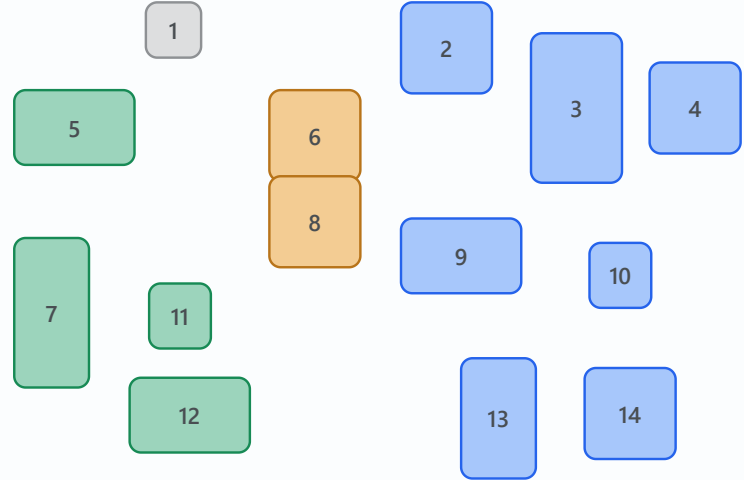
- Birbirini güçlü şekilde çeken mıknatıslar düşünelim: iki mıknatıs değince **ayrılmaz**.
- Birleşmeler sürerken "hangi mıknatıs hangi grupta?" sorusunu izlemek istiyoruz.
- Önce her mıknatısa bir **numara** verilir.
- Birleşme = `union`, "aynı grupta mı?" = iki `find` karşılaştırması.



`union(5, 7)`, `union(7, 11)`, `union(11, 12)` → {5, 7, 11, 12}. Beş grup kaldı (1 tek başına).

Mıknatıs Örneği

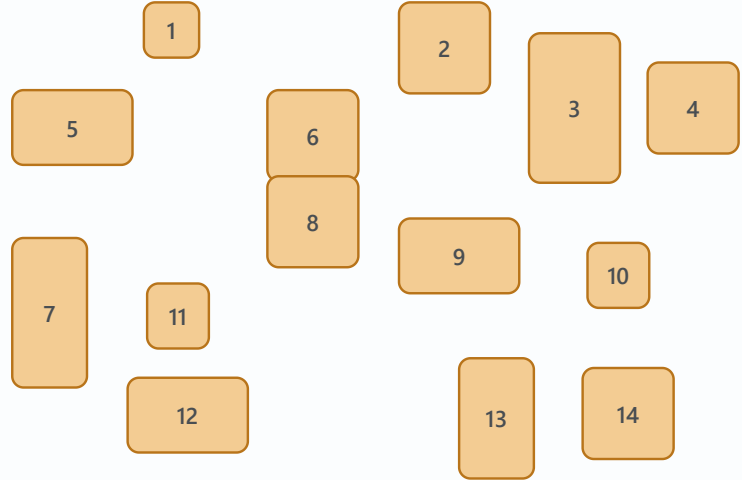
- Birbirini güçlü şekilde çeken mıknatıslar düşünelim: iki mıknatıs değince **ayrılmaz**.
- Birleşmeler sürerken "hangi mıknatıs hangi grupta?" sorusunu izlemek istiyoruz.
- Önce her mıknatısa bir **numara** verilir.
- Birleşme = `union`, "aynı grupta mı?" = iki `find` karşılaştırması.



4 ile 10 aynı grupta mı? $\text{find}(4) \neq \text{find}(10) \rightarrow$ hayır.
`union(4, 10)` iki grubu birleştirir.

Mıknatıs Örneği

- Birbirini güçlü şekilde çeken mıknatıslar düşünelim: iki mıknatıs değince **ayrılmaz**.
- Birleşmeler sürerken "hangi mıknatıs hangi grupta?" sorusunu izlemek istiyoruz.
- Önce her mıknatısa bir **numara** verilir.
- Birleşme = `union`, "aynı grupta mı?" = iki `find` karşılaştırması.



Birleşmeler sürer ve sonunda tüm mıknatıslar **tek grupta** toplanır: `find(x)` herkes için aynı kökü verir.

Find ve Union İşlemleri

- **find(x)**: x'i içeren kümenin adını (kökünü) döndürür. Ebeveyn bağlantıları kök bulunana kadar izlenir.
- Bu, arama ağaçlarındaki (Bölüm 10) *arama* ile aynı şey değildir: değer aranmaz, yukarı doğru köke çıkılır.
- **union(x, y)**: iki kümenin yerine, ikisinin birleşimi olan tek bir küme yaratır: bir kökü diğerinin altına bağlar.

Kümeler baştan ayrık olduğu için birleşimde tekrar kontrolü yok: yalnızca **bir ebeveyn bağlantısı** değişir.

```
find(x):  
    if x.ebeveyn != x:  
        x.ebeveyn = find(x.ebeveyn) // yol kısaltma  
    return x.ebeveyn  
  
union(x, y):  
    kök1 = find(x)  
    kök2 = find(y)  
    if kök1 != kök2:  
        // bir kümenin kökünü diğer  
        // kümenin köküne ebeveyn  
        kök1.ebeveyn = kök2
```

Nesneleri İndislere Eşleme

- Union-Find en verimli hâliyle **dizi** üzerinde çalışır: öğeler $0..n-1$ tamsayılarıdır.
- Öğeler harf, şehir, kişi... ise önce her nesne bir indise **rastgele (ama kalıcı) eşlenir** ve bu eşleme bir hash tablosunda saklanır.
- Sonra `ebeveyn[i] = i` ile her öğe kendi kümesinin kökü olarak başlar (`makeSet`).

E → 0

F → 1

I → 2

D → 3

C → 4

A → 5

J → 6

L → 7

G → 8

K → 9

B → 10

H → 11

```
Map<Character, Integer> indis = new HashMap<>(); // 'E'→0, 'F'→1, ...  
int[] ebeveyn = new int[12];  
for (int i = 0; i < 12; i++) ebeveyn[i] = i; // 12 tekil küme
```

Union İşlemi Adım Adım

öge	E	F	I	D	C	A	J	L	G	K	B	H
indis	0	1	2	3	4	5	6	7	8	9	10	11
ebeveyn	0	1	2	3	4	5	6	7	8	9	10	11

İşlemler

```

union(C, K)
union(F, E)
union(A, J)
union(A, B)
union(C, D)
union(D, I)
union(L, F)
union(C, A)
union(A, B)
union(H, G)
union(H, F)
union(H, B)

```

Başlangıç: 12 tekil küme, `ebeveyn[i] = i`. Kural: **küçük ağacın kökü büyüğün köküne** bağlanır; eşitse ikinci kök birinciye bağlanır. Yol kısaltma yok.

Union İşlemi Adım Adım

öge	E	F	I	D	C	A	J	L	G	K	B	H
indis	0	1	2	3	4	5	6	7	8	9	10	11
ebeveyn	0	1	2	3	4	5	6	7	8	4	10	11

İşlemler

union(C, K)

union(F, E)

union(A, J)

union(A, B)

union(C, D)

union(D, I)

union(L, F)

union(C, A)

union(A, B)

union(H, G)

union(H, F)

union(H, B)

union(C, K): find(C) = C, find(K) = K; boyutlar eşit (1) → **K**, C'nin altına bağlanır (ebeveyn[9] = 4).

Union İşlemi Adım Adım

öge	E	F	I	D	C	A	J	L	G	K	B	H
indis	0	1	2	3	4	5	6	7	8	9	10	11
ebeveyn	1	1	2	3	4	5	6	7	8	4	10	11

İşlemler

union(C, K)

union(F, E)

union(A, J)

union(A, B)

union(C, D)

union(D, I)

union(L, F)

union(C, A)

union(A, B)

union(H, G)

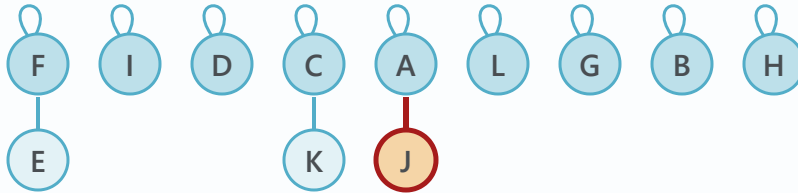
union(H, F)

union(H, B)

union(F, E): find(F) = F, find(E) = E; boyutlar eşit (1) → **E**, F'nin altına bağlanır (ebeveyn[0] = 1).

Union İşlemi Adım Adım

öge	E	F	I	D	C	A	J	L	G	K	B	H
indis	0	1	2	3	4	5	6	7	8	9	10	11
ebeveyn	1	1	2	3	4	5	5	7	8	4	10	11



İşlemler

union(C, K)

union(F, E)

union(A, J)

union(A, B)

union(C, D)

union(D, I)

union(L, F)

union(C, A)

union(A, B)

union(H, G)

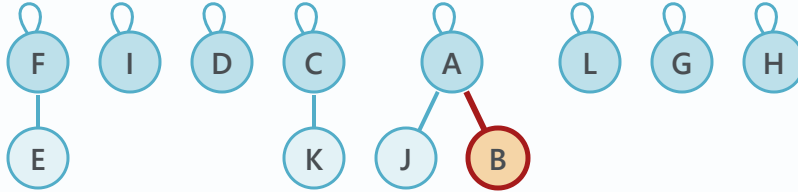
union(H, F)

union(H, B)

union(A, J): find(A) = A, find(J) = J; boyutlar eşit (1) → **J**, A'nın altına bağlanır (ebeveyn[6] = 5).

Union İşlemi Adım Adım

öge	E	F	I	D	C	A	J	L	G	K	B	H
indis	0	1	2	3	4	5	6	7	8	9	10	11
ebeveyn	1	1	2	3	4	5	5	7	8	4	5	11



İşlemler

union(C, K)

union(F, E)

union(A, J)

union(A, B)

union(C, D)

union(D, I)

union(L, F)

union(C, A)

union(A, B)

union(H, G)

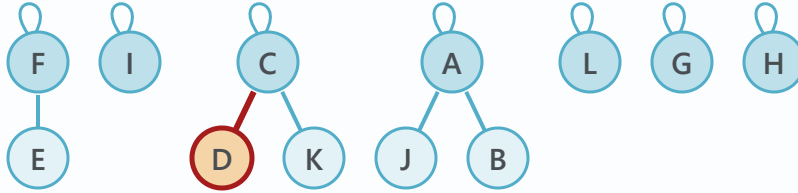
union(H, F)

union(H, B)

union(A, B): find(A) = A, find(B) = B; boyut $1 < 2 \rightarrow \mathbf{B}$, A'nın altına bağlanır (ebeveyn[10] = 5).

Union İşlemi Adım Adım

öge	E	F	I	D	C	A	J	L	G	K	B	H
indis	0	1	2	3	4	5	6	7	8	9	10	11
ebeveyn	1	1	2	4	4	5	5	7	8	4	5	11



İşlemler

union(C, K)

union(F, E)

union(A, J)

union(A, B)

union(C, D)

union(D, I)

union(L, F)

union(C, A)

union(A, B)

union(H, G)

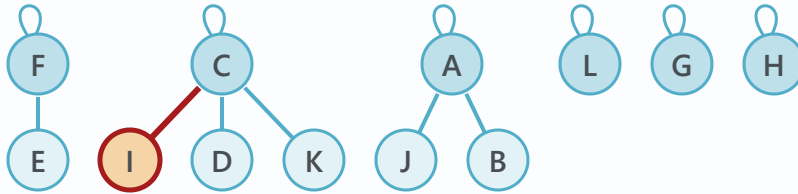
union(H, F)

union(H, B)

union(C, D): find(C) = C, find(D) = D; boyut 1 < 2 → **D**, C'nin altına bağlanır (ebeveyn[3] = 4).

Union İşlemi Adım Adım

öge	E	F	I	D	C	A	J	L	G	K	B	H
indis	0	1	2	3	4	5	6	7	8	9	10	11
ebeveyn	1	1	4	4	4	5	5	7	8	4	5	11



İşlemler

union(C, K)

union(F, E)

union(A, J)

union(A, B)

union(C, D)

union(D, I)

union(L, F)

union(C, A)

union(A, B)

union(H, G)

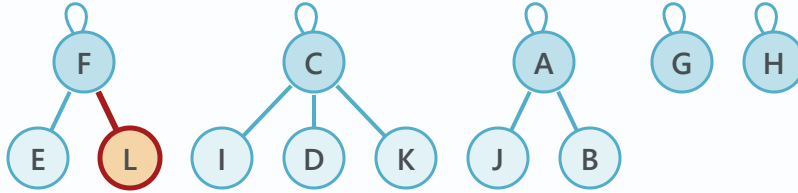
union(H, F)

union(H, B)

union(D, I): find(D) = C, find(I) = I; boyut 1 < 3 → I, C'nin altına bağlanır (ebeveyn[2] = 4).

Union İşlemi Adım Adım

öge	E	F	I	D	C	A	J	L	G	K	B	H
indis	0	1	2	3	4	5	6	7	8	9	10	11
ebeveyn	1	1	4	4	4	5	5	1	8	4	5	11



İşlemler

union(C, K)

union(F, E)

union(A, J)

union(A, B)

union(C, D)

union(D, I)

union(L, F)

union(C, A)

union(A, B)

union(H, G)

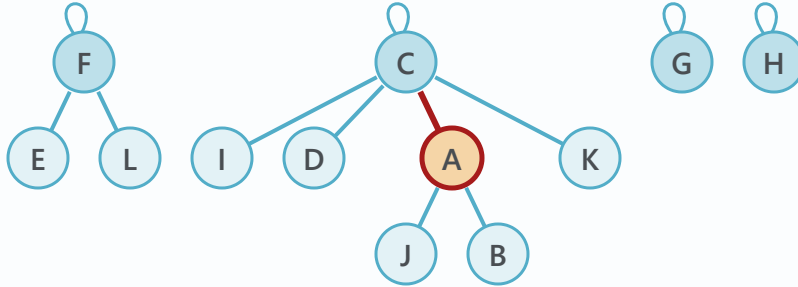
union(H, F)

union(H, B)

union(L, F): find(L) = L, find(F) = F; boyut 1 < 2 → **L**, F'nin altına bağlanır (ebeveyn[7] = 1).

Union İşlemi Adım Adım

öge	E	F	I	D	C	A	J	L	G	K	B	H
indis	0	1	2	3	4	5	6	7	8	9	10	11
ebeveyn	1	1	4	4	4	4	5	1	8	4	5	11



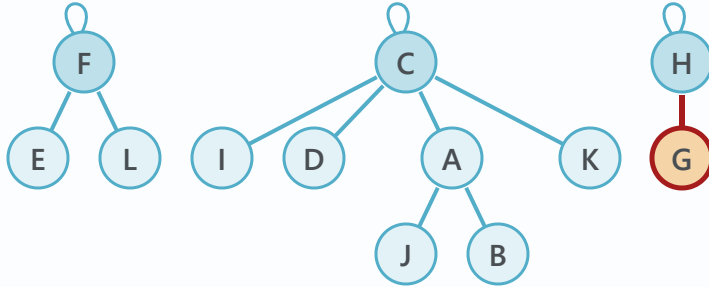
İşlemler

- union(C, K)
- union(F, E)
- union(A, J)
- union(A, B)
- union(C, D)
- union(D, I)
- union(L, F)
- union(C, A)**
- union(A, B)
- union(H, G)
- union(H, F)
- union(H, B)

union(C, A): find(C) = C, find(A) = A; boyut 3 < 4 → **A**, C'nin altına bağlanır (ebeveyn[5] = 4).

Union İşlemi Adım Adım

öge	E	F	I	D	C	A	J	L	G	K	B	H
indis	0	1	2	3	4	5	6	7	8	9	10	11
ebeveyn	1	1	4	4	4	4	5	1	11	4	5	11



İşlemler

union(C, K)

union(F, E)

union(A, J)

union(A, B)

union(C, D)

union(D, I)

union(L, F)

union(C, A)

union(A, B)

union(H, G)

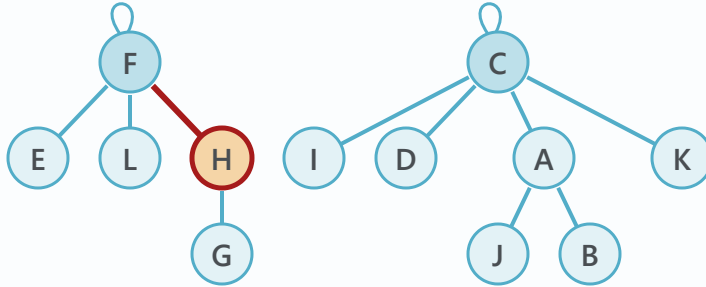
union(H, F)

union(H, B)

union(A, B): $\text{find}(A) = \text{find}(B) = C \rightarrow$ zaten aynı kümede, hiçbir şey değişmez. **union(H, G):** $\text{find}(H) = H, \text{find}(G) = G$; boyutlar eşit (1) $\rightarrow$ **G**, H'nin altına bağlanır (ebeveyn[8] = 11).

Union İşlemi Adım Adım

öge	E	F	I	D	C	A	J	L	G	K	B	H
indis	0	1	2	3	4	5	6	7	8	9	10	11
ebeveyn	1	1	4	4	4	4	5	1	11	4	5	1



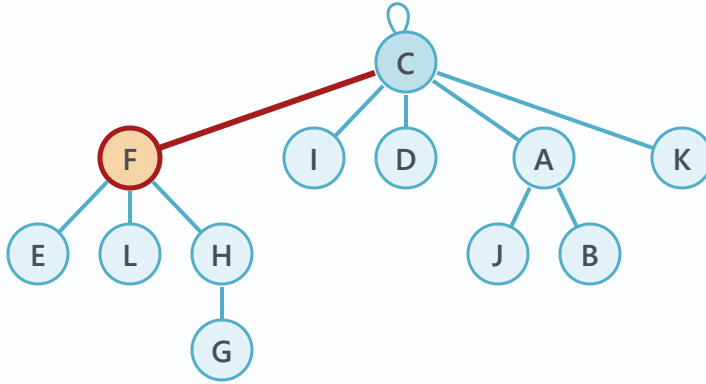
İşlemler

- union(C, K)
- union(F, E)
- union(A, J)
- union(A, B)
- union(C, D)
- union(D, I)
- union(L, F)
- union(C, A)
- union(A, B)
- union(H, G)
- union(H, F)**
- union(H, B)

union(H, F): find(H) = H, find(F) = F; boyut $2 < 3 \rightarrow \mathbf{H}$, F'nin altına bağlanır (ebeveyn[11] = 1).

Union İşlemi Adım Adım

öge	E	F	I	D	C	A	J	L	G	K	B	H
indis	0	1	2	3	4	5	6	7	8	9	10	11
ebeveyn	1	4	4	4	4	4	5	1	11	4	5	1



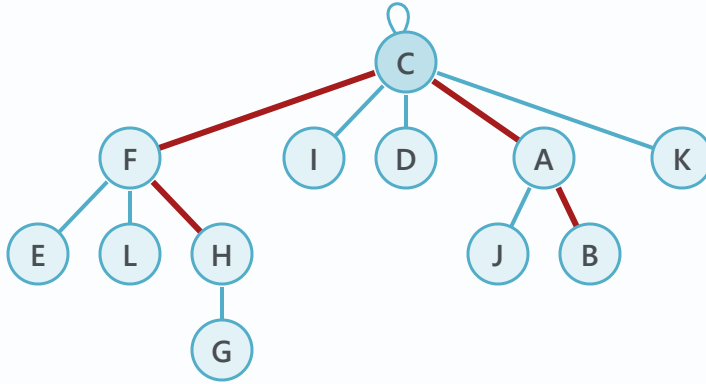
İşlemler

- union(C, K)
- union(F, E)
- union(A, J)
- union(A, B)
- union(C, D)
- union(D, I)
- union(L, F)
- union(C, A)
- union(A, B)
- union(H, G)
- union(H, F)
- union(H, B)**

union(H, B): find(H) = F, find(B) = C; boyut 5 < 7 → **F**, C'nin altına bağlanır (ebeveyn[1] = 4).

Union İşlemi Adım Adım

öge	E	F	I	D	C	A	J	L	G	K	B	H
indis	0	1	2	3	4	5	6	7	8	9	10	11
ebeveyn	1	4	4	4	4	4	5	1	11	4	5	1



İşlemler
 union(C, K)
 union(F, E)
 union(A, J)
 union(A, B)
 union(C, D)
 union(D, I)
 union(L, F)
 union(C, A)
 union(A, B)
 union(H, G)
 union(H, F)
 union(H, B)

Sonuç: tek küme, kök **C**. H ile B aynı kümede mi? find(H): $H \rightarrow F \rightarrow C$, find(B): $B \rightarrow A \rightarrow C$; kökler aynı → **evet** (toplam 4 adım).

Başka Bir Fikir: Quick-Find (Hızlı Bul)

- Ağaç yerine düz dizi: $id[i]$ = i'nin **küme adı**; aynı kümedekilerin id'si aynı. $find(p) = id[p] \rightarrow O(1)$.
- $union(p, q)$: id'si $id[p]$ olan *herkesi* $id[q]$ yap $\rightarrow$ tüm dizi taranır, **$O(n)$** .

i	0	1	2	3	4	5
id[i]	1	1	3	3	4	5

kümeler: {0, 1} {2, 3} {4} {5}

$union(0,1)$, $union(2,3)$ sonrası: 4 küme (aynı renk = aynı küme)

Yöntem (Sedgewick)	find	union	n – 1 union toplamı
Quick-Find (dizi, küme adı)	$O(1)$	$O(n)$	$O(n^2)$
Quick-Union (ebeveyn ağacı, naif)	$O(h)$, en kötü $O(n)$	$O(h)$	en kötü $O(n^2)$
Rütbe/boyut + yol kısaltma	$O(\alpha(n))$ amortize		$\approx O(n)$

Kod İzleme: Quick-Find union

i	0	1	2	3	4	5	
id[i]	1	1	3	3	4	5	pid = 1 qid = 3

kümeler: {0, 1} {2, 3} {4} {5}

union(1, 3): p'nin küme adı **pid = id[1] = 1**, q'nunki **qid = id[3] = 3**.

Adım 1 / 16

```
1 int find(int p) { return id[p]; } // O(1)
2 void union(int p, int q) {
3     int pid = id[p], qid = id[q];
4     if (pid == qid) return;
5     for (int i = 0; i < id.length; i++)
6         if (id[i] == pid)
7             id[i] = qid;          // O(n) tarama
8 }
```

DEĞİŞKENLER

p = 1

q = 3

pid = 1

qid = 3

id = [1, 1, 3, 3, 4, 5]

ÇAĞRI YIĞINI

union(1, 3)

main()

Kod İzleme: Quick-Find union

i	0	1	2	3	4	5	
id[i]	1	1	3	3	4	5	pid = 1 qid = 3

kümeler: {0, 1} {2, 3} {4} {5}

pid ≠ qid: farklı kümeler, birleştirme gerekli. Şimdi **tüm dizi** taranacak.

Adım 2 / 16

```
1 int find(int p) { return id[p]; } // O(1)
2 void union(int p, int q) {
3   int pid = id[p], qid = id[q];
4   ► if (pid == qid) return;
5   for (int i = 0; i < id.length; i++)
6     if (id[i] == pid)
7       id[i] = qid;          // O(n) tarama
8 }
```

DEĞİŞKENLER

p = 1

q = 3

pid = 1

qid = 3

id = [1, 1, 3, 3, 4, 5]

ÇAĞRI YIĞINI

union(1, 3)

main()

Kod İzleme: Quick-Find union



kümeler: {0, 2, 3} {1} {4} {5}

`id[0] = 1 = pid → id[0] = 3` yapılır.

Adım 3 / 16

```
1 int find(int p) { return id[p]; } // O(1)
2 void union(int p, int q) {
3     int pid = id[p], qid = id[q];
4     if (pid == qid) return;
5     for (int i = 0; i < id.length; i++)
6         if (id[i] == pid)
7             id[i] = qid; // O(n) tarama
8 }
```

DEĞİŞKENLER

`p = 1` `q = 3` `pid = 1` `qid = 3`
`i = 0` `id = [3, 1, 3, 3, 4, 5]`

ÇAĞRI YIĞINI

`union(1, 3)`

`main()`

Kod İzleme: Quick-Find union



kümeler: {0, 1, 2, 3} {4} {5}

`id[1] = 1 = pid → id[1] = 3` yapılır.

Adım 4 / 16

```
1 int find(int p) { return id[p]; } // O(1)
2 void union(int p, int q) {
3     int pid = id[p], qid = id[q];
4     if (pid == qid) return;
5     for (int i = 0; i < id.length; i++)
6         if (id[i] == pid)
7             id[i] = qid;          // O(n) tarama
8 }
```

DEĞİŞKENLER

`p = 1`

`q = 3`

`pid = 1`

`qid = 3`

`i = 1`

`id = [3, 3, 3, 3, 4, 5]`

ÇAĞRI YIĞINI

`union(1, 3)`

`main()`

Kod İzleme: Quick-Find union



kümeler: {0, 1, 2, 3} {4} {5}

id[2] = 3 ≠ pid: dokunulmaz (ama yine de bakıldı).

Adım 5 / 16

```
1 int find(int p) { return id[p]; } // O(1)
2 void union(int p, int q) {
3     int pid = id[p], qid = id[q];
4     if (pid == qid) return;
5     for (int i = 0; i < id.length; i++)
6         if (id[i] == pid)
7             id[i] = qid; // O(n) tarama
8 }
```

DEĞİŞKENLER

p = 1

q = 3

pid = 1

qid = 3

i = 2

id = [3, 3, 3, 3, 4, 5]

ÇAĞRI YIĞINI

union(1, 3)

main()

Kod İzleme: Quick-Find union



kümeler: {0, 1, 2, 3} {4} {5}

id[3] = 3 ≠ pid: dokunulmaz (ama yine de bakıldı).

Adım 6 / 16

```

1 int find(int p) { return id[p]; } // O(1)
2 void union(int p, int q) {
3     int pid = id[p], qid = id[q];
4     if (pid == qid) return;
5     for (int i = 0; i < id.length; i++)
6         if (id[i] == pid)
7             id[i] = qid;          // O(n) tarama
8 }
```

DEĞİŞKENLER

p = 1

q = 3

pid = 1

qid = 3

i = 3

id = [3, 3, 3, 3, 4, 5]

ÇAĞRI YIĞINI

union(1, 3)

main()

Kod İzleme: Quick-Find union



kümeler: {0, 1, 2, 3} {4} {5}

id[4] = 4 ≠ pid: dokunulmaz (ama yine de bakıldı).

Adım 7 / 16

```

1 int find(int p) { return id[p]; } // O(1)
2 void union(int p, int q) {
3     int pid = id[p], qid = id[q];
4     if (pid == qid) return;
5     for (int i = 0; i < id.length; i++)
6         if (id[i] == pid)
7             id[i] = qid;          // O(n) tarama
8 }
```

DEĞİŞKENLER

p = 1

q = 3

pid = 1

qid = 3

i = 4

id = [3, 3, 3, 3, 4, 5]

ÇAĞRI YIĞINI

union(1, 3)

main()

Kod İzleme: Quick-Find union



kümeler: {0, 1, 2, 3} {4} {5}

id[5] = 5 ≠ pid: dokunulmaz (ama yine de bakıldı).

Adım 8 / 16

```

1 int find(int p) { return id[p]; } // O(1)
2 void union(int p, int q) {
3     int pid = id[p], qid = id[q];
4     if (pid == qid) return;
5     for (int i = 0; i < id.length; i++)
6         if (id[i] == pid)
7             id[i] = qid;          // O(n) tarama
8 }

```

DEĞİŞKENLER

p = 1

q = 3

pid = 1

qid = 3

i = 5

id = [3, 3, 3, 3, 4, 5]

ÇAĞRI YIĞINI

union(1, 3)

main()

Kod İzleme: Quick-Find union

i	0	1	2	3	4	5	
id[i]	3	3	3	3	4	5	pid = 4 qid = 3

kümeler: {0, 1, 2, 3} {4} {5}

union(4, 3): p'nin küme adı **pid = id[4] = 4**, q'nunki **qid = id[3] = 3**.

Adım 9 / 16

```
1 int find(int p) { return id[p]; } // O(1)
2 void union(int p, int q) {
3     int pid = id[p], qid = id[q];
4     if (pid == qid) return;
5     for (int i = 0; i < id.length; i++)
6         if (id[i] == pid)
7             id[i] = qid; // O(n) tarama
8 }
```

DEĞİŞKENLER

p = 4

q = 3

pid = 4

qid = 3

id = [3, 3, 3, 3, 4, 5]

ÇAĞRI YIĞINI

union(4, 3)

main()

Kod İzleme: Quick-Find union

i	0	1	2	3	4	5	
id[i]	3	3	3	3	4	5	pid = 4 qid = 3

kümeler: {0, 1, 2, 3} {4} {5}

pid ≠ qid: farklı kümeler, birleştirme gerekli. Şimdi **tüm dizi** taranacak.

Adım 10 / 16

```
1 int find(int p) { return id[p]; } // O(1)
2 void union(int p, int q) {
3     int pid = id[p], qid = id[q];
4     ► if (pid == qid) return;
5     for (int i = 0; i < id.length; i++)
6         if (id[i] == pid)
7             id[i] = qid;          // O(n) tarama
8 }
```

DEĞİŞKENLER

p = 4

q = 3

pid = 4

qid = 3

id = [3, 3, 3, 3, 4, 5]

ÇAĞRI YIĞINI

union(4, 3)

main()

Kod İzleme: Quick-Find union



kümeler: {0, 1, 2, 3} {4} {5}

id[0] = 3 ≠ pid: dokunulmaz (ama yine de bakıldı).

Adım 11 / 16

```

1 int find(int p) { return id[p]; } // O(1)
2 void union(int p, int q) {
3     int pid = id[p], qid = id[q];
4     if (pid == qid) return;
5     for (int i = 0; i < id.length; i++)
6         if (id[i] == pid)
7             id[i] = qid;          // O(n) tarama
8 }
```

DEĞİŞKENLER

p = 4

q = 3

pid = 4

qid = 3

i = 0

id = [3, 3, 3, 3, 4, 5]

ÇAĞRI YIĞINI

union(4, 3)

main()

Kod İzleme: Quick-Find union



kümeler: {0, 1, 2, 3} {4} {5}

id[1] = 3 ≠ pid: dokunulmaz (ama yine de bakıldı).

Adım 12 / 16

```

1 int find(int p) { return id[p]; } // O(1)
2 void union(int p, int q) {
3     int pid = id[p], qid = id[q];
4     if (pid == qid) return;
5     for (int i = 0; i < id.length; i++)
6         if (id[i] == pid)
7             id[i] = qid;          // O(n) tarama
8 }

```

DEĞİŞKENLER

p = 4

q = 3

pid = 4

qid = 3

i = 1

id = [3, 3, 3, 3, 4, 5]

ÇAĞRI YIĞINI

union(4, 3)

main()

Kod İzleme: Quick-Find union



kümeler: {0, 1, 2, 3} {4} {5}

id[2] = 3 ≠ pid: dokunulmaz (ama yine de bakıldı).

Adım 13 / 16

```

1 int find(int p) { return id[p]; } // O(1)
2 void union(int p, int q) {
3     int pid = id[p], qid = id[q];
4     if (pid == qid) return;
5     for (int i = 0; i < id.length; i++)
6         if (id[i] == pid)
7             id[i] = qid;          // O(n) tarama
8 }
```

DEĞİŞKENLER

p = 4

q = 3

pid = 4

qid = 3

i = 2

id = [3, 3, 3, 3, 4, 5]

ÇAĞRI YIĞINI

union(4, 3)

main()

Kod İzleme: Quick-Find union



kümeler: {0, 1, 2, 3} {4} {5}

id[3] = 3 ≠ pid: dokunulmaz (ama yine de bakıldı).

Adım 14 / 16

```

1 int find(int p) { return id[p]; } // O(1)
2 void union(int p, int q) {
3     int pid = id[p], qid = id[q];
4     if (pid == qid) return;
5     for (int i = 0; i < id.length; i++)
6         if (id[i] == pid)
7             id[i] = qid;          // O(n) tarama
8 }
```

DEĞİŞKENLER

p = 4

q = 3

pid = 4

qid = 3

i = 3

id = [3, 3, 3, 3, 4, 5]

ÇAĞRI YIĞINI

union(4, 3)

main()

Kod İzleme: Quick-Find union



kümeler: {0, 1, 2, 3, 4} {5}

id[4] = 4 = pid → **id[4] = 3** yapılır.

Adım 15 / 16

```

1 int find(int p) { return id[p]; } // O(1)
2 void union(int p, int q) {
3     int pid = id[p], qid = id[q];
4     if (pid == qid) return;
5     for (int i = 0; i < id.length; i++)
6         if (id[i] == pid)
7             id[i] = qid;          // O(n) tarama
8 }
```

DEĞİŞKENLER

p = 4

q = 3

pid = 4

qid = 3

i = 4

id = [3, 3, 3, 3, 3, 5]

ÇAĞRI YIĞINI

union(4, 3)

main()

Kod İzleme: Quick-Find union



kümeler: {0, 1, 2, 3, 4} {5}

id[5] = 5 ≠ pid: dokunulmaz. Her union **6 hücrenin hepsine** baktı: n öğede $O(n)$; n - 1 union → $O(n^2)$.

Adım 16 / 16

```

1 int find(int p) { return id[p]; } // O(1)
2 void union(int p, int q) {
3     int pid = id[p], qid = id[q];
4     if (pid == qid) return;
5     for (int i = 0; i < id.length; i++)
6         if (id[i] == pid)
7             id[i] = qid;          // O(n) tarama
8 }
```

DEĞİŞKENLER

p = 4

q = 3

pid = 4

qid = 3

i = 5

id = [3, 3, 3, 3, 3, 5]

ÇAĞRI YIĞINI

union(4, 3)

main()

Neden İyileştirme Gerekir?

- find, kökten uzaklık kadar adım atar. Birleştirme sırası kötü seçilirse ağaç bir **zincire** dönüşür → en kötü $O(n)$.

Naif: eski kök yeninin altına



yükseklik 0 · find(1) = 0 adım

Boyuta göre: küçük ağaç büyüğün altına



yükseklik 0 · find(1) = 0 adım

Başlangıç: 5 tekil küme; her düğüm kendi kökü. Sırayla union(1,2), union(2,3), union(3,4), union(4,5) yapılacak.

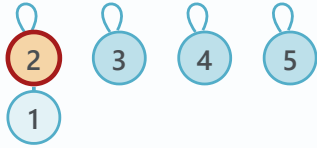
Adım 1 / 5

İki basit fikir ağaçları basık tutar: **(1) boyuta / rütbeye göre birleştirme** ve **(2) yol kısaltma**. Birlikte işlem başına neredeyse sabit süre: $O(\alpha(n))$.

Neden İyileştirme Gerekir?

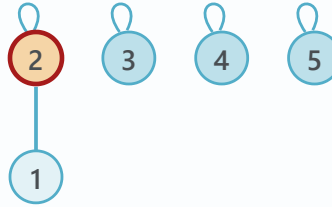
- find, kökten uzaklık kadar adım atar. Birleştirme sırası kötü seçilirse ağaç bir **zincire** dönüşür → en kötü $O(n)$.

Naif: eski kök yeninin altına



yükseklik 1 · find(1) = 1 adım

Boyuta göre: küçük ağaç büyüğün altına



yükseklik 1 · find(1) = 1 adım

union(1, 2): iki tekil küme, iki yöntem de 1'i 2'nin altına bağlar.

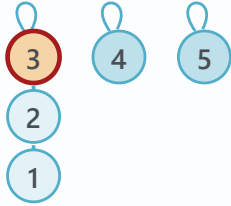
Adım 2 / 5

İki basit fikir ağaçları basık tutar: **(1) boyuta / rütbeğe göre birleştirme** ve **(2) yol kısaltma**. Birlikte işlem başına neredeyse sabit süre: $O(\alpha(n))$.

Neden İyileştirme Gerekir?

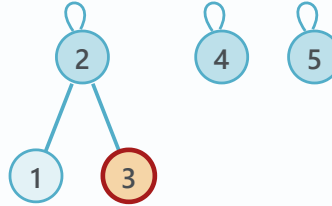
- find, kökten uzaklık kadar adım atar. Birleştirme sırası kötü seçilirse ağaç bir **zincire** dönüşür → en kötü $O(n)$.

Naif: eski kök yeninin altına



yükseklik 2 · find(1) = 2 adım

Boyuta göre: küçük ağaç büyüğün altına



yükseklik 1 · find(1) = 1 adım

union(2, 3): naif yöntem 2 kökünü (2 öğeli ağaç) tekil 3'ün altına bağlar → zincir uzar. Boyuta göre: tekil 3, büyük ağacın kökü 2'ye bağlanır.

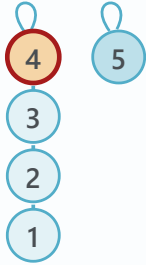
Adım 3 / 5

İki basit fikir ağaçları basık tutar: **(1) boyuta / rütbeye göre birleştirme** ve **(2) yol kısaltma**. Birlikte işlem başına neredeyse sabit süre: $O(\alpha(n))$.

Neden İyileştirme Gerekir?

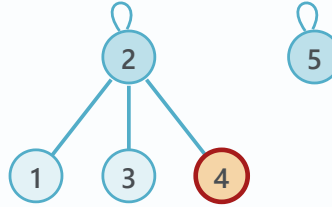
- find, kökten uzaklık kadar adım atar. Birleştirme sırası kötü seçilirse ağaç bir **zincire** dönüşür → en kötü $O(n)$.

Naif: eski kök yeninin altına



yükseklik 3 · find(1) = 3 adım

Boyuta göre: küçük ağaç büyüğün altına



yükseklik 1 · find(1) = 1 adım

union(3, 4): naif zincir bir kat daha uzar; boyuta göre 4 yine köke eklenir.

Adım 4 / 5

İki basit fikir ağaçları basık tutar: **(1) boyuta / rütbeye göre birleştirme** ve **(2) yol kısaltma**. Birlikte işlem başına neredeyse sabit süre: $O(\alpha(n))$.

Neden İyileştirme Gerekir?

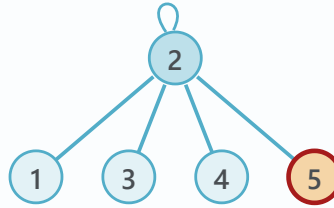
- find, kökten uzaklık kadar adım atar. Birleştirme sırası kötü seçilirse ağaç bir **zincir** dönüşür → en kötü $O(n)$.

Naif: eski kök yeninin altına



yükseklik 4 · find(1) = 4 adım

Boyuta göre: küçük ağaç büyüğün altına



yükseklik 1 · find(1) = 1 adım

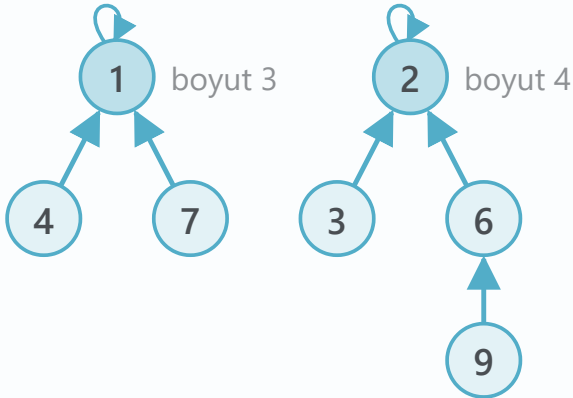
union(4, 5): naif ağaç **zincir** oldu (yükseklik $n - 1 \rightarrow \text{find } O(n)$); boyuta göre ağaç **yıldız** (yükseklik 1).

Adım 5 / 5

İki basit fikir ağaçları basık tutar: **(1) boyuta / rütbeğe göre birleştirme** ve **(2) yol kısaltma**. Birlikte işlem başına neredeyse sabit süre: $O(\alpha(n))$.

Boyuta Göre Birleştirme (Union by Size)

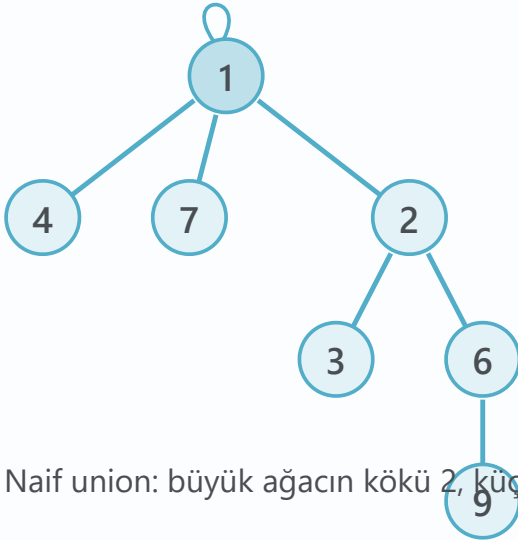
- Her kökte ağacın **boyutunu** (öge sayısı) sakla.
- Birleştirirken **küçük ağacın kökünü büyük ağacın köküne** bağla; yeni boyut = iki boyutun toplamı.



İki küme: {1, 4, 7} (boyut 3) ve {2, 3, 6, 9} (boyut 4). Yükseklikler 1 ve 2.

Boyuta Göre Birleştirme (Union by Size)

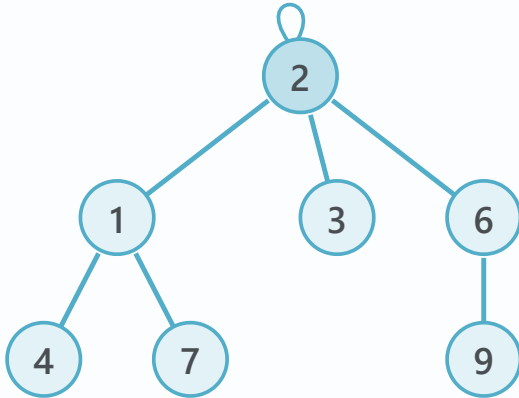
- Her kökte ağacın **boyutunu** (öge sayısı) sakla.
- Birleştirirken **küçük ağacın kökünü büyük ağacın köküne** bağla; yeni boyut = iki boyutun toplamı.



Naif union: büyük ağacın kökü 2, küçüğün altına bağlanırsa yükseklik **3**'e çıkar; 9'un find'ı uzar.

Boyuta Göre Birleştirme (Union by Size)

- Her kökte ağacın **boyutunu** (öğesi sayısı) sakla.
- Birleştirirken **küçük ağacın kökünü büyük ağacın köküne** bağla; yeni boyut = iki boyutun toplamı.



Boyuta göre: küçük ağacın kökü 1, büyüğün kökü 2'ye bağlanır. Yükseklik **2** kalır, $\text{boyut}[2] = 3 + 4 = \mathbf{7}$.

Rütbeye Göre Birleştirme (Union by Rank)

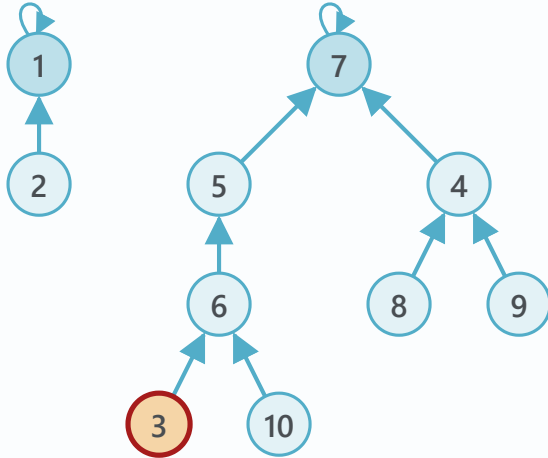
- **Rütbe**, yüksekliğin üst sınırıdır; yeni kümede 0'dır.
- Küçük rütbeli kök, büyük rütbeli köke bağlanır.
- Rütbeler eşitse yeni kökün rütbesi bir artar.
- Bu seçim, yol kısaltma olmadan bile yüksekliği $O(\log n)$ tutar.

Yol kısaltma yüksekliği azaltır ama rütbeyi güncellemez; bu yüzden rütbe yalnızca bir *üst sınırdır*.

```
makeSet(x):  
    ebeveyn[x] = x  
    sıra[x] = 0  
  
find(x):    // yol kısaltmalı  
    if x != ebeveyn[x]:  
        ebeveyn[x] = find(ebeveyn[x])  
    return ebeveyn[x]  
  
link(x, y):  // x, y kök  
    if sıra[x] > sıra[y]:  
        ebeveyn[y] = x  
    else:  
        ebeveyn[x] = y  
        if sıra[x] == sıra[y]:  
            sıra[y] = sıra[y] + 1  
  
union(x, y):  
    link(find(x), find(y))
```

Yol Kısaltma (Path Compression)

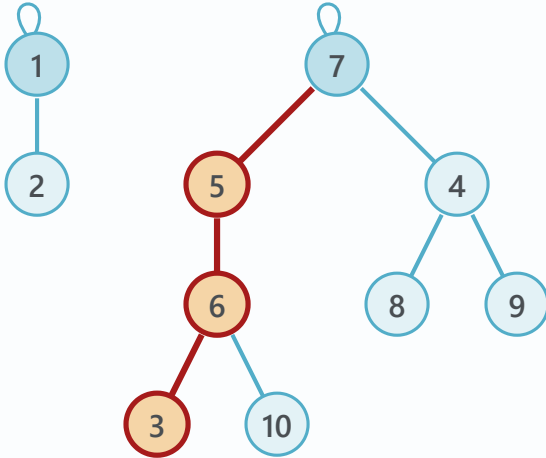
Yol kısaltma (path compression): find sırasında rastlanan tüm düğümleri doğrudan **köke bağla**. Sonraki find'lar tek adımda biter.



Başlangıç: 3'ün kökü 7, ama aradaki yol $3 \rightarrow 6 \rightarrow 5 \rightarrow 7$ (3 adım).

Yol Kısaltma (Path Compression)

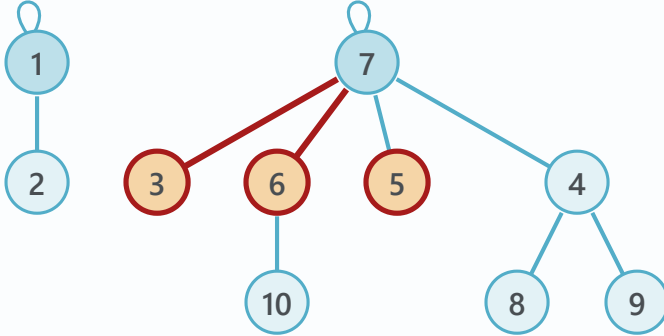
Yol kısaltma (path compression): find sırasında rastlanan tüm düğümleri doğrudan **köke bağla**. Sonraki find'lar tek adımda biter.



find(3): özyineleme $3 \rightarrow 6 \rightarrow 5 \rightarrow 7$ diye köke çıkar; kök 7 bulunur.

Yol Kısaltma (Path Compression)

Yol kısaltma (path compression): find sırasında rastlanan tüm düğümleri doğrudan **köke bağla**. Sonraki find'lar tek adımda biter.



Dönüşte her düğümün ebeveyni 7 yapılır: `ebeveyn[3] = ebeveyn[6] = 7` (5 zaten 7'deydi). Artık `find(3)`, `find(6)` tek adım; 10'un yolu da kısaldı.

Union-Find Karmaşıklığı

Uygulama	find	union	Not
Naif (rastgele bağlama)	$O(n)$	$O(n)$	ağaç zincire dönüşebilir
Boyuta veya rütbeye göre	$O(\log n)$	$O(\log n)$	yükseklik $\leq \log_2 n$
Yalnızca yol kısaltma	$O(\log n)$ amortize		m işlem için
Rütbe + yol kısaltma	$O(\alpha(n))$ amortize		pratikte sabit

$\alpha(n)$: ters Ackermann fonksiyonu

Son derece yavaş büyür: evrendeki atom sayısı kadar öge için bile $\alpha(n) \leq 4$.

Bellek

İki tamsayı dizisi: `ebeveyn[n]` ve `sira[n]`
(veya `boyut[n]`) $\rightarrow O(n)$.

Tarjan (1975): m işlem dizisi $O(m \cdot \alpha(n))$ sürede biter. Kruskal, bağlantılı bileşenler, resim işleme (bölge etiketleme), ağ bağlantı sorguları gibi yerlerde kullanılır.

Java: Düğüm (Nesne) Tabanlı Union-Find

```
class SetDugum {
    int veri;
    int sıra;    // rütbe
    SetDugum ebeveyn;

    SetDugum(int veri) {
        this.veri = veri;
        this.ebeveyn = this;
        this.sıra = 0;
    }
}

SetDugum find(SetDugum d) {
    if (d != d.ebeveyn)
        d.ebeveyn = find(d.ebeveyn); // kısaltma
    return d.ebeveyn;
}
```

```
void union(SetDugum x, SetDugum y) {
    SetDugum kokX = find(x);
    SetDugum kokY = find(y);
    if (kokX == kokY) return;
    // rütbeye göre birleştirme
    if (kokX.sıra < kokY.sıra) {
        kokX.ebeveyn = kokY;
    } else if (kokX.sıra > kokY.sıra) {
        kokY.ebeveyn = kokX;
    } else { // eşit: birini seç
        kokX.ebeveyn = kokY;
        kokY.sıra++;
    }
}
```

Java: Dizi Tabanlı Union-Find

```
class UnionFind {
    private final int[] ebeveyn, sıra;
    UnionFind(int n) {
        ebeveyn = new int[n];  sıra = new int[n];
        for (int i = 0; i < n; i++) ebeveyn[i] = i;
    }
    int find(int x) {
        if (ebeveyn[x] != x) ebeveyn[x] = find(ebeveyn[x]);
        return ebeveyn[x];
    }
    boolean union(int x, int y) {
        int kx = find(x), ky = find(y);
        if (kx == ky) return false;
        if (sıra[kx] < sıra[ky]) ebeveyn[kx] = ky;
        else if (sıra[kx] > sıra[ky]) ebeveyn[ky] = kx;
        else { ebeveyn[kx] = ky; sıra[ky]++; }
        return true;
    }
}
```

Kullanım

```
UnionFind uf = new UnionFind(5);
uf.union(0, 1); uf.union(3, 4);
uf.find(1) == uf.find(0) → true
uf.find(1) == uf.find(3) → false
```

union neden boolean?

Birleşme olduysa true: Kruskal'da "kenar döngü oluşturuyor mu?" sorusunun cevabı.

Derin özyineleme

Çok büyük n'de find döngüyle de yazılabilir.

Kod İzleme: find + union

```

1 int find(int x) {
2   if (ebeveyn[x] != x) ebeveyn[x] = find(ebeveyn[x]);
3   return ebeveyn[x];
4 }
5 boolean union(int x, int y) {
6   int kx = find(x), ky = find(y);
7   if (kx == ky) return false;
8   if (sira[kx] < sira[ky]) ebeveyn[kx] = ky;
9   else if (sira[kx] > sira[ky]) ebeveyn[ky] = kx;
10  else { ebeveyn[kx] = ky; sira[ky]++; }
11  return true;
12 }

```

ÇAĞRI YIĞINI

```
main()
```



i	0	1	2	3	4	5
ebeveyn	0	1	2	3	4	5
sira	0	0	0	0	0	0

6 öge, `ebeveyn[i] = i`, `sira[i] = 0` : 6
tekil küme (her kök kendini gösterir).

Adım 1 / 14

DEĞİŞKENLER

—

Kod İzleme: find + union

```

1 int find(int x) {
2   if (ebeveyn[x] != x) ebeveyn[x] = find(ebeveyn[x]);
3   return ebeveyn[x];
4 }
5 boolean union(int x, int y) {
6   int kx = find(x), ky = find(y);
7   if (kx == ky) return false;
8   if (sira[kx] < sira[ky]) ebeveyn[kx] = ky;
9   else if (sira[kx] > sira[ky]) ebeveyn[ky] = kx;
10  else { ebeveyn[kx] = ky; sira[ky]++; }
11  return true;
12 }

```



i	0	1	2	3	4	5
ebeveyn	0	1	2	3	4	5
sira	0	0	0	0	0	0

find(0): ebeveyn[0] == 0 → **0 köktür**, döndürülür.

ÇAĞRI YIĞINI

find(0)

union(0, 1)

main()

Adım 2 / 14

DEĞİŞKENLER

x = 0

Kod İzleme: find + union

```

1 int find(int x) {
2   if (ebeveyn[x] != x) ebeveyn[x] = find(ebeveyn[x]);
3   return ebeveyn[x];
4 }
5 boolean union(int x, int y) {
6   int kx = find(x), ky = find(y);
7   if (kx == ky) return false;
8   if (sira[kx] < sira[ky]) ebeveyn[kx] = ky;
9   else if (sira[kx] > sira[ky]) ebeveyn[ky] = kx;
10  else { ebeveyn[kx] = ky; sira[ky]++; }
11  return true;
12 }

```



i	0	1	2	3	4	5
ebeveyn	0	1	2	3	4	5
sira	0	0	0	0	0	0

find(1): ebeveyn[1] == 1 → **1 köktür**, döndürülür.

ÇAĞRI YIĞINI

find(1)

union(0, 1)

main()

Adım 3 / 14

DEĞİŞKENLER

x = 1

Kod İzleme: find + union

```

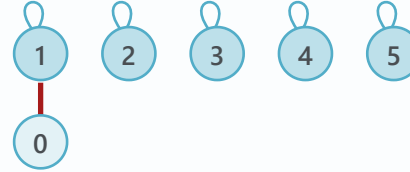
1 int find(int x) {
2   if (ebeveyn[x] != x) ebeveyn[x] = find(ebeveyn[x]);
3   return ebeveyn[x];
4 }
5 boolean union(int x, int y) {
6   int kx = find(x), ky = find(y);
7   if (kx == ky) return false;
8   if (sira[kx] < sira[ky]) ebeveyn[kx] = ky;
9   else if (sira[kx] > sira[ky]) ebeveyn[ky] = kx;
10  ▶ else { ebeveyn[kx] = ky; sira[ky]++; }
11  return true;
12  ▶ }

```

ÇAĞRI YIĞINI

union(0, 1)

main()



i	0	1	2	3	4	5
ebeveyn	1	1	2	3	4	5
sira	0	1	0	0	0	0

Rütbeler eşit (0) → ebeveyn[0] = 1 ve yeni kökün rütbesi artar: **sira[1] = 1**.

Adım 4 / 14

DEĞİŞKENLER

x = 0

y = 1

kx = 0

ky = 1

dönüş = true

Kod İzleme: find + union

```

1 int find(int x) {
2   if (ebeveyn[x] != x) ebeveyn[x] = find(ebeveyn[x]);
3   return ebeveyn[x];
4 }
5 boolean union(int x, int y) {
6   int kx = find(x), ky = find(y);
7   if (kx == ky) return false;
8   if (sira[kx] < sira[ky]) ebeveyn[kx] = ky;
9   else if (sira[kx] > sira[ky]) ebeveyn[ky] = kx;
10  else { ebeveyn[kx] = ky; sira[ky]++; }
11  return true;
12 }

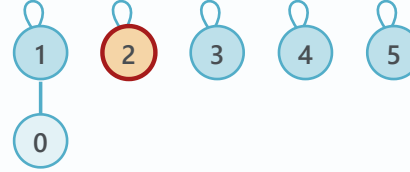
```

ÇAĞRI YIĞINI

find(2)

union(2, 3)

main()



i	0	1	2	3	4	5
ebeveyn	1	1	2	3	4	5
sira	0	1	0	0	0	0

find(2): ebeveyn[2] == 2 → **2 köktür**, döndürülür.

Adım 5 / 14

DEĞİŞKENLER

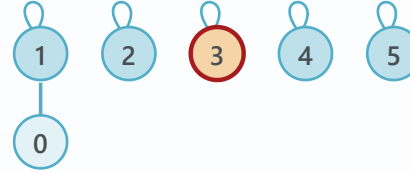
x = 2

Kod İzleme: find + union

```

1 int find(int x) {
2   if (ebeveyn[x] != x) ebeveyn[x] = find(ebeveyn[x]);
3   return ebeveyn[x];
4 }
5 boolean union(int x, int y) {
6   int kx = find(x), ky = find(y);
7   if (kx == ky) return false;
8   if (sira[kx] < sira[ky]) ebeveyn[kx] = ky;
9   else if (sira[kx] > sira[ky]) ebeveyn[ky] = kx;
10  else { ebeveyn[kx] = ky; sira[ky]++; }
11  return true;
12 }

```



i	0	1	2	3	4	5
ebeveyn	1	1	2	3	4	5
sira	0	1	0	0	0	0

find(3): ebeveyn[3] == 3 → **3 köktür**, döndürülür.

ÇAĞRI YIĞINI

find(3)

union(2, 3)

main()

Adım 6 / 14

DEĞİŞKENLER

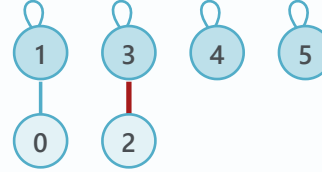
x = 3

Kod İzleme: find + union

```

1 int find(int x) {
2   if (ebeveyn[x] != x) ebeveyn[x] = find(ebeveyn[x]);
3   return ebeveyn[x];
4 }
5 boolean union(int x, int y) {
6   int kx = find(x), ky = find(y);
7   if (kx == ky) return false;
8   if (sira[kx] < sira[ky]) ebeveyn[kx] = ky;
9   else if (sira[kx] > sira[ky]) ebeveyn[ky] = kx;
10  ▶ else { ebeveyn[kx] = ky; sira[ky]++; }
11  return true;
12  ▶ }

```



i	0	1	2	3	4	5
ebeveyn	1	1	3	3	4	5
sira	0	1	0	1	0	0

Rütbeler eşit (0) → ebeveyn[2] = 3 ve yeni kökün rütbesi artar: **sira[3] = 1**.

Adım 7 / 14

ÇAĞRI YIĞINI

union(2, 3)

main()

DEĞİŞKENLER

x = 2

y = 3

kx = 2

ky = 3

dönüş = true

Kod İzleme: find + union

```

1 int find(int x) {
2     if (ebeveyn[x] != x) ebeveyn[x] = find(ebeveyn[x]);
3     return ebeveyn[x];
4 }
5 boolean union(int x, int y) {
6     int kx = find(x), ky = find(y);
7     if (kx == ky) return false;
8     if (sira[kx] < sira[ky]) ebeveyn[kx] = ky;
9     else if (sira[kx] > sira[ky]) ebeveyn[ky] = kx;
10    else { ebeveyn[kx] = ky; sira[ky]++; }
11    return true;
12 }

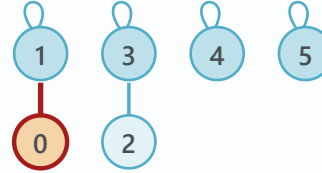
```

ÇAĞRI YIĞINI

find(0)

union(0, 2)

main()



i	0	1	2	3	4	5
ebeveyn	1	1	3	3	4	5
sira	0	1	0	1	0	0

find(0): ebeveyn[0] = 1 ≠ 0 → kök değil;

find(1) ile yukarı çıkılır.

Adım 8 / 14

DEĞİŞKENLER

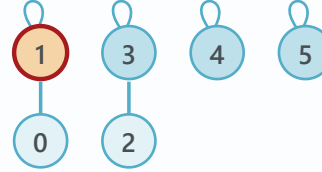
x = 0

Kod İzleme: find + union

```

1 int find(int x) {
2   if (ebeveyn[x] != x) ebeveyn[x] = find(ebeveyn[x]);
3   return ebeveyn[x];
4 }
5 boolean union(int x, int y) {
6   int kx = find(x), ky = find(y);
7   if (kx == ky) return false;
8   if (sira[kx] < sira[ky]) ebeveyn[kx] = ky;
9   else if (sira[kx] > sira[ky]) ebeveyn[ky] = kx;
10  else { ebeveyn[kx] = ky; sira[ky]++; }
11  return true;
12 }

```



i	0	1	2	3	4	5
ebeveyn	1	1	3	3	4	5
sira	0	1	0	1	0	0

find(1): ebeveyn[1] == 1 → **1 köktür**, döndürülür.

ÇAĞRI YIĞINI

find(1)

find(0)

union(0, 2)

main()

Adım 9 / 14

DEĞİŞKENLER

x = 1

Kod İzleme: find + union

```

1 int find(int x) {
2     if (ebeveyn[x] != x) ebeveyn[x] = find(ebeveyn[x]);
3     return ebeveyn[x];
4 }
5 boolean union(int x, int y) {
6     int kx = find(x), ky = find(y);
7     if (kx == ky) return false;
8     if (sira[kx] < sira[ky]) ebeveyn[kx] = ky;
9     else if (sira[kx] > sira[ky]) ebeveyn[ky] = kx;
10    else { ebeveyn[kx] = ky; sira[ky]++; }
11    return true;
12 }

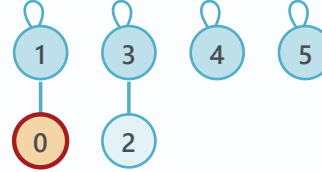
```

ÇAĞRI YIĞINI

find(0)

union(0, 2)

main()



i	0	1	2	3	4	5
ebeveyn	1	1	3	3	4	5
sira	0	1	0	1	0	0

Dönüşte ebeveyn[0] = 1 (zaten kökün çocuğu, değişmez); find(0) = 1 döner.

Adım 10 / 14

DEĞİŞKENLER

x = 0

Kod İzleme: find + union

```

1 int find(int x) {
2     if (ebeveyn[x] != x) ebeveyn[x] = find(ebeveyn[x]);
3     return ebeveyn[x];
4 }
5 boolean union(int x, int y) {
6     int kx = find(x), ky = find(y);
7     if (kx == ky) return false;
8     if (sira[kx] < sira[ky]) ebeveyn[kx] = ky;
9     else if (sira[kx] > sira[ky]) ebeveyn[ky] = kx;
10    else { ebeveyn[kx] = ky; sira[ky]++; }
11    return true;
12 }

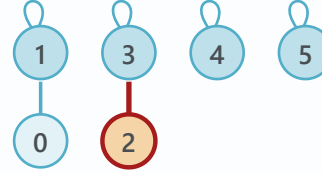
```

ÇAĞRI YIĞINI

find(2)

union(0, 2)

main()



i	0	1	2	3	4	5
ebeveyn	1	1	3	3	4	5
sira	0	1	0	1	0	0

find(2): ebeveyn[2] = 3 ≠ 2 → kök değil;
 find(3) ile yukarı çıkılır.

Adım 11 / 14

DEĞİŞKENLER

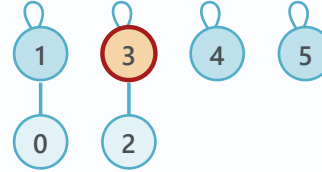
x = 2

Kod İzleme: find + union

```

1 int find(int x) {
2   if (ebeveyn[x] != x) ebeveyn[x] = find(ebeveyn[x]);
3   return ebeveyn[x];
4 }
5 boolean union(int x, int y) {
6   int kx = find(x), ky = find(y);
7   if (kx == ky) return false;
8   if (sira[kx] < sira[ky]) ebeveyn[kx] = ky;
9   else if (sira[kx] > sira[ky]) ebeveyn[ky] = kx;
10  else { ebeveyn[kx] = ky; sira[ky]++; }
11  return true;
12 }

```



i	0	1	2	3	4	5
ebeveyn	1	1	3	3	4	5
sira	0	1	0	1	0	0

find(3): ebeveyn[3] == 3 → **3 köktür**, döndürülür.

ÇAĞRI YIĞINI

find(3)

find(2)

union(0, 2)

main()

Adım 12 / 14

DEĞİŞKENLER

x = 3

Kod İzleme: find + union

```

1 int find(int x) {
2     if (ebeveyn[x] != x) ebeveyn[x] = find(ebeveyn[x]);
3     return ebeveyn[x];
4 }
5 boolean union(int x, int y) {
6     int kx = find(x), ky = find(y);
7     if (kx == ky) return false;
8     if (sira[kx] < sira[ky]) ebeveyn[kx] = ky;
9     else if (sira[kx] > sira[ky]) ebeveyn[ky] = kx;
10    else { ebeveyn[kx] = ky; sira[ky]++; }
11    return true;
12 }

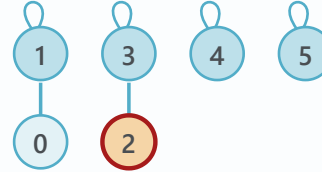
```

ÇAĞRI YIĞINI

find(2)

union(0, 2)

main()



i	0	1	2	3	4	5
ebeveyn	1	1	3	3	4	5
sira	0	1	0	1	0	0

Dönüşte ebeveyn[2] = 3 (zaten kökün çocuğu, değişmez); find(2) = 3 döner.

Adım 13 / 14

DEĞİŞKENLER

x = 2

Kod İzleme: find + union

```

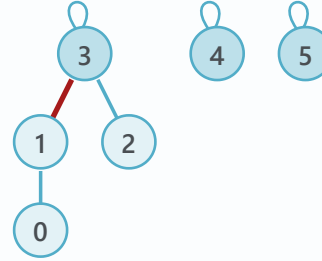
1 int find(int x) {
2   if (ebeveyn[x] != x) ebeveyn[x] = find(ebeveyn[x]);
3   return ebeveyn[x];
4 }
5 boolean union(int x, int y) {
6   int kx = find(x), ky = find(y);
7   if (kx == ky) return false;
8   if (sira[kx] < sira[ky]) ebeveyn[kx] = ky;
9   else if (sira[kx] > sira[ky]) ebeveyn[ky] = kx;
10  ▶ else { ebeveyn[kx] = ky; sira[ky]++; }
11  return true;
12  ▶ }

```

ÇAĞRI YIĞINI

```
union(0, 2)
```

```
main()
```



i	0	1	2	3	4	5
ebeveyn	1	3	3	3	4	5
sıra	0	1	0	2	0	0

Rütbeler eşit (1) → ebeveyn[1] = 3 ve yeni kökün rütbesi artar: **sıra[3] = 2**.

Adım 14 / 14

DEĞİŞKENLER

x = 0

y = 2

kx = 1

ky = 3

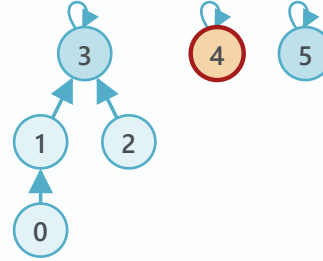
dönüş = true

Kod İzleme: find + union (devam)

```

1 int find(int x) {
2   if (ebeveyn[x] != x) ebeveyn[x] = find(ebeveyn[x]);
3   return ebeveyn[x];
4 }
5 boolean union(int x, int y) {
6   int kx = find(x), ky = find(y);
7   if (kx == ky) return false;
8   if (sira[kx] < sira[ky]) ebeveyn[kx] = ky;
9   else if (sira[kx] > sira[ky]) ebeveyn[ky] = kx;
10  else { ebeveyn[kx] = ky; sira[ky]++; }
11  return true;
12 }

```



i	0	1	2	3	4	5
ebeveyn	1	3	3	3	4	5
sira	0	1	0	2	0	0

ÇAĞRI YIĞINI

find(4)

union(4, 0)

main()

find(4): ebeveyn[4] == 4 → **4 köktür**, döndürülür.

Adım 1 / 15

DEĞİŞKENLER

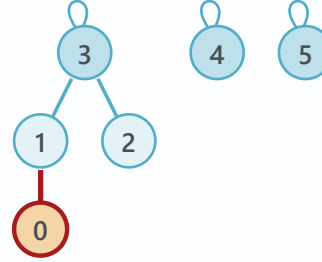
x = 4

Kod İzleme: find + union (devam)

```

1 int find(int x) {
2     if (ebeveyn[x] != x) ebeveyn[x] = find(ebeveyn[x]);
3     return ebeveyn[x];
4 }
5 boolean union(int x, int y) {
6     int kx = find(x), ky = find(y);
7     if (kx == ky) return false;
8     if (sira[kx] < sira[ky]) ebeveyn[kx] = ky;
9     else if (sira[kx] > sira[ky]) ebeveyn[ky] = kx;
10    else { ebeveyn[kx] = ky; sira[ky]++; }
11    return true;
12 }

```



i	0	1	2	3	4	5
ebeveyn	1	3	3	3	4	5
sira	0	1	0	2	0	0

find(0): ebeveyn[0] = 1 ≠ 0 → kök değil;
 find(1) ile yukarı çıkılır.

Adım 2 / 15

ÇAĞRI YIĞINI

find(0)

union(4, 0)

main()

DEĞİŞKENLER

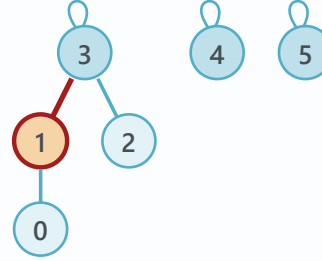
x = 0

Kod İzleme: find + union (devam)

```

1 int find(int x) {
2     if (ebeveyn[x] != x) ebeveyn[x] = find(ebeveyn[x]);
3     return ebeveyn[x];
4 }
5 boolean union(int x, int y) {
6     int kx = find(x), ky = find(y);
7     if (kx == ky) return false;
8     if (sira[kx] < sira[ky]) ebeveyn[kx] = ky;
9     else if (sira[kx] > sira[ky]) ebeveyn[ky] = kx;
10    else { ebeveyn[kx] = ky; sira[ky]++; }
11    return true;
12 }

```



i	0	1	2	3	4	5
ebeveyn	1	3	3	3	4	5
sira	0	1	0	2	0	0

ÇAĞRI YIĞINI

find(1)

find(0)

union(4, 0)

main()

find(1): ebeveyn[1] = 3 ≠ 1 → kök değil;
 find(3) ile yukarı çıkılır.

Adım 3 / 15

DEĞİŞKENLER

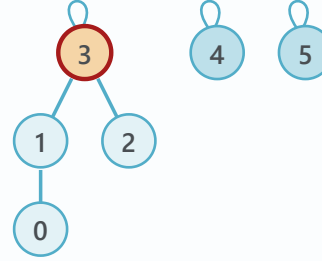
x = 1

Kod İzleme: find + union (devam)

```

1 int find(int x) {
2   if (ebeveyn[x] != x) ebeveyn[x] = find(ebeveyn[x]);
3   return ebeveyn[x];
4 }
5 boolean union(int x, int y) {
6   int kx = find(x), ky = find(y);
7   if (kx == ky) return false;
8   if (sira[kx] < sira[ky]) ebeveyn[kx] = ky;
9   else if (sira[kx] > sira[ky]) ebeveyn[ky] = kx;
10  else { ebeveyn[kx] = ky; sira[ky]++; }
11  return true;
12 }

```



i	0	1	2	3	4	5
ebeveyn	1	3	3	3	4	5
sira	0	1	0	2	0	0

ÇAĞRI YIĞINI

find(3)

find(1)

find(0)

union(4, 0)

main()

find(3): ebeveyn[3] == 3 → **3 köktür**, döndürülür.

Adım 4 / 15

DEĞİŞKENLER

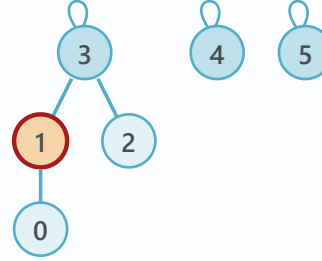
x = 3

Kod İzleme: find + union (devam)

```

1 int find(int x) {
2     if (ebeveyn[x] != x) ebeveyn[x] = find(ebeveyn[x]);
3     return ebeveyn[x];
4 }
5 boolean union(int x, int y) {
6     int kx = find(x), ky = find(y);
7     if (kx == ky) return false;
8     if (sira[kx] < sira[ky]) ebeveyn[kx] = ky;
9     else if (sira[kx] > sira[ky]) ebeveyn[ky] = kx;
10    else { ebeveyn[kx] = ky; sira[ky]++; }
11    return true;
12 }

```



i	0	1	2	3	4	5
ebeveyn	1	3	3	3	4	5
sira	0	1	0	2	0	0

Dönüşte ebeveyn[1] = 3 (zaten kökün çocuğu, değişmez); find(1) = 3 döner.

Adım 5 / 15

ÇAĞRI YIĞINI

find(1)

find(0)

union(4, 0)

main()

DEĞİŞKENLER

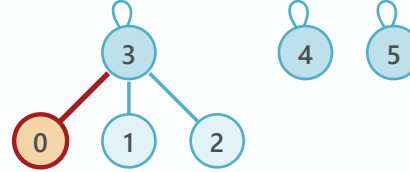
x = 1

Kod İzleme: find + union (devam)

```

1 int find(int x) {
2     if (ebeveyn[x] != x) ebeveyn[x] = find(ebeveyn[x]);
3     return ebeveyn[x];
4 }
5 boolean union(int x, int y) {
6     int kx = find(x), ky = find(y);
7     if (kx == ky) return false;
8     if (sira[kx] < sira[ky]) ebeveyn[kx] = ky;
9     else if (sira[kx] > sira[ky]) ebeveyn[ky] = kx;
10    else { ebeveyn[kx] = ky; sira[ky]++; }
11    return true;
12 }

```



i	0	1	2	3	4	5
ebeveyn	3	3	3	3	4	5
sira	0	1	0	2	0	0

ÇAĞRI YIĞINI

find(0)

union(4, 0)

main()

Yol kısaltma: dönüşte ebeveyn[0] = 1 → **3**; 0 artık doğrudan köke bağlı. find(0) = 3.

Adım 6 / 15

DEĞİŞKENLER

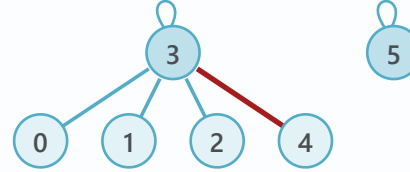
x = 0

Kod İzleme: find + union (devam)

```

1 int find(int x) {
2   if (ebeveyn[x] != x) ebeveyn[x] = find(ebeveyn[x]);
3   return ebeveyn[x];
4 }
5 boolean union(int x, int y) {
6   int kx = find(x), ky = find(y);
7   if (kx == ky) return false;
8   ► if (sira[kx] < sıra[ky]) ebeveyn[kx] = ky;
9   else if (sira[kx] > sıra[ky]) ebeveyn[ky] = kx;
10  else { ebeveyn[kx] = ky; sıra[ky]++; }
11  return true;
12 ► }

```



i	0	1	2	3	4	5
ebeveyn	3	3	3	3	3	5
sıra	0	1	0	2	0	0

sıra[4] = 0 < sıra[3] = 2 → alçak ağacın kökü 4, 3'e bağlanır: ebeveyn[4] = 3. Yükseklik artmaz.

Adım 7 / 15

ÇAĞRI YIĞINI

union(4, 0)

main()

DEĞİŞKENLER

x = 4

y = 0

kx = 4

ky = 3

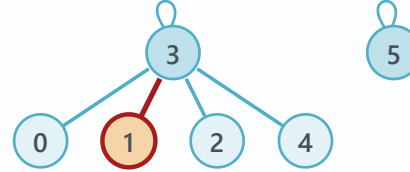
dönüş = true

Kod İzleme: find + union (devam)

```

1 int find(int x) {
2     if (ebeveyn[x] != x) ebeveyn[x] = find(ebeveyn[x]);
3     return ebeveyn[x];
4 }
5 boolean union(int x, int y) {
6     int kx = find(x), ky = find(y);
7     if (kx == ky) return false;
8     if (sira[kx] < sira[ky]) ebeveyn[kx] = ky;
9     else if (sira[kx] > sira[ky]) ebeveyn[ky] = kx;
10    else { ebeveyn[kx] = ky; sira[ky]++; }
11    return true;
12 }

```



i	0	1	2	3	4	5
ebeveyn	3	3	3	3	3	5
sira	0	1	0	2	0	0

find(1): ebeveyn[1] = 3 $\neq$ 1 $\rightarrow$ kök değil;
 find(3) ile yukarı çıkılır.

Adım 8 / 15

ÇAĞRI YIĞINI

find(1)

union(1, 4)

main()

DEĞİŞKENLER

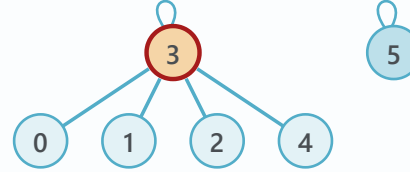
x = 1

Kod İzleme: find + union (devam)

```

1 int find(int x) {
2   if (ebeveyn[x] != x) ebeveyn[x] = find(ebeveyn[x]);
3   return ebeveyn[x];
4 }
5 boolean union(int x, int y) {
6   int kx = find(x), ky = find(y);
7   if (kx == ky) return false;
8   if (sira[kx] < sira[ky]) ebeveyn[kx] = ky;
9   else if (sira[kx] > sira[ky]) ebeveyn[ky] = kx;
10  else { ebeveyn[kx] = ky; sira[ky]++; }
11  return true;
12 }

```



i	0	1	2	3	4	5
ebeveyn	3	3	3	3	3	5
sira	0	1	0	2	0	0

ÇAĞRI YIĞINI

find(3)

find(1)

union(1, 4)

main()

find(3): ebeveyn[3] == 3 → **3 köktür**, döndürülür.

Adım 9 / 15

DEĞİŞKENLER

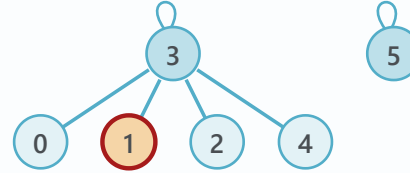
x = 3

Kod İzleme: find + union (devam)

```

1 int find(int x) {
  ► if (ebeveyn[x] != x) ebeveyn[x] = find(ebeveyn[x]);
  ► return ebeveyn[x];
4 }
5 boolean union(int x, int y) {
6   int kx = find(x), ky = find(y);
7   if (kx == ky) return false;
8   if (sira[kx] < sira[ky]) ebeveyn[kx] = ky;
9   else if (sira[kx] > sira[ky]) ebeveyn[ky] = kx;
10  else { ebeveyn[kx] = ky; sira[ky]++; }
11  return true;
12 }

```



i	0	1	2	3	4	5
ebeveyn	3	3	3	3	3	5
sira	0	1	0	2	0	0

Dönüşte ebeveyn[1] = 3 (zaten kökün çocuğu, değişmez); find(1) = 3 döner.

Adım 10 / 15

ÇAĞRI YIĞINI

find(1)

union(1, 4)

main()

DEĞİŞKENLER

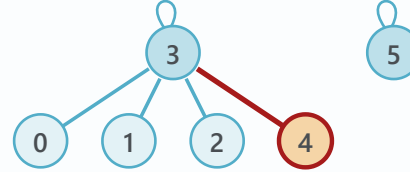
x = 1

Kod İzleme: find + union (devam)

```

1 int find(int x) {
2     if (ebeveyn[x] != x) ebeveyn[x] = find(ebeveyn[x]);
3     return ebeveyn[x];
4 }
5 boolean union(int x, int y) {
6     int kx = find(x), ky = find(y);
7     if (kx == ky) return false;
8     if (sira[kx] < sira[ky]) ebeveyn[kx] = ky;
9     else if (sira[kx] > sira[ky]) ebeveyn[ky] = kx;
10    else { ebeveyn[kx] = ky; sira[ky]++; }
11    return true;
12 }

```



i	0	1	2	3	4	5
ebeveyn	3	3	3	3	3	5
sira	0	1	0	2	0	0

ÇAĞRI YIĞINI

find(4)

union(1, 4)

main()

find(4): ebeveyn[4] = 3 ≠ 4 → kök değil;
find(3) ile yukarı çıkılır.

Adım 11 / 15

DEĞİŞKENLER

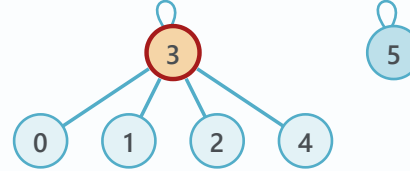
x = 4

Kod İzleme: find + union (devam)

```

1 int find(int x) {
2   if (ebeveyn[x] != x) ebeveyn[x] = find(ebeveyn[x]);
3   return ebeveyn[x];
4 }
5 boolean union(int x, int y) {
6   int kx = find(x), ky = find(y);
7   if (kx == ky) return false;
8   if (sira[kx] < sira[ky]) ebeveyn[kx] = ky;
9   else if (sira[kx] > sira[ky]) ebeveyn[ky] = kx;
10  else { ebeveyn[kx] = ky; sira[ky]++; }
11  return true;
12 }

```



i	0	1	2	3	4	5
ebeveyn	3	3	3	3	3	5
sira	0	1	0	2	0	0

ÇAĞRI YIĞINI

find(3)

find(4)

union(1, 4)

main()

find(3): ebeveyn[3] == 3 → **3 köktür**, döndürülür.

Adım 12 / 15

DEĞİŞKENLER

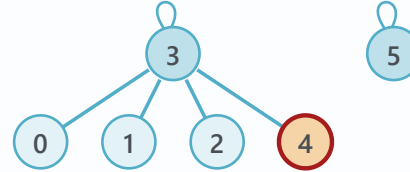
x = 3

Kod İzleme: find + union (devam)

```

1 int find(int x) {
2     if (ebeveyn[x] != x) ebeveyn[x] = find(ebeveyn[x]);
3     return ebeveyn[x];
4 }
5 boolean union(int x, int y) {
6     int kx = find(x), ky = find(y);
7     if (kx == ky) return false;
8     if (sira[kx] < sira[ky]) ebeveyn[kx] = ky;
9     else if (sira[kx] > sira[ky]) ebeveyn[ky] = kx;
10    else { ebeveyn[kx] = ky; sira[ky]++; }
11    return true;
12 }

```



i	0	1	2	3	4	5
ebeveyn	3	3	3	3	3	5
sira	0	1	0	2	0	0

Dönüşte ebeveyn[4] = 3 (zaten kökün çocuğu, değişmez); find(4) = 3 döner.

Adım 13 / 15

ÇAĞRI YIĞINI

find(4)

union(1, 4)

main()

DEĞİŞKENLER

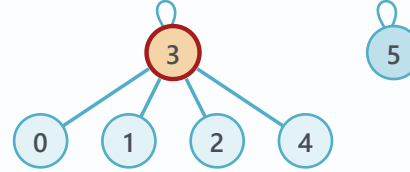
x = 4

Kod İzleme: find + union (devam)

```

1 int find(int x) {
2   if (ebeveyn[x] != x) ebeveyn[x] = find(ebeveyn[x]);
3   return ebeveyn[x];
4 }
5 boolean union(int x, int y) {
6   int kx = find(x), ky = find(y);
7   ► if (kx == ky) return false;
8   if (sira[kx] < sira[ky]) ebeveyn[kx] = ky;
9   else if (sira[kx] > sira[ky]) ebeveyn[ky] = kx;
10  else { ebeveyn[kx] = ky; sira[ky]++; }
11  return true;
12 }

```



i	0	1	2	3	4	5
ebeveyn	3	3	3	3	3	5
sira	0	1	0	2	0	0

$kx = ky = 3 \rightarrow 1$ ve 4 **zaten aynı kümede**:
return false (Kruskal: döngü).

Adım 14 / 15

ÇAĞRI YIĞINI

```
union(1, 4)
```

```
main()
```

DEĞİŞKENLER

$x = 1$

$y = 4$

$kx = 3$

$ky = 3$

dönüş = **false**

Kod İzleme: find + union (devam)

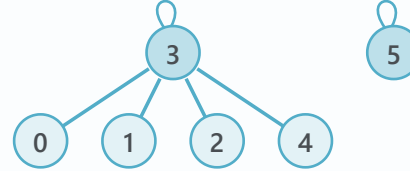
```

1 int find(int x) {
2   if (ebeveyn[x] != x) ebeveyn[x] = find(ebeveyn[x]);
3   return ebeveyn[x];
4 }
5 boolean union(int x, int y) {
6   int kx = find(x), ky = find(y);
7   if (kx == ky) return false;
8   if (sira[kx] < sira[ky]) ebeveyn[kx] = ky;
9   else if (sira[kx] > sira[ky]) ebeveyn[ky] = kx;
10  else { ebeveyn[kx] = ky; sira[ky]++; }
11  return true;
12 }

```

ÇAĞRI YIĞINI

main()



i	0	1	2	3	4	5
ebeveyn	3	3	3	3	3	5
sıra	0	1	0	2	0	0

Tek küme, kök 3. Yol kısaltma 0'ı köke taşıdı;
 sıra[3] = 2 yalnızca üst sınır. Birlikte: işlem başına
 $\approx O(\alpha(n))$.

Adım 15 / 15

DEĞİŞKENLER

—

Kod İzleme: döngüyle **find** + yol kısaltma



i	0	1	2	3	4
ebeveyn	0	0	1	2	3

Zincir $4 \rightarrow 3 \rightarrow 2 \rightarrow 1 \rightarrow 0$ (kötü birleştirmeler sonucu). **find(4)** önce kökü arar: **kok = 4**.

Adım 1 / 16

```

1 int find(int x) {
2     int kok = x;
3     while (ebeveyn[kok] != kok) kok = ebeveyn[kok];
4     while (ebeveyn[x] != kok) { // yol kısaltma
5         int sonraki = ebeveyn[x];
6         ebeveyn[x] = kok;
7         x = sonraki;
8     }
9     return kok;
10 }
```

DEĞİŞKENLER

x = 4

kok = 4

ebeveyn = [0, 0, 1, 2, 3]

ÇAĞRI YIĞINI

find(4)

main()

Kod İzleme: döngüyle **find** + yol kısaltma



i	0	1	2	3	4
ebeveyn	0	0	1	2	3

1. tur: $ebeveyn[4] = 3 \neq 4 \rightarrow$ yukarı çık, **kok = 3**.

Adım 2 / 16

```

1 int find(int x) {
2   int kok = x;
3   while (ebeveyn[kok] != kok) kok = ebeveyn[kok];
4   while (ebeveyn[x] != kok) {    // yol kısaltma
5     int sonraki = ebeveyn[x];
6     ebeveyn[x] = kok;
7     x = sonraki;
8   }
9   return kok;
10 }
```

DEĞİŞKENLER

x = 4

kok = 3

ebeveyn = [0, 0, 1, 2, 3]

ÇAĞRI YIĞINI

find(4)

main()

Kod İzleme: döngüyle **find** + yol kısaltma



i	0	1	2	3	4
ebeveyn	0	0	1	2	3

1. tur: $ebeveyn[3] = 2 \neq 3 \rightarrow$ yukarı çık, **kok = 2**.

Adım 3 / 16

```

1 int find(int x) {
2   int kok = x;
3   ► while (ebeveyn[kok] != kok) kok = ebeveyn[kok];
4   while (ebeveyn[x] != kok) {    // yol kısaltma
5     int sonraki = ebeveyn[x];
6     ebeveyn[x] = kok;
7     x = sonraki;
8   }
9   return kok;
10 }
```

DEĞİŞKENLER

x = 4

kok = 2

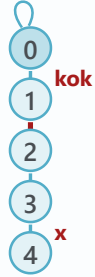
ebeveyn = [0, 0, 1, 2, 3]

ÇAĞRI YIĞINI

find(4)

main()

Kod İzleme: döngüyle **find** + yol kısaltma



i	0	1	2	3	4
ebeveyn	0	0	1	2	3

1. tur: $ebeveyn[2] = 1 \neq 2 \rightarrow$ yukarı çık, **kok = 1**.

Adım 4 / 16

```

1 int find(int x) {
2   int kok = x;
3   ► while (ebeveyn[kok] != kok) kok = ebeveyn[kok];
4   while (ebeveyn[x] != kok) {    // yol kısaltma
5     int sonraki = ebeveyn[x];
6     ebeveyn[x] = kok;
7     x = sonraki;
8   }
9   return kok;
10 }
```

DEĞİŞKENLER

x = 4

kok = 1

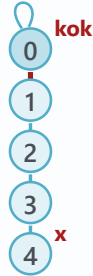
ebeveyn = [0, 0, 1, 2, 3]

ÇAĞRI YIĞINI

find(4)

main()

Kod İzleme: döngüyle **find** + yol kısaltma



i	0	1	2	3	4
ebeveyn	0	0	1	2	3

ebeveyn[1] = 0 → **kok** = 0; ebeveyn[0] = 0 olduğundan döngü biter: kök bulundu (4 adım).

Adım 5 / 16

```

1 int find(int x) {
2   int kok = x;
3   ► while (ebeveyn[kok] != kok) kok = ebeveyn[kok];
4   while (ebeveyn[x] != kok) {    // yol kısaltma
5     int sonraki = ebeveyn[x];
6     ebeveyn[x] = kok;
7     x = sonraki;
8   }
9   return kok;
10 }
```

DEĞİŞKENLER

x = 4

kok = 0

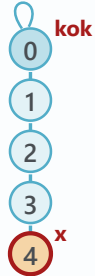
ebeveyn = [0, 0, 1, 2, 3]

ÇAĞRI YIĞINI

find(4)

main()

Kod İzleme: döngüyle **find** + yol kısaltma



i	0	1	2	3	4
ebeveyn	0	0	1	2	3

2. tur: `ebeveyn[4] = 3`, kök değil → bu düğüm kısaltılacak.

Adım 6 / 16

```

1 int find(int x) {
2   int kok = x;
3   while (ebeveyn[kok] != kok) kok = ebeveyn[kok];
4   ► while (ebeveyn[x] != kok) { // yol kısaltma
5     int sonraki = ebeveyn[x];
6     ebeveyn[x] = kok;
7     x = sonraki;
8   }
9   return kok;
10 }
```

DEĞİŞKENLER

`x = 4`

`kok = 0`

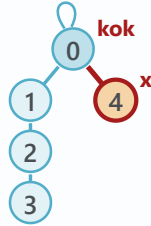
`ebeveyn = [0, 0, 1, 2, 3]`

ÇAĞRI YIĞINI

`find(4)`

`main()`

Kod İzleme: döngüyle **find** + yol kısaltma



i	0	1	2	3	4
ebeveyn	0	0	1	2	0

Önce eski ebeveyn saklanır (**sonraki = 3**), sonra **ebeveyn[4] = 0**: 4 artık doğrudan köke bağlı.

Adım 7 / 16

```

1 int find(int x) {
2   int kok = x;
3   while (ebeveyn[kok] != kok) kok = ebeveyn[kok];
4   while (ebeveyn[x] != kok) { // yol kısaltma
5     ▶ int sonraki = ebeveyn[x];
6     ▶ ebeveyn[x] = kok;
7     x = sonraki;
8   }
9   return kok;
10 }
```

DEĞİŞKENLER

x = 4

kok = 0

sonraki = 3

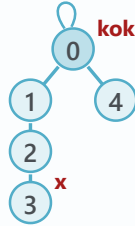
ebeveyn = [0, 0, 1, 2, 0]

ÇAĞRI YIĞINI

find(4)

main()

Kod İzleme: döngüyle **find** + yol kısaltma



i	0	1	2	3	4
ebeveyn	0	0	1	2	0

Yolda bir yukarı: **x = 3**.

Adım 8 / 16

```

1 int find(int x) {
2   int kok = x;
3   while (ebeveyn[kok] != kok) kok = ebeveyn[kok];
4   while (ebeveyn[x] != kok) {    // yol kısaltma
5     int sonraki = ebeveyn[x];
6     ebeveyn[x] = kok;
7     x = sonraki;
8   }
9   return kok;
10 }
```

DEĞİŞKENLER

x = 3

kok = 0

sonraki = 3

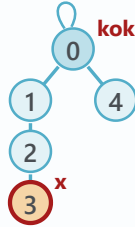
ebeveyn = [0, 0, 1, 2, 0]

ÇAĞRI YIĞINI

find(4)

main()

Kod İzleme: döngüyle **find** + yol kısaltma



i	0	1	2	3	4
ebeveyn	0	0	1	2	0

2. tur: $ebeveyn[3] = 2$, kök değil → bu düğüm kısaltılacak.

Adım 9 / 16

```

1 int find(int x) {
2   int kok = x;
3   while (ebeveyn[kok] != kok) kok = ebeveyn[kok];
4   ► while (ebeveyn[x] != kok) { // yol kısaltma
5     int sonraki = ebeveyn[x];
6     ebeveyn[x] = kok;
7     x = sonraki;
8   }
9   return kok;
10 }
```

DEĞİŞKENLER

x = 3

kok = 0

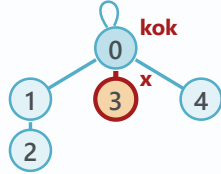
ebeveyn = [0, 0, 1, 2, 0]

ÇAĞRI YIĞINI

find(4)

main()

Kod İzleme: döngüyle **find** + yol kısaltma



i	0	1	2	3	4
ebeveyn	0	0	1	0	0

Önce eski ebeveyn saklanır (**sonraki = 2**), sonra **ebeveyn[3] = 0**: 3 artık doğrudan köke bağlı.

Adım 10 / 16

```

1 int find(int x) {
2   int kok = x;
3   while (ebeveyn[kok] != kok) kok = ebeveyn[kok];
4   while (ebeveyn[x] != kok) { // yol kısaltma
5     ▶ int sonraki = ebeveyn[x];
6     ▶ ebeveyn[x] = kok;
7     x = sonraki;
8   }
9   return kok;
10 }
```

DEĞİŞKENLER

x = 3 kok = 0 sonraki = 2

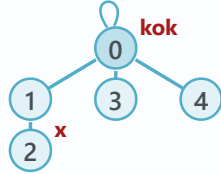
ebeveyn = [0, 0, 1, 0, 0]

ÇAĞRI YIĞINI

find(4)

main()

Kod İzleme: döngüyle **find** + yol kısaltma



i	0	1	2	3	4
ebeveyn	0	0	1	0	0

Yolda bir yukarı: **x = 2**.

Adım 11 / 16

```

1 int find(int x) {
2   int kok = x;
3   while (ebeveyn[kok] != kok) kok = ebeveyn[kok];
4   while (ebeveyn[x] != kok) {    // yol kısaltma
5     int sonraki = ebeveyn[x];
6     ebeveyn[x] = kok;
7     x = sonraki;
8   }
9   return kok;
10 }
```

DEĞİŞKENLER

x = 2

kok = 0

sonraki = 2

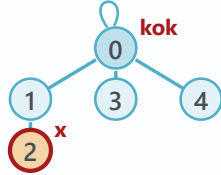
ebeveyn = [0, 0, 1, 0, 0]

ÇAĞRI YIĞINI

find(4)

main()

Kod İzleme: döngüyle **find** + yol kısaltma



i	0	1	2	3	4
ebeveyn	0	0	1	0	0

2. tur: $ebeveyn[2] = 1$, kök değil → bu düğüm kısaltılacak.

Adım 12 / 16

```

1 int find(int x) {
2   int kok = x;
3   while (ebeveyn[kok] != kok) kok = ebeveyn[kok];
4   ► while (ebeveyn[x] != kok) { // yol kısaltma
5     int sonraki = ebeveyn[x];
6     ebeveyn[x] = kok;
7     x = sonraki;
8   }
9   return kok;
10 }
```

DEĞİŞKENLER

x = 2

kok = 0

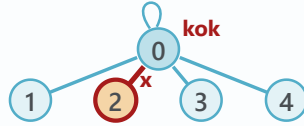
ebeveyn = [0, 0, 1, 0, 0]

ÇAĞRI YIĞINI

find(4)

main()

Kod İzleme: döngüyle **find** + yol kısaltma



i	0	1	2	3	4
ebeveyn	0	0	0	0	0

Önce eski ebeveyn saklanır (**sonraki = 1**), sonra **ebeveyn[2] = 0**: 2 artık doğrudan köke bağlı.

Adım 13 / 16

```

1 int find(int x) {
2   int kok = x;
3   while (ebeveyn[kok] != kok) kok = ebeveyn[kok];
4   while (ebeveyn[x] != kok) {    // yol kısaltma
5     ▶ int sonraki = ebeveyn[x];
6     ▶ ebeveyn[x] = kok;
7     x = sonraki;
8   }
9   return kok;
10 }
```

DEĞİŞKENLER

x = 2 kok = 0 sonraki = 1

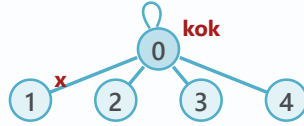
ebeveyn = [0, 0, 0, 0, 0]

ÇAĞRI YIĞINI

find(4)

main()

Kod İzleme: döngüyle **find** + yol kısaltma



i	0	1	2	3	4
ebeveyn	0	0	0	0	0

Yolda bir yukarı: **x = 1**.

Adım 14 / 16

```

1 int find(int x) {
2   int kok = x;
3   while (ebeveyn[kok] != kok) kok = ebeveyn[kok];
4   while (ebeveyn[x] != kok) {    // yol kısaltma
5     int sonraki = ebeveyn[x];
6     ebeveyn[x] = kok;
7     x = sonraki;
8   }
9   return kok;
10 }
```

DEĞİŞKENLER

x = 1

kok = 0

sonraki = 1

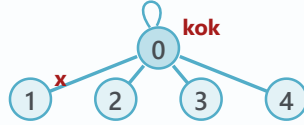
ebeveyn = [0, 0, 0, 0, 0]

ÇAĞRI YIĞINI

find(4)

main()

Kod İzleme: döngüyle **find** + yol kısaltma



i	0	1	2	3	4
ebeveyn	0	0	0	0	0

ebeveyn[1] = 0 = kok: 1 zaten köke bağlı, döngü biter.

Adım 15 / 16

```

1 int find(int x) {
2   int kok = x;
3   while (ebeveyn[kok] != kok) kok = ebeveyn[kok];
4   ► while (ebeveyn[x] != kok) { // yol kısaltma
5     int sonraki = ebeveyn[x];
6     ebeveyn[x] = kok;
7     x = sonraki;
8   }
9   return kok;
10 }
```

DEĞİŞKENLER

x = 1

kok = 0

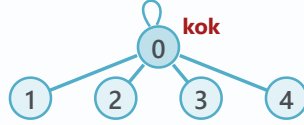
ebeveyn = [0, 0, 0, 0, 0]

ÇAĞRI YIĞINI

find(4)

main()

Kod İzleme: döngüyle find + yol kısaltma



i	0	1	2	3	4
ebeveyn	0	0	0	0	0

Kök **0** döner. Yol üzerindeki herkes köke bağlandı: sonraki find'lar 1 adım. Özyineleme yok → yığın taşması riski de yok.

Adım 16 / 16

```

1 int find(int x) {
2   int kok = x;
3   while (ebeveyn[kok] != kok) kok = ebeveyn[kok];
4   while (ebeveyn[x] != kok) { // yol kısaltma
5     int sonraki = ebeveyn[x];
6     ebeveyn[x] = kok;
7     x = sonraki;
8   }
9   return kok;
10 }
```

DEĞİŞKENLER

x = 1

kok = 0

ebeveyn = [0, 0, 0, 0, 0]

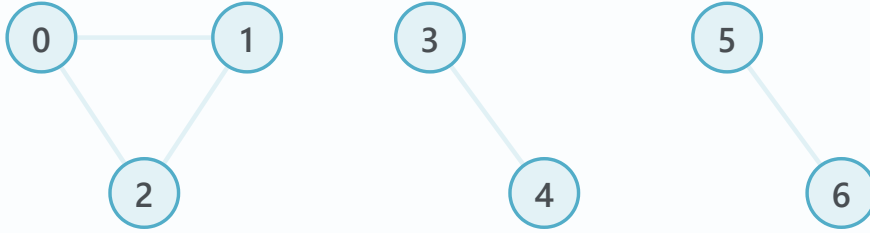
dönüş = 0

ÇAĞRI YIĞINI

find(4)

main()

Kod İzleme: Union-Find ile bağlantılı bileşen sayısı



i	0	1	2	3	4	5	6
ebeveyn	0	1	2	3	4	5	6
sıra	0	0	0	0	0	0	0

say = 7

7 düğüm, 5 kenar. `UnionFind(7)` : her düğüm kendi kümesinin kökü (`ebeveyn[i] = i`).

Adım 1 / 12

```

1 int bilesenSay(int n, int[][] kenar) {
2     ▶ UnionFind uf = new UnionFind(n);
3     int say = n;           // n tekil küme
4     for (int[] e : kenar)
5         if (uf.union(e[0], e[1]))
6             say--;         // iki bileşen birleşti
7     return say;
8 }
  
```

DEĞİŞKENLER

`n = 7`

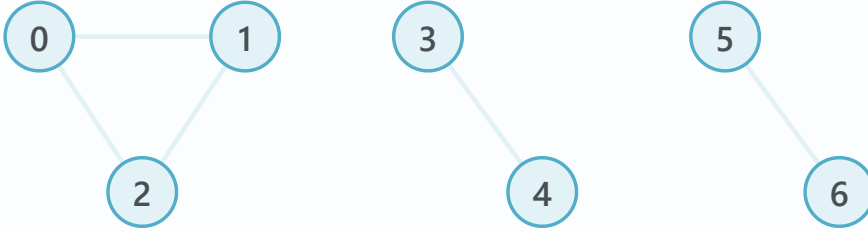
`kenar` `[[0,1], [1,2], [3,4], [0,2], [5,6]]`

ÇAĞRI YIĞINI

`bilesenSay(7, kenar)`

`main()`

Kod İzleme: Union-Find ile bağlantılı bileşen sayısı



i	0	1	2	3	4	5	6
ebeveyn	0	1	2	3	4	5	6
sıra	0	0	0	0	0	0	0

say = 7

Başta hiç kenar yok: **7 bileşen**.

Adım 2 / 12

```

1 int bilesenSay(int n, int[][] kenar) {
2   UnionFind uf = new UnionFind(n);
3   int say = n;           // n tekil küme
4   for (int[] e : kenar)
5     if (uf.union(e[0], e[1]))
6       say--;             // iki bileşen birleşti
7   return say;
8 }

```

DEĞİŞKENLER

n = 7

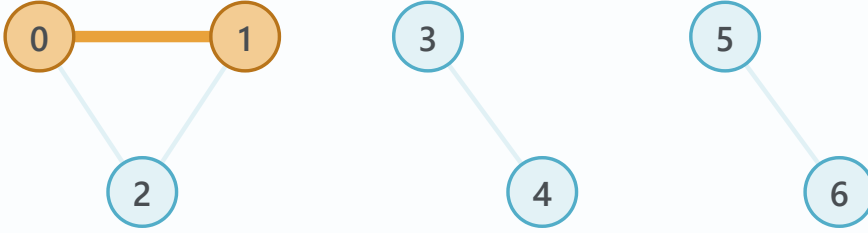
say = 7

ÇAĞRI YIĞINI

bilesenSay(7, kenar)

main()

Kod İzleme: Union-Find ile bağlantılı bileşen sayısı



i	0	1	2	3	4	5	6
ebeveyn	1	1	2	3	4	5	6
sıra	0	1	0	0	0	0	0

say = 7

Kenar (0, 1): kökler farklı → **union** kümeleri birleştirir, **true** döner.

Adım 3 / 12

```

1 int bilesenSay(int n, int[][] kenar) {
2   UnionFind uf = new UnionFind(n);
3   int say = n;          // n tekil küme
4   for (int[] e : kenar)
5     if (uf.union(e[0], e[1]))
6       say--;            // iki bileşen birleşti
7   return say;
8 }
  
```

DEĞİŞKENLER

n = 7

say = 7

e = [0, 1]

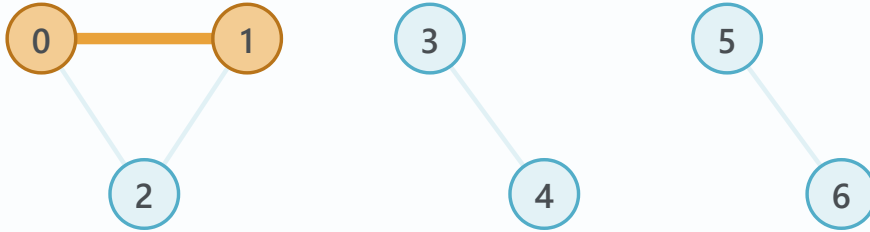
union = true

ÇAĞRI YIĞINI

bilesenSay(7, kenar)

main()

Kod İzleme: Union-Find ile bağlantılı bileşen sayısı



i	0	1	2	3	4	5	6
ebeveyn	1	1	2	3	4	5	6
sıra	0	1	0	0	0	0	0

say = 6

İki bileşen bire indi: **say = 6**.

Adım 4 / 12

```

1 int bilesenSay(int n, int[][] kenar) {
2   UnionFind uf = new UnionFind(n);
3   int say = n;           // n tekil küme
4   for (int[] e : kenar)
5     if (uf.union(e[0], e[1]))
6       say--;             // iki bileşen birleşti
7   return say;
8 }

```

DEĞİŞKENLER

n = 7 say = 6 e = [0, 1]

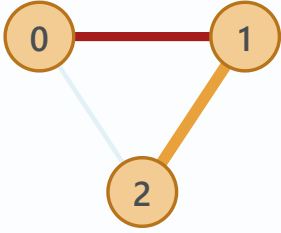
union = true

ÇAĞRI YIĞINI

bilesenSay(7, kenar)

main()

Kod İzleme: Union-Find ile bağlantılı bileşen sayısı



i	0	1	2	3	4	5	6
ebeveyn	1	1	1	3	4	5	6
sıra	0	1	0	0	0	0	0

say = 6

Kenar (1, 2): kökler farklı → **union** kümeleri birleştirir, **true** döner.

Adım 5 / 12

```

1 int bilesenSay(int n, int[][] kenar) {
2   UnionFind uf = new UnionFind(n);
3   int say = n;           // n tekil küme
4   for (int[] e : kenar)
5     if (uf.union(e[0], e[1]))
6       say--;             // iki bileşen birleşti
7   return say;
8 }

```

DEĞİŞKENLER

n = 7

say = 6

e = [1, 2]

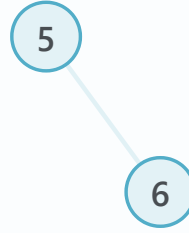
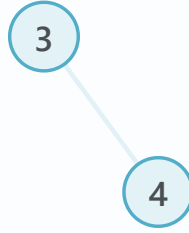
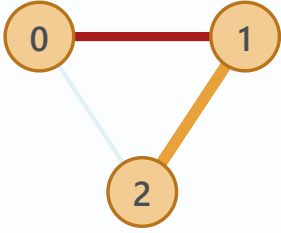
union = true

ÇAĞRI YIĞINI

bilesenSay(7, kenar)

main()

Kod İzleme: Union-Find ile bağlantılı bileşen sayısı



i	0	1	2	3	4	5	6
ebeveyn	1	1	1	3	4	5	6
sıra	0	1	0	0	0	0	0

say = 5

İki bileşen bire indi: **say = 5**.

Adım 6 / 12

```

1 int bilesenSay(int n, int[][] kenar) {
2   UnionFind uf = new UnionFind(n);
3   int say = n;           // n tekil küme
4   for (int[] e : kenar)
5     if (uf.union(e[0], e[1]))
6       say--;             // iki bileşen birleşti
7   return say;
8 }

```

DEĞİŞKENLER

n = 7

say = 5

e = [1, 2]

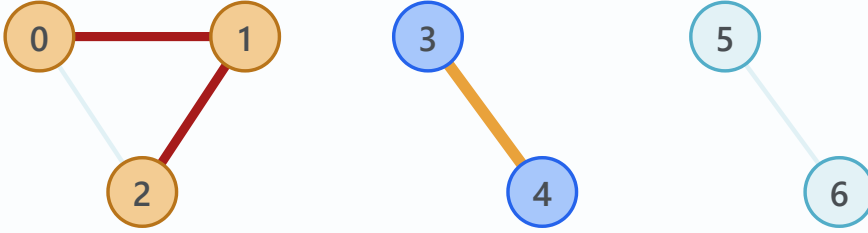
union = true

ÇAĞRI YIĞINI

bilesenSay(7, kenar)

main()

Kod İzleme: Union-Find ile bağlantılı bileşen sayısı



i	0	1	2	3	4	5	6
ebeveyn	1	1	1	4	4	5	6
sıra	0	1	0	0	1	0	0

say = 5

Kenar (3, 4): kökler farklı → **union** kümeleri birleştirir, **true** döner.

Adım 7 / 12

```

1 int bilesenSay(int n, int[][] kenar) {
2   UnionFind uf = new UnionFind(n);
3   int say = n;           // n tekil küme
4   for (int[] e : kenar)
5     if (uf.union(e[0], e[1]))
6       say--;             // iki bileşen birleşti
7   return say;
8 }

```

DEĞİŞKENLER

n = 7

say = 5

e = [3, 4]

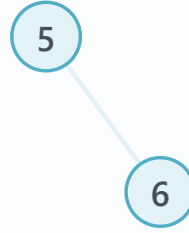
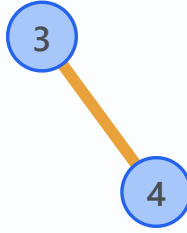
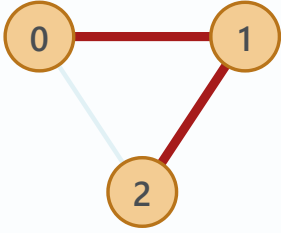
union = true

ÇAĞRI YIĞINI

bilesenSay(7, kenar)

main()

Kod İzleme: Union-Find ile bağlantılı bileşen sayısı



i	0	1	2	3	4	5	6
ebeveyn	1	1	1	4	4	5	6
sıra	0	1	0	0	1	0	0

say = 4

İki bileşen bire indi: **say = 4**.

Adım 8 / 12

```

1 int bilesenSay(int n, int[][] kenar) {
2   UnionFind uf = new UnionFind(n);
3   int say = n;           // n tekil küme
4   for (int[] e : kenar)
5     if (uf.union(e[0], e[1]))
6       say--;             // iki bileşen birleşti
7   return say;
8 }

```

DEĞİŞKENLER

n = 7

say = 4

e = [3, 4]

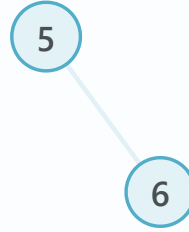
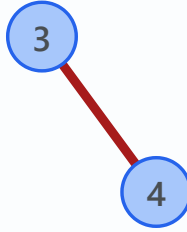
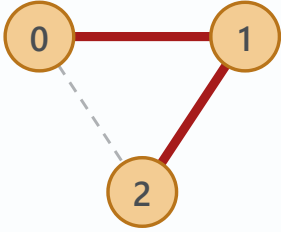
union = true

ÇAĞRI YIĞINI

bilesenSay(7, kenar)

main()

Kod İzleme: Union-Find ile bağlantılı bileşen sayısı



i	0	1	2	3	4	5	6
ebeveyn	1	1	1	4	4	5	6
sıra	0	1	0	0	1	0	0

say = 4

Kenar (0, 2): $\text{find}(0) = \text{find}(2) \rightarrow$ zaten **aynı bileşende**; **union** false, say değişmez (bu kenar bir döngü kapatır).

Adım 9 / 12

```

1 int bilestenSay(int n, int[][] kenar) {
2   UnionFind uf = new UnionFind(n);
3   int say = n;           // n tekil küme
4   for (int[] e : kenar)
5     if (uf.union(e[0], e[1]))
6       say--;             // iki bileşen birleşti
7   return say;
8 }

```

DEĞİŞKENLER

n = 7

say = 4

e = [0, 2]

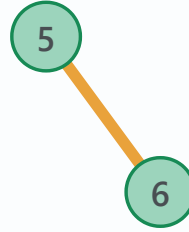
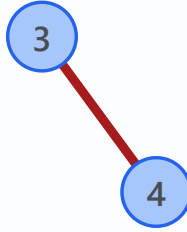
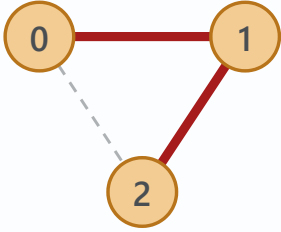
union = false

ÇAĞRI YIĞINI

bilestenSay(7, kenar)

main()

Kod İzleme: Union-Find ile bağlantılı bileşen sayısı



i	0	1	2	3	4	5	6
ebeveyn	1	1	1	4	4	6	6
sıra	0	1	0	0	1	0	1

say = 4

Kenar (5, 6): kökler farklı → **union** kümeleri birleştirir, **true** döner.

Adım 10 / 12

```

1 int bilesenSay(int n, int[][] kenar) {
2   UnionFind uf = new UnionFind(n);
3   int say = n;           // n tekil küme
4   for (int[] e : kenar)
5     if (uf.union(e[0], e[1]))
6       say--;             // iki bileşen birleşti
7   return say;
8 }

```

DEĞİŞKENLER

n = 7

say = 4

e = [5, 6]

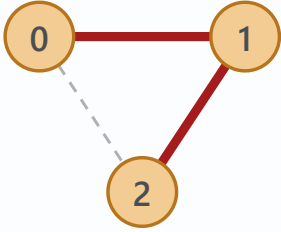
union = true

ÇAĞRI YIĞINI

bilesenSay(7, kenar)

main()

Kod İzleme: Union-Find ile bağlantılı bileşen sayısı



i	0	1	2	3	4	5	6
ebeveyn	1	1	1	4	4	6	6
sıra	0	1	0	0	1	0	1

say = 3

İki bileşen bire indi: **say = 3**.

Adım 11 / 12

```

1 int bilesenSay(int n, int[][] kenar) {
2   UnionFind uf = new UnionFind(n);
3   int say = n;           // n tekil küme
4   for (int[] e : kenar)
5     if (uf.union(e[0], e[1]))
6       say--;             // iki bileşen birleşti
7   return say;
8 }

```

DEĞİŞKENLER

n = 7

say = 3

e = [5, 6]

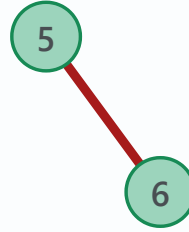
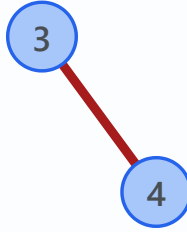
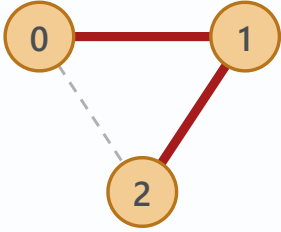
union = true

ÇAĞRI YIĞINI

bilesenSay(7, kenar)

main()

Kod İzleme: Union-Find ile bağlantılı bileşen sayısı



i	0	1	2	3	4	5	6
ebeveyn	1	1	1	4	4	6	6
sıra	0	1	0	0	1	0	1

say = 3

Sonuç **3 bileşen**: {0, 1, 2}, {3, 4}, {5, 6}. Toplam $O(n + m \cdot \alpha(n))$; kenarlar sonradan gelse de sayaç anında güncel (dinamik bağlılık).

Adım 12 / 12

```

1 int bilesenSay(int n, int[][] kenar) {
2   UnionFind uf = new UnionFind(n);
3   int say = n;           // n tekil küme
4   for (int[] e : kenar)
5     if (uf.union(e[0], e[1]))
6       say--;             // iki bileşen birleşti
7   return say;
8 }

```

DEĞİŞKENLER

n = 7

say = 3

dönüş = 3

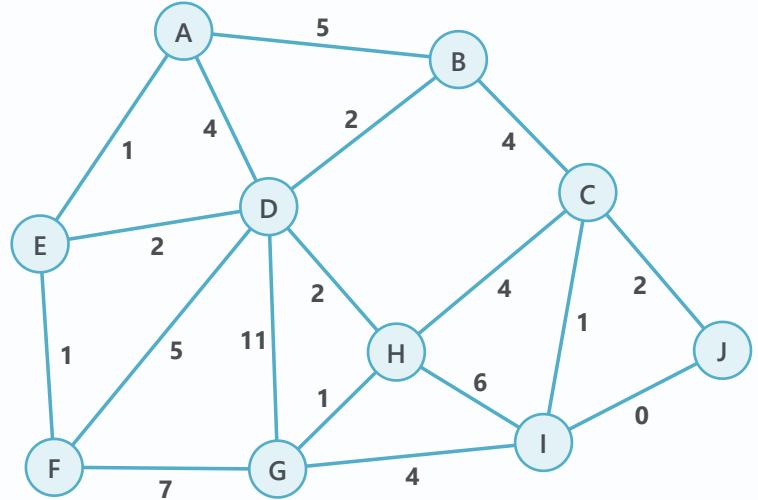
ÇAĞRI YIĞINI

bilesenSay(7, kenar)

main()

Minimum Kapsayan Ağaç (Minimum Spanning Tree)

- **Kapsayan ağaç** (spanning tree): bağlantılı, ağırlıklı bir çizgenin **tüm düğümlerini** içeren ve döngü içermeyen alt çizgesi; n düğüm için $n - 1$ kenar.
- **Minimum kapsayan ağaç** (MKA, MST): toplam kenar ağırlığı **en düşük** olan kapsayan ağaç.
- Uygulama: şehirleri en az kabloyla / boruyla birbirine bağlamak.
- Kruskal algoritması, "bu kenar döngü oluşturur mu?" sorusunu **Union-Find** ile yanıtlar.



Kruskal Algoritması

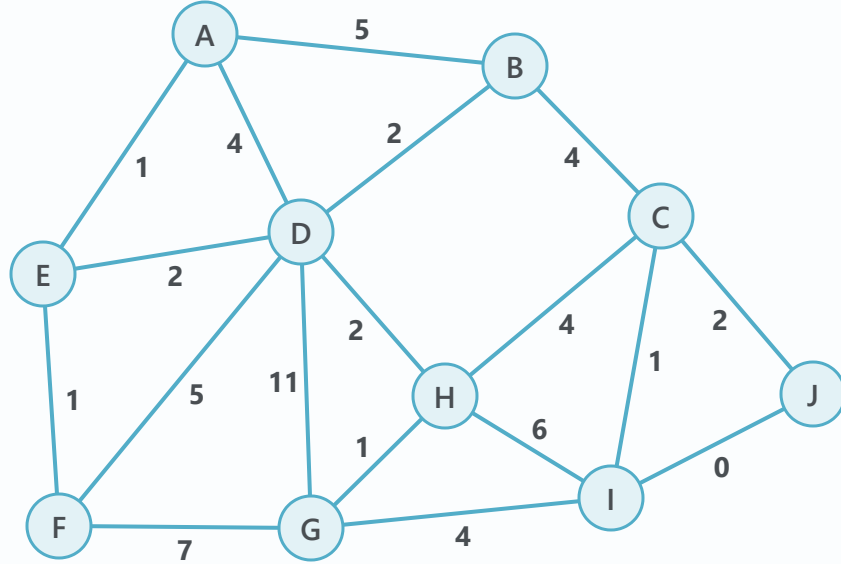
```
Kruskal(G):  
  T = ∅           // seçilen kenarlar  
  for her düğüm v: makeSet(v)  
  kenarları ağırlığa göre sırala  
  for her (u, v), küçükten büyüğe:  
    if find(u) != find(v): // döngü yok  
      T.ekle((u, v))  
      union(u, v)  
    if |T| == n - 1: break  
  return T // MST (bağlı değilse: orman)
```

- **Açgözlü:** her zaman döngü oluşturmeyen en hafif kenarı seç.
- u ve v zaten aynı kümedeyse, kenar bir **döngü** kapatır → atla.
- Başta n ayrık küme; her kabul edilen kenar iki kümeyi birleştirir.
- Çizge bağlantısızsa n-1 kenara ulaşamaz; algoritma her bağlantılı bileşen için bir ağaçtan oluşan **minimum kapsayan orman** döndürür.

Sıralama	$O(m \log m)$
$2m$ find + $(n-1)$ union	$O(m \cdot \alpha(n))$
Toplam	$O(m \log n)$

Kruskal Algoritması Adım Adım

I-J = 0
 A-E = 1
 C-I = 1
 E-F = 1
 G-H = 1
 B-D = 2
 C-J = 2
 D-E = 2
 D-H = 2
 A-D = 4
 B-C = 4
 C-H = 4
 G-I = 4
 A-B = 5
 D-F = 5
 H-I = 6
 F-G = 7
 D-G = 11



Seçilen kenar

0 / 9

Toplam ağırlık

0

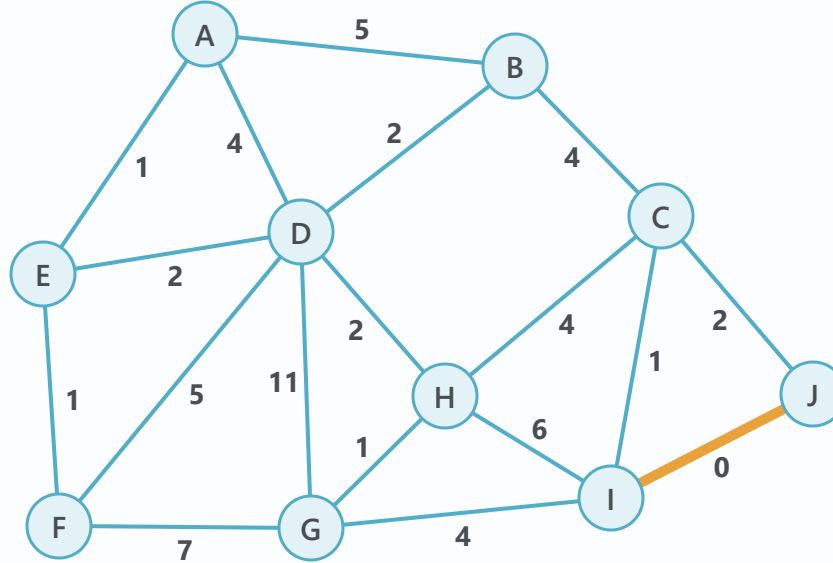
Kenarlar ağırlığa göre sıralandı (solda). Başlangıçta 10 düğüm = 10 ayrı küme, T boş.

Adım 1 / 13

Kruskal Algoritması Adım Adım

I-J = 0 ✓

A-E = 1
 C-I = 1
 E-F = 1
 G-H = 1
 B-D = 2
 C-J = 2
 D-E = 2
 D-H = 2
 A-D = 4
 B-C = 4
 C-H = 4
 G-I = 4
 A-B = 5
 D-F = 5
 H-I = 6
 F-G = 7
 D-G = 11



Seçilen kenar

1 / 9

Toplam ağırlık

0

I-J (0): $\text{find}(I) \neq \text{find}(J) \rightarrow$ kabul, kümeler birleşir.

Adım 2 / 13

Kruskal Algoritması Adım Adım

I-J = 0 ✓

A-E = 1 ✓

C-I = 1

E-F = 1

G-H = 1

B-D = 2

C-J = 2

D-E = 2

D-H = 2

A-D = 4

B-C = 4

C-H = 4

G-I = 4

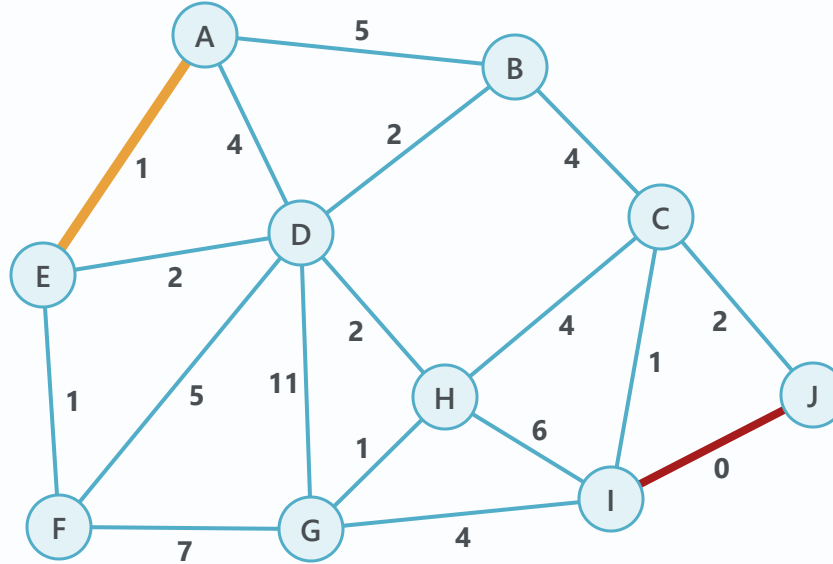
A-B = 5

D-F = 5

H-I = 6

F-G = 7

D-G = 11



Seçilen kenar

2 / 9

Toplam ağırlık

1

A-E (1): find(A) ≠ find(E) → kabul, kümeler birleşir.

Adım 3 / 13

Kruskal Algoritması Adım Adım

I-J = 0 ✓

A-E = 1 ✓

C-I = 1 ✓

E-F = 1

G-H = 1

B-D = 2

C-J = 2

D-E = 2

D-H = 2

A-D = 4

B-C = 4

C-H = 4

G-I = 4

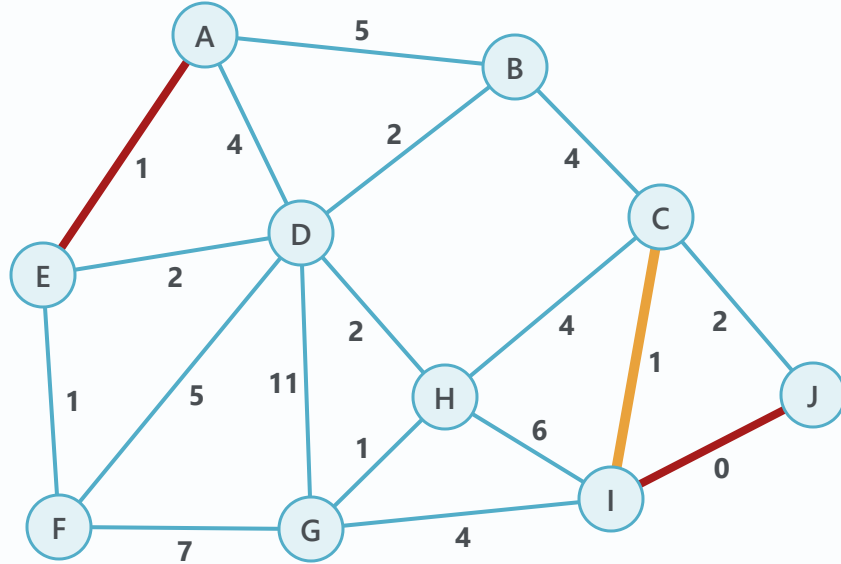
A-B = 5

D-F = 5

H-I = 6

F-G = 7

D-G = 11



Seçilen kenar

3 / 9

Toplam ağırlık

2

C-I (1): $\text{find}(C) \neq \text{find}(I) \rightarrow$ kabul, kümeler birleşir.

Kruskal Algoritması Adım Adım

I-J = 0 ✓

A-E = 1 ✓

C-I = 1 ✓

E-F = 1 ✓

G-H = 1

B-D = 2

C-J = 2

D-E = 2

D-H = 2

A-D = 4

B-C = 4

C-H = 4

G-I = 4

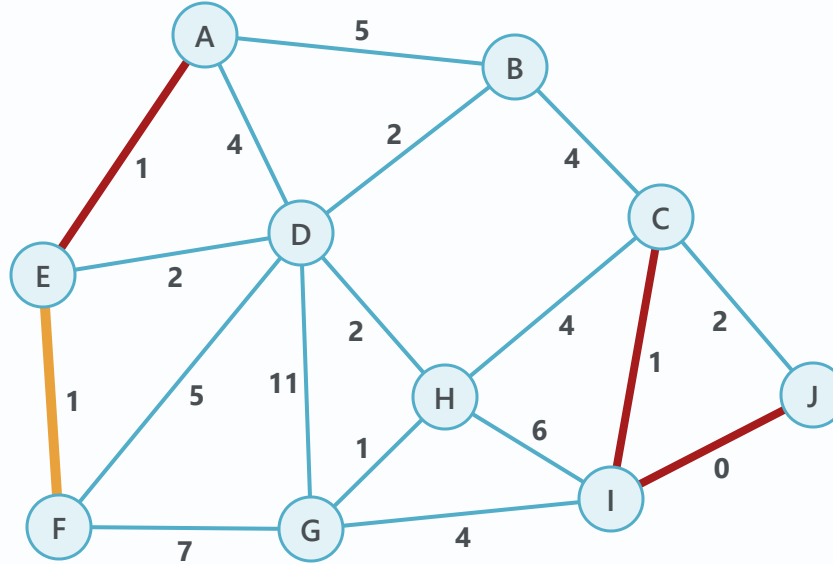
A-B = 5

D-F = 5

H-I = 6

F-G = 7

D-G = 11



Seçilen kenar

4 / 9

Toplam ağırlık

3

E-F (1): find(E) ≠ find(F) → kabul, kümeler birleşir.

Adım 5 / 13

Kruskal Algoritması Adım Adım

I-J = 0 ✓

A-E = 1 ✓

C-I = 1 ✓

E-F = 1 ✓

G-H = 1 ✓

B-D = 2

C-J = 2

D-E = 2

D-H = 2

A-D = 4

B-C = 4

C-H = 4

G-I = 4

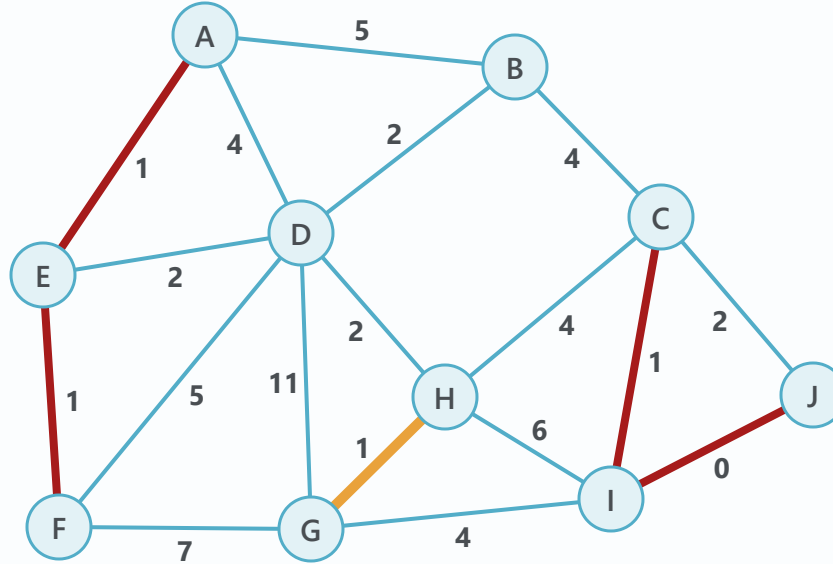
A-B = 5

D-F = 5

H-I = 6

F-G = 7

D-G = 11



Seçilen kenar

5 / 9

Toplam ağırlık

4

G-H (1): find(G) ≠ find(H) → kabul, kümeler birleşir.

Adım 6 / 13

Kruskal Algoritması Adım Adım

I-J = 0 ✓

A-E = 1 ✓

C-I = 1 ✓

E-F = 1 ✓

G-H = 1 ✓

B-D = 2 ✓

C-J = 2

D-E = 2

D-H = 2

A-D = 4

B-C = 4

C-H = 4

G-I = 4

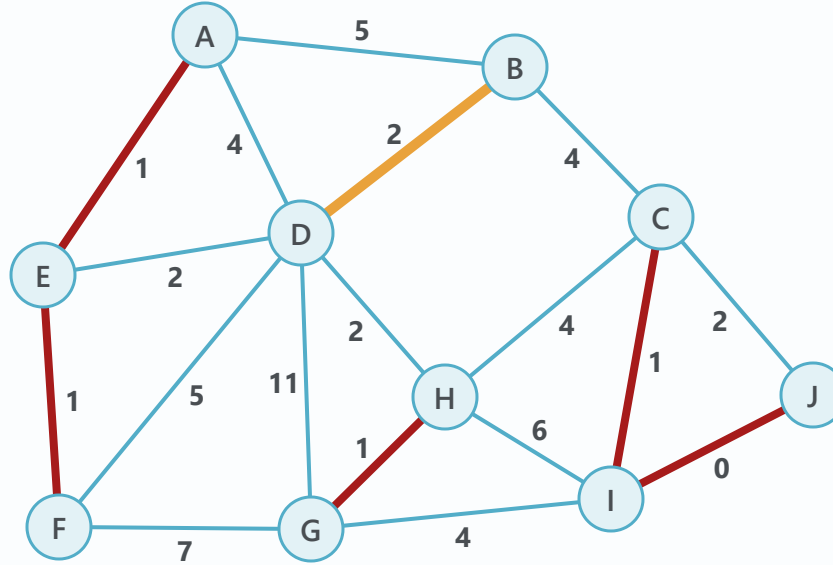
A-B = 5

D-F = 5

H-I = 6

F-G = 7

D-G = 11



Seçilen kenar

6 / 9

Toplam ağırlık

6

B-D (2): $\text{find}(B) \neq \text{find}(D) \rightarrow$ kabul, kümeler birleşir.

Adım 7 / 13

Kruskal Algoritması Adım Adım

I-J = 0 ✓

A-E = 1 ✓

C-I = 1 ✓

E-F = 1 ✓

G-H = 1 ✓

B-D = 2 ✓

~~C-J = 2 ✗~~

D-E = 2

D-H = 2

A-D = 4

B-C = 4

C-H = 4

G-I = 4

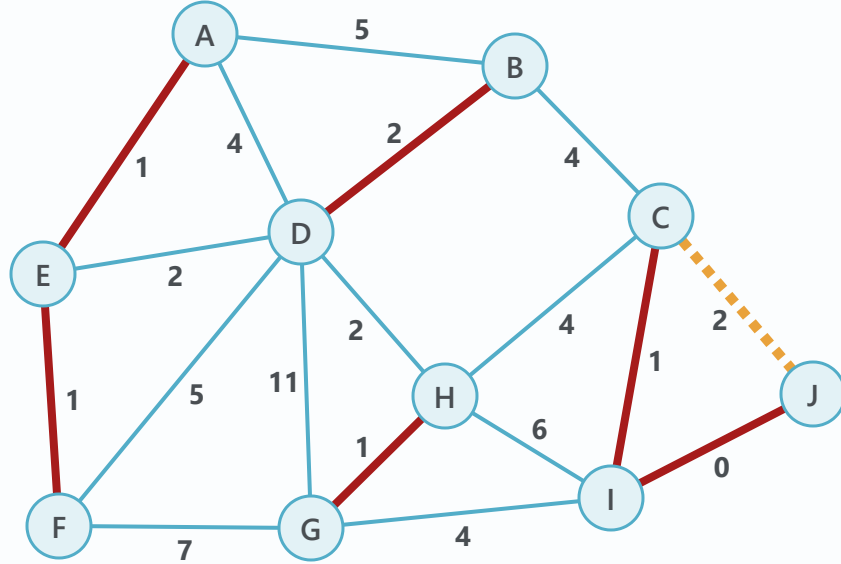
A-B = 5

D-F = 5

H-I = 6

F-G = 7

D-G = 11



Seçilen kenar

6 / 9

Toplam ağırlık

6

C-J (2): C ve J zaten aynı kümede → döngü oluşturur, **reddedilir**.

Kruskal Algoritması Adım Adım

I-J = 0 ✓

A-E = 1 ✓

C-I = 1 ✓

E-F = 1 ✓

G-H = 1 ✓

B-D = 2 ✓

~~C-J = 2 ✗~~**D-E = 2 ✓**

D-H = 2

A-D = 4

B-C = 4

C-H = 4

G-I = 4

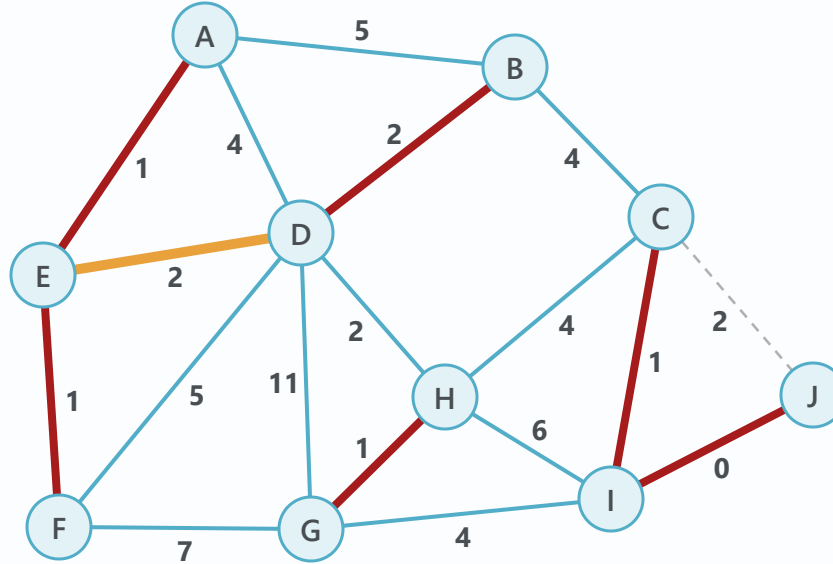
A-B = 5

D-F = 5

H-I = 6

F-G = 7

D-G = 11



Seçilen kenar

7 / 9

Toplam ağırlık

8

D-E (2): find(D) ≠ find(E) → kabul, kümeler birleşir.

Kruskal Algoritması Adım Adım

I-J = 0 ✓

A-E = 1 ✓

C-I = 1 ✓

E-F = 1 ✓

G-H = 1 ✓

B-D = 2 ✓

~~C-J = 2 ✗~~

D-E = 2 ✓

D-H = 2 ✓

A-D = 4

B-C = 4

C-H = 4

G-I = 4

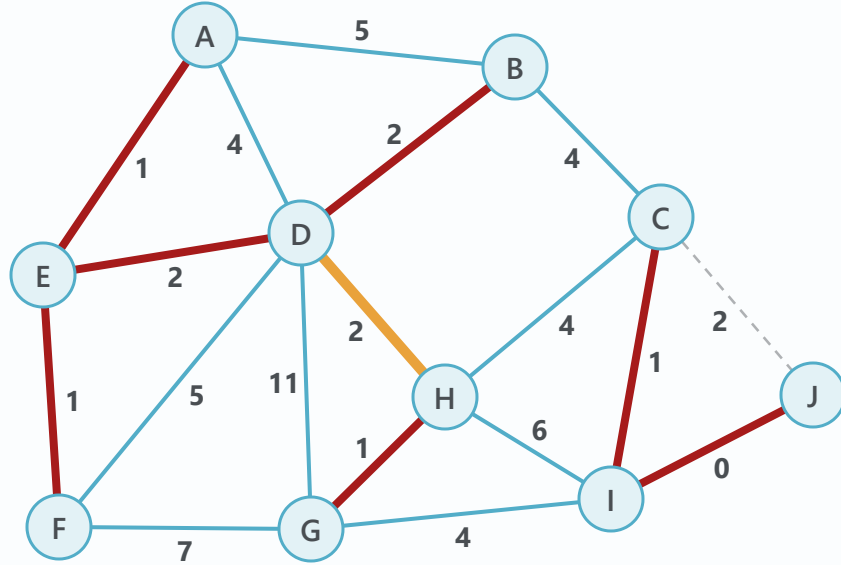
A-B = 5

D-F = 5

H-I = 6

F-G = 7

D-G = 11



Seçilen kenar

8 / 9

Toplam ağırlık

10

D-H (2): find(D) ≠ find(H) → kabul, kümeler birleşir.

Adım 10 / 13

Kruskal Algoritması Adım Adım

I-J = 0 ✓

A-E = 1 ✓

C-I = 1 ✓

E-F = 1 ✓

G-H = 1 ✓

B-D = 2 ✓

~~C-J = 2 ✗~~

D-E = 2 ✓

D-H = 2 ✓

~~A-D = 4 ✗~~

B-C = 4

C-H = 4

G-I = 4

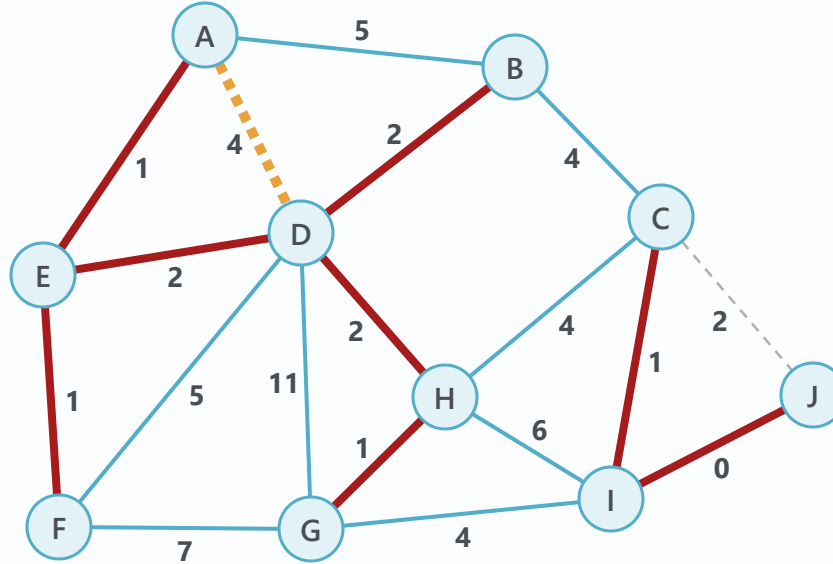
A-B = 5

D-F = 5

H-I = 6

F-G = 7

D-G = 11



Seçilen kenar

8 / 9

Toplam ağırlık

10

A-D (4): A ve D zaten aynı kümede → döngü oluşturur, **reddedilir**.

Kruskal Algoritması Adım Adım

I-J = 0 ✓

A-E = 1 ✓

C-I = 1 ✓

E-F = 1 ✓

G-H = 1 ✓

B-D = 2 ✓

~~C-J = 2 ✗~~

D-E = 2 ✓

D-H = 2 ✓

~~A-D = 4 ✗~~**B-C = 4 ✓**

C-H = 4

G-I = 4

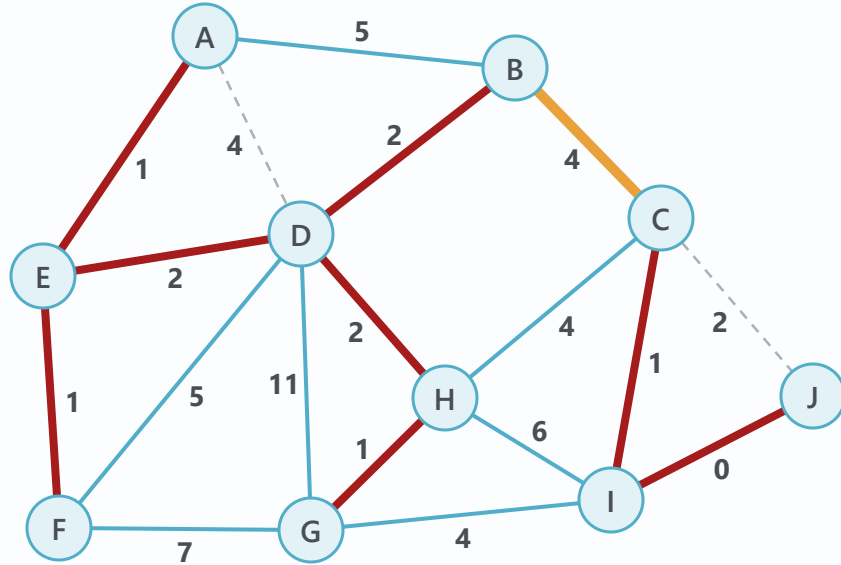
A-B = 5

D-F = 5

H-I = 6

F-G = 7

D-G = 11



Seçilen kenar

9 / 9

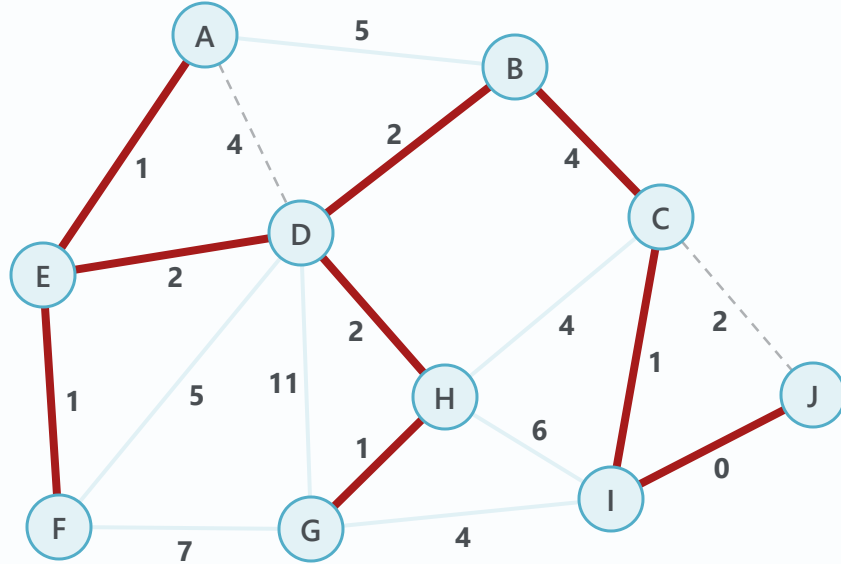
Toplam ağırlık

14

B-C (4): $\text{find}(B) \neq \text{find}(C) \rightarrow$ kabul, kümeler birleşir. $9 = n - 1$ kenar seçildi: **bitti**, toplam ağırlık 14.

Kruskal Algoritması Adım Adım

I-J = 0 ✓
 A-E = 1 ✓
 C-I = 1 ✓
 E-F = 1 ✓
 G-H = 1 ✓
 B-D = 2 ✓
~~C-J = 2 ✗~~
 D-E = 2 ✓
 D-H = 2 ✓
~~A-D = 4 ✗~~
B-C = 4 ✓
 C-H = 4
 G-I = 4
 A-B = 5
 D-F = 5
 H-I = 6
 F-G = 7
 D-G = 11



Seçilen kenar
9 / 9

Toplam ağırlık
14

Minimum kapsayan ağaç: A-E, E-F, D-E, B-D, D-H, G-H, B-C, C-I, I-J. Kalan kenarlara bakmaya gerek yok.

Kod İzleme: `kruskal()` + Union-Find

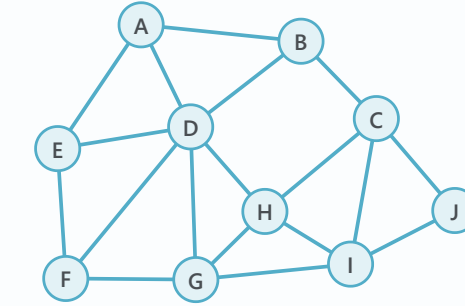
```

1 List<int[]> kruskal(int n, List<int[]> E) {
    ▶ E.sort((a, b) ->           // {u, v, w}
    ▶   Integer.compare(a[2], b[2]));
4   UnionFind uf = new UnionFind(n);
5   List<int[]> T = new ArrayList<>();
6   for (int[] e : E) {
7       if (uf.union(e[0], e[1])) // döngüsüz
8           T.add(e);
9       if (T.size() == n - 1) break;
10  }
11  return T;
12 }

```

DEĞİŞKENLER

`n = 10`



uf ormanı (ok → ebeveyn)



kenar (sıralı)

I-J 0

A-E 1

C-I 1

E-F 1

G-H 1

B-D 2

C-J 2

D-E 2

D-H 2

A-D 4

B-C 4

C-H 4

G-I 4

A-B 5

D-F 5

H-I 6

F-G 7

D-G 11

|T|=0, ağırlık=0

Kenarlar ağırlığa göre sıralanır (A=0 ... J=9; `sort` kararlıdır, eşit ağırlıklar girdi sırasında kalır). $O(m \log m)$.

Kod İzleme: `kruskal()` + Union-Find

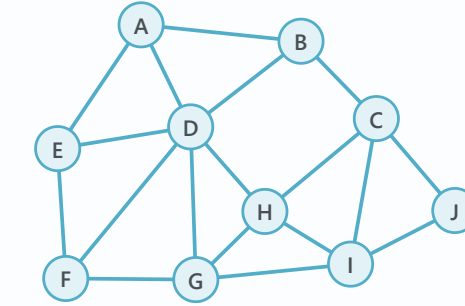
```

1 List<int[]> kruskal(int n, List<int[]> E) {
2   E.sort((a, b) ->           // {u, v, w}
3     Integer.compare(a[2], b[2]));
4   ▶ UnionFind uf = new UnionFind(n);
5   ▶ List<int[]> T = new ArrayList<>();
6   for (int[] e : E) {
7     if (uf.union(e[0], e[1])) // döngüsüz
8       T.add(e);
9     if (T.size() == n - 1) break;
10  }
11  return T;
12 }

```

DEĞİŞKENLER

`n = 10`



uf ormanı (ok → ebeveyn)



kenar (sıralı)

I-J 0

A-E 1

C-I 1

E-F 1

G-H 1

B-D 2

C-J 2

D-E 2

D-H 2

A-D 4

B-C 4

C-H 4

G-I 4

A-B 5

D-F 5

H-I 6

F-G 7

D-G 11

|T|=0, ağırlık=0

`new UnionFind(10)` : her düğüm kendi kümesi (10 kök). T boş.

Kod İzleme: `kruskal()` + Union-Find

```

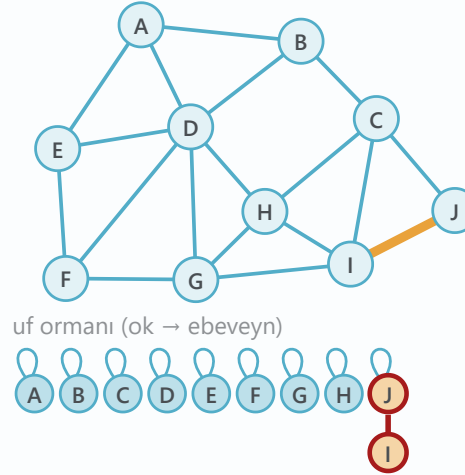
1 List<int[]> kruskal(int n, List<int[]> E) {
2   E.sort((a, b) ->           // {u, v, w}
3     Integer.compare(a[2], b[2]));
4   UnionFind uf = new UnionFind(n);
5   List<int[]> T = new ArrayList<>();
6   for (int[] e : E) {
7     ▶ if (uf.union(e[0], e[1])) // döngüsüz
8       ▶ T.add(e);
9     if (T.size() == n - 1) break;
10  }
11  return T;
12 }

```

DEĞİŞKENLER

`e = {I, J, 0}`

`|T| = 1`



kenar (sıralı)

I-J 0 ✓

A-E 1

C-I 1

E-F 1

G-H 1

B-D 2

C-J 2

D-E 2

D-H 2

A-D 4

B-C 4

C-H 4

G-I 4

A-B 5

D-F 5

H-I 6

F-G 7

D-G 11

$|T|=1$, ağırlık=0

I-J (0): kökler $I \neq J \rightarrow$ **union** true: ağaçlar birleşir ($I \rightarrow J$), kenar T'ye eklenir.

Kod İzleme: `kruskal()` + Union-Find

```

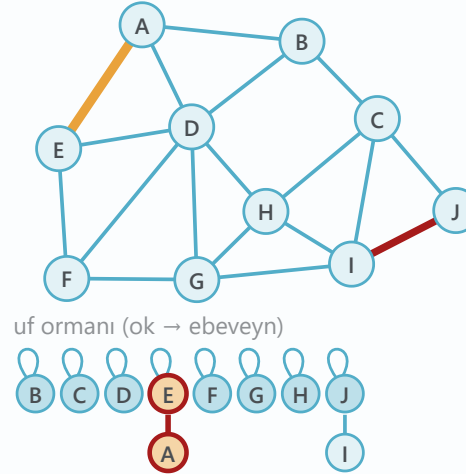
1 List<int[]> kruskal(int n, List<int[]> E) {
2   E.sort((a, b) ->           // {u, v, w}
3     Integer.compare(a[2], b[2]));
4   UnionFind uf = new UnionFind(n);
5   List<int[]> T = new ArrayList<>();
6   for (int[] e : E) {
7     ▶ if (uf.union(e[0], e[1])) // döngüsüz
8       ▶ T.add(e);
9     if (T.size() == n - 1) break;
10  }
11  return T;
12 }

```

DEĞİŞKENLER

`e = {A,E,1}`

`|T| = 2`



kenar (sıralı)

I-J 0 ✓

A-E 1 ✓

C-I 1

E-F 1

G-H 1

B-D 2

C-J 2

D-E 2

D-H 2

A-D 4

B-C 4

C-H 4

G-I 4

A-B 5

D-F 5

H-I 6

F-G 7

D-G 11

|T|=2, ağırlık=1

A-E (1): kökler $A \neq E \rightarrow$ **union** true: ağaçlar birleşir ($A \rightarrow E$), kenar T'ye eklenir.

Kod İzleme: `kruskal()` + Union-Find

```

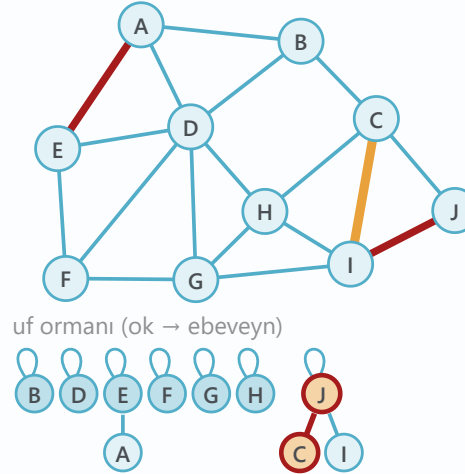
1 List<int[]> kruskal(int n, List<int[]> E) {
2   E.sort((a, b) ->           // {u, v, w}
3     Integer.compare(a[2], b[2]));
4   UnionFind uf = new UnionFind(n);
5   List<int[]> T = new ArrayList<>();
6   for (int[] e : E) {
7     ▶ if (uf.union(e[0], e[1])) // döngüsüz
8       ▶ T.add(e);
9     if (T.size() == n - 1) break;
10  }
11  return T;
12 }

```

DEĞİŞKENLER

`e = {C, I, 1}`

`|T| = 3`



kenar (sıralı)

I-J 0 ✓

A-E 1 ✓

C-I 1 ✓

E-F 1

G-H 1

B-D 2

C-J 2

D-E 2

D-H 2

A-D 4

B-C 4

C-H 4

G-I 4

A-B 5

D-F 5

H-I 6

F-G 7

D-G 11

|T|=3, ağırlık=2

C-I (1): kökler $C \neq J \rightarrow$ `union` true: ağaçlar birleşir ($C \rightarrow J$), kenar T'ye eklenir.

Kod İzleme: `kruskal()` + Union-Find

```

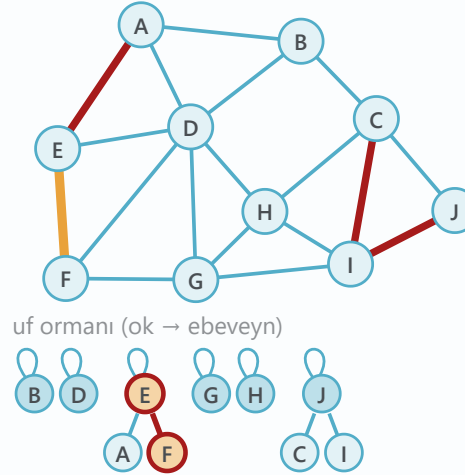
1 List<int[]> kruskal(int n, List<int[]> E) {
2   E.sort((a, b) ->           // {u, v, w}
3     Integer.compare(a[2], b[2]));
4   UnionFind uf = new UnionFind(n);
5   List<int[]> T = new ArrayList<>();
6   for (int[] e : E) {
7     ▶ if (uf.union(e[0], e[1])) // döngüsüz
8       T.add(e);
9     if (T.size() == n - 1) break;
10  }
11  return T;
12 }

```

DEĞİŞKENLER

`e = {E, F, 1}`

`|T| = 4`



kenar (sıralı)

I-J 0 ✓

A-E 1 ✓

C-I 1 ✓

E-F 1 ✓

G-H 1

B-D 2

C-J 2

D-E 2

D-H 2

A-D 4

B-C 4

C-H 4

G-I 4

A-B 5

D-F 5

H-I 6

F-G 7

D-G 11

|T|=4, ağırlık=3

E-F (1): kökler $E \neq F \rightarrow$ `union` true: ağaçlar birleşir ($F \rightarrow E$), kenar T'ye eklenir.

Kod İzleme: `kruskal()` + Union-Find

```

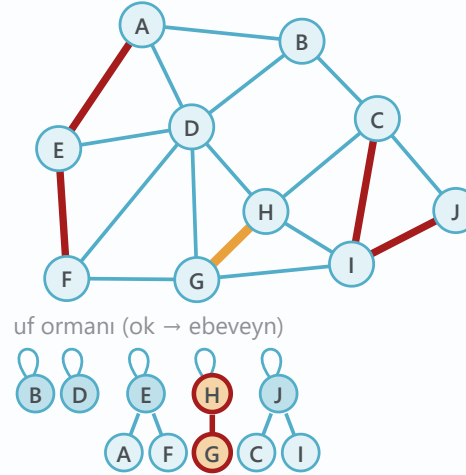
1 List<int[]> kruskal(int n, List<int[]> E) {
2   E.sort((a, b) ->           // {u, v, w}
3     Integer.compare(a[2], b[2]));
4   UnionFind uf = new UnionFind(n);
5   List<int[]> T = new ArrayList<>();
6   for (int[] e : E) {
7     ▶ if (uf.union(e[0], e[1])) // döngüsüz
8       T.add(e);
9     if (T.size() == n - 1) break;
10  }
11  return T;
12 }

```

DEĞİŞKENLER

`e = {G,H,1}`

`|T| = 5`



kenar (sıralı)

I-J 0 ✓

A-E 1 ✓

C-I 1 ✓

E-F 1 ✓

G-H 1 ✓

B-D 2

C-J 2

D-E 2

D-H 2

A-D 4

B-C 4

C-H 4

G-I 4

A-B 5

D-F 5

H-I 6

F-G 7

D-G 11

|T|=5, ağırlık=4

G-H (1): kökler $G \neq H \rightarrow$ `union` true: ağaçlar birleşir ($G \rightarrow H$), kenar T'ye eklenir.

Kod İzleme: `kruskal()` + Union-Find

```

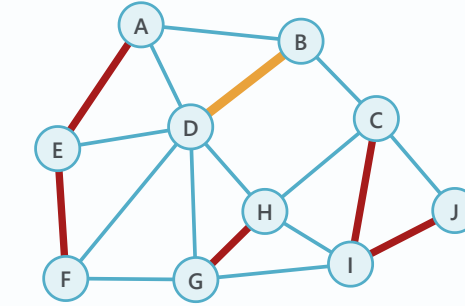
1 List<int[]> kruskal(int n, List<int[]> E) {
2   E.sort((a, b) ->           // {u, v, w}
3     Integer.compare(a[2], b[2]));
4   UnionFind uf = new UnionFind(n);
5   List<int[]> T = new ArrayList<>();
6   for (int[] e : E) {
7     ▶ if (uf.union(e[0], e[1])) // döngüsüz
8       ▶ T.add(e);
9     if (T.size() == n - 1) break;
10  }
11  return T;
12 }

```

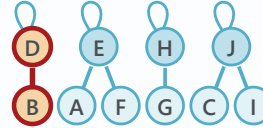
DEĞİŞKENLER

`e = {B,D,2}`

`|T| = 6`



uf ormanı (ok → ebeveyn)



kenar (sıralı)

I-J 0 ✓

A-E 1 ✓

C-I 1 ✓

E-F 1 ✓

G-H 1 ✓

B-D 2 ✓

C-J 2

D-E 2

D-H 2

A-D 4

B-C 4

C-H 4

G-I 4

A-B 5

D-F 5

H-I 6

F-G 7

D-G 11

|T|=6, ağırlık=6

B-D (2): kökler $B \neq D \rightarrow$ `union` true: ağaçlar birleşir ($B \rightarrow D$), kenar T'ye eklenir.

Kod İzleme: `kruskal()` + Union-Find

```

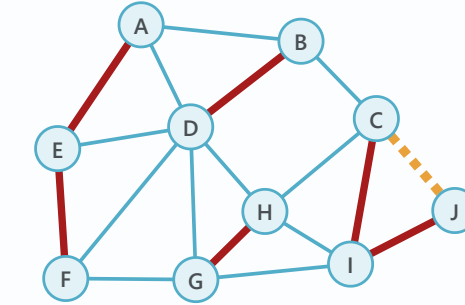
1 List<int[]> kruskal(int n, List<int[]> E) {
2   E.sort((a, b) ->           // {u, v, w}
3     Integer.compare(a[2], b[2]));
4   UnionFind uf = new UnionFind(n);
5   List<int[]> T = new ArrayList<>();
6   for (int[] e : E) {
7     ▶ if (uf.union(e[0], e[1])) // döngüsüz
8       T.add(e);
9     if (T.size() == n - 1) break;
10  }
11  return T;
12 }

```

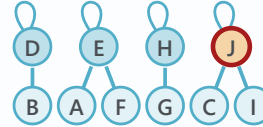
DEĞİŞKENLER

`e = {C, J, 2}`

`|T| = 6`



uf ormanı (ok → ebeveyn)



kenar (sıralı)

I-J 0 ✓

A-E 1 ✓

C-I 1 ✓

E-F 1 ✓

G-H 1 ✓

B-D 2 ✓

~~C-J 2 ✗~~

D-E 2

D-H 2

A-D 4

B-C 4

C-H 4

G-I 4

A-B 5

D-F 5

H-I 6

F-G 7

D-G 11

|T|=6, ağırlık=6

C-J (2): $\text{find}(C) = \text{find}(J) = J \rightarrow$ aynı ağaç, `union` false döner: kenar **döngü kurar, atlanır**.

Kod İzleme: `kruskal()` + Union-Find

```

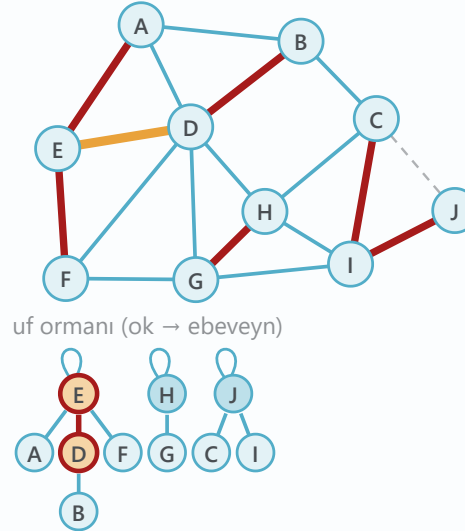
1 List<int[]> kruskal(int n, List<int[]> E) {
2   E.sort((a, b) ->           // {u, v, w}
3     Integer.compare(a[2], b[2]));
4   UnionFind uf = new UnionFind(n);
5   List<int[]> T = new ArrayList<>();
6   for (int[] e : E) {
7     ▶ if (uf.union(e[0], e[1])) // döngüsüz
8       ▶ T.add(e);
9     if (T.size() == n - 1) break;
10  }
11  return T;
12 }

```

DEĞİŞKENLER

`e = {D,E,2}`

`|T| = 7`



uf ormanı (ok → ebeveyn)

kenar (sıralı)

I-J 0 ✓

A-E 1 ✓

C-I 1 ✓

E-F 1 ✓

G-H 1 ✓

B-D 2 ✓

~~E-J 2 ✗~~

D-E 2 ✓

D-H 2

A-D 4

B-C 4

C-H 4

G-I 4

A-B 5

D-F 5

H-I 6

F-G 7

D-G 11

|T|=7, ağırlık=8

D-E (2): kökler $D \neq E \rightarrow$ `union` true: ağaçlar birleşir ($D \rightarrow E$), kenar T'ye eklenir.

Kod İzleme: `kruskal()` + Union-Find

```

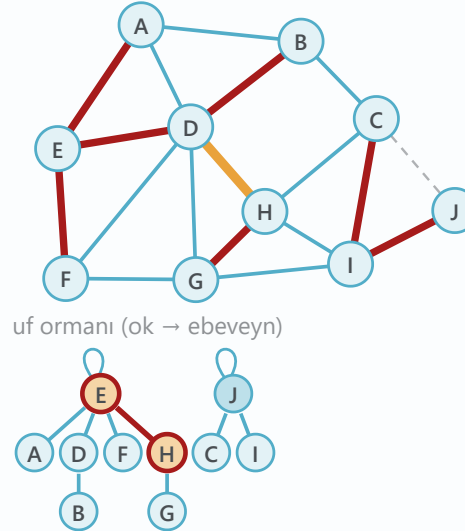
1 List<int[]> kruskal(int n, List<int[]> E) {
2   E.sort((a, b) ->           // {u, v, w}
3     Integer.compare(a[2], b[2]));
4   UnionFind uf = new UnionFind(n);
5   List<int[]> T = new ArrayList<>();
6   for (int[] e : E) {
7     ▶ if (uf.union(e[0], e[1])) // döngüsüz
8       T.add(e);
9     if (T.size() == n - 1) break;
10  }
11  return T;
12 }

```

DEĞİŞKENLER

`e = {D,H,2}`

`|T| = 8`



kenar (sıralı)

I-J 0 ✓

A-E 1 ✓

C-I 1 ✓

E-F 1 ✓

G-H 1 ✓

B-D 2 ✓

~~C-J 2 ✗~~

D-E 2 ✓

D-H 2 ✓

A-D 4

B-C 4

C-H 4

G-I 4

A-B 5

D-F 5

H-I 6

F-G 7

D-G 11

|T|=8, ağırlık=10

D-H (2): kökler $E \neq H \rightarrow$ `union` true: ağaçlar birleşir (H → E), kenar T'ye eklenir.

Kod İzleme: `kruskal()` + Union-Find

```

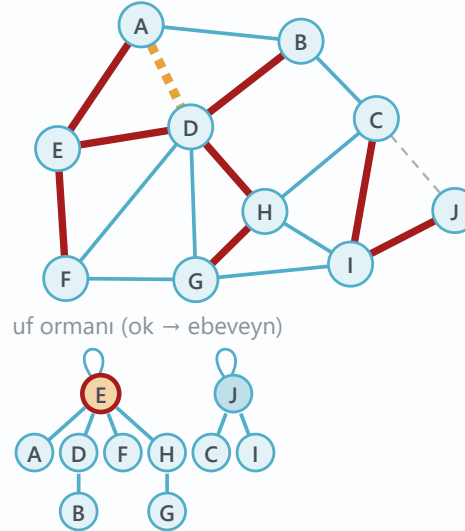
1 List<int[]> kruskal(int n, List<int[]> E) {
2   E.sort((a, b) ->           // {u, v, w}
3     Integer.compare(a[2], b[2]));
4   UnionFind uf = new UnionFind(n);
5   List<int[]> T = new ArrayList<>();
6   for (int[] e : E) {
7     ► if (uf.union(e[0], e[1])) // döngüsüz
8       T.add(e);
9     if (T.size() == n - 1) break;
10  }
11  return T;
12 }

```

DEĞİŞKENLER

`e = {A,D,4}`

`|T| = 8`



kenar (sıralı)

I-J 0 ✓

A-E 1 ✓

C-I 1 ✓

E-F 1 ✓

G-H 1 ✓

B-D 2 ✓

E-J 2 ✗

D-E 2 ✓

D-H 2 ✓

~~A-D 4 ✗~~

B-C 4

C-H 4

G-I 4

A-B 5

D-F 5

H-I 6

F-G 7

D-G 11

|T|=8, ağırlık=10

A-D (4): `find(A) = find(D) = E` → aynı ağaç,
`union` false döner: kenar **döngü kurar, atlanır**.

Kod İzleme: `kruskal()` + Union-Find

```

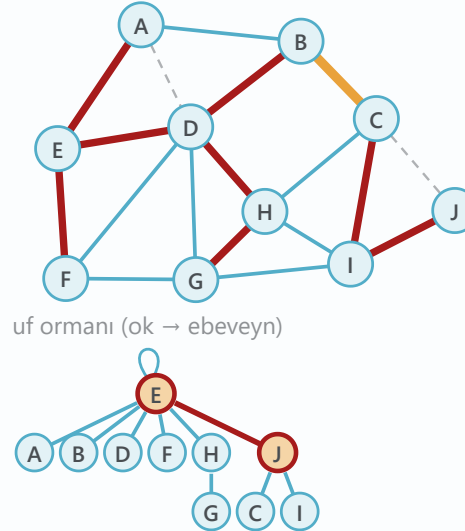
1 List<int[]> kruskal(int n, List<int[]> E) {
2   E.sort((a, b) ->           // {u, v, w}
3     Integer.compare(a[2], b[2]));
4   UnionFind uf = new UnionFind(n);
5   List<int[]> T = new ArrayList<>();
6   for (int[] e : E) {
7     ▶ if (uf.union(e[0], e[1])) // döngüsüz
8       T.add(e);
9     ▶ if (T.size() == n - 1) break;
10  }
11  return T;
12 }

```

DEĞİŞKENLER

`e = {B,C,4}`

`|T| = 9`



kenar (sıralı)

I-J 0 ✓

A-E 1 ✓

C-I 1 ✓

E-F 1 ✓

G-H 1 ✓

B-D 2 ✓

~~E-J 2 ✗~~

D-E 2 ✓

D-H 2 ✓

~~A-D 4 ✗~~

B-C 4 ✓

C-H 4

G-I 4

A-B 5

D-F 5

H-I 6

F-G 7

D-G 11

|T|=9, ağırlık=14

B-C (4): kökler $E \neq J \rightarrow$ `union` true: ağaçlar birleşir ($J \rightarrow E$), kenar T'ye eklenir. $|T| = 9 = n - 1 \rightarrow$ **break.**

Kod İzleme: `kruskal()` + Union-Find

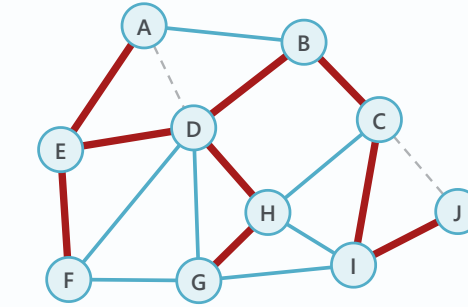
```

1 List<int[]> kruskal(int n, List<int[]> E) {
2   E.sort((a, b) ->           // {u, v, w}
3     Integer.compare(a[2], b[2]));
4   UnionFind uf = new UnionFind(n);
5   List<int[]> T = new ArrayList<>();
6   for (int[] e : E) {
7     if (uf.union(e[0], e[1])) // döngüsüz
8       T.add(e);
9     if (T.size() == n - 1) break;
10  }
11  return T;
12 }

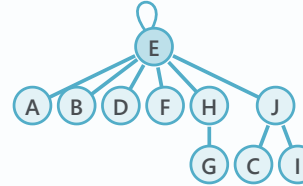
```

DEĞİŞKENLER

`|T| = 9`



uf ormanı (ok → ebeveyn)



kenar (sıralı)

I-J 0 ✓

A-E 1 ✓

C-I 1 ✓

E-F 1 ✓

G-H 1 ✓

B-D 2 ✓

~~E-J 2 ✗~~

D-E 2 ✓

D-H 2 ✓

~~A-D 4 ✗~~

B-C 4 ✓

C-H 4

G-I 4

A-B 5

D-F 5

H-I 6

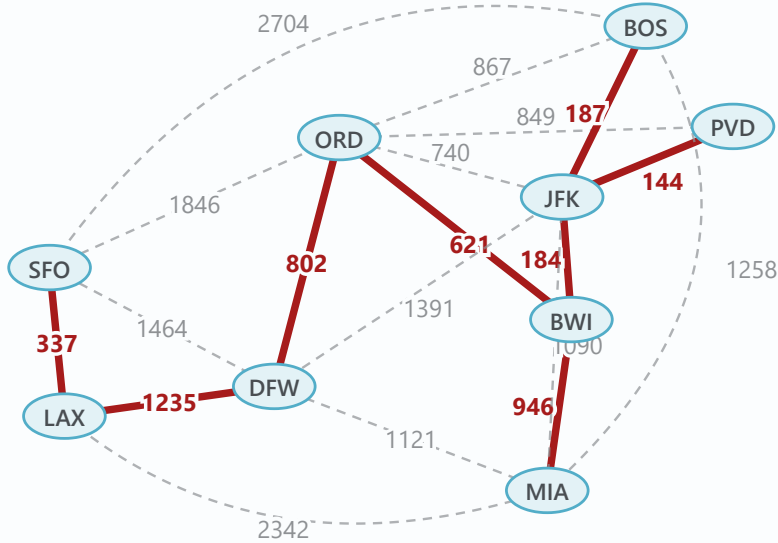
F-G 7

D-G 11

|T|=9, ağırlık=14

`return T` : 9 kenar, toplam ağırlık **14**. Tek ağaç kaldı. Kalan 7 kenara hiç bakılmadı.

İkinci Örnek: Havalimanları



Goodrich'teki örnek: 9 havalimanı, 19 uçuş hattı (mil). Kruskal sırası:

144, 184, 187, 337, 621	kabul
740 ORD-JFK	ret (döngü)
802	kabul
849, 867	ret
946	kabul
1090, 1121	ret
1235 LAX-DFW	kabul → 8 kenar

Toplam: **4456** mil.

Sık Yapılan Hatalar

1. Kümedeki öğeyi değiştirmek

Eklendikten sonra hashCode'u değişen nesne yanlış kovada kalır: `contains` false döner.

```
p.x = 5; set.contains(p) → false
```

2. equals var, hashCode yok

"Eşit" iki nesne büyük olasılıkla farklı kovalara düşer
→ HashSet'te

iki kez

bulunur. İkisi birlikte ezilmeli.

3. Gezinirken silmek

for-each içinde `set.remove(x)` →
`ConcurrentModificationException`.
`set.removeIf(x → x % 2 == 0);`

4. Kökler yerine öğeleri bağlamak

`ebeveyn[x] = y` yanlış: x'in ağacı bölünür.
Doğrusu:
`ebeveyn[find(x)] = find(y);`

5. Eşit rütbede artırmayı unutmak

`sira[ky]++` yoksa rütbe yüksekliği izlemez; $O(\log n)$ garantisi kaybolur.

6. TreeSet ile tutarsız compare

`compare(a, b) == 0` olan farklı nesnelerden yalnız biri saklanır (ör. yalnız soyada göre sıralama).

Özet

Yapı	Temel işlemler	Maliyet	Ne zaman?
HashSet	add, remove, contains	$O(1)^*$	sıra gerekmiyorsa
TreeSet	+ first, floor, aralık	$O(\log n)$	sıralı erişim gerekiyorsa
Çoklu küme	add, count, remove	$O(1)^*$	tekrar sayısı önemliyse
Union-Find	find, union	$O(\alpha(n))$	ayrık kümeler, bağlantı sorguları

* ortalama durum. Union-Find maliyeti rütbe + yol kısaltma ile amortize.

Seçim: üyelik için HashSet , sıralı erişim için TreeSet , dinamik bağlılık için Union-Find.

Etkileşimli Simülatörler

Bu bölümün konularını tarayıcıda deneyerek pekiştirin:

Küme ADT

Öge ekleme/çıkarma, üyelik testi, birleşim, kesişim ve fark işlemleri.

Ayrık Küme Birleşimi (Union-Find)

find ve union işlemlerini, rütbeye göre birleştirmeyi ve yol kısaltmayı ağaç üzerinde izleyin.

Örnek Problemler

1. $A = \{2, 4, 6, 8\}$, $B = \{1, 2, 3, 4\}$ için $A \cup B$, $A \cap B$, $B - A$ nedir?
2. Bir sözlükteki kelimeleri alfabetik sırayla ve tekrarsız yazdırmak için hangi Set uygulamasını seçersiniz?
3. "kavak" kelimesinin harflerini çoklu kümede tutun: `count('a')` ve `size()` ne döner?
4. 0..5 için sırayla `union(0,1)`, `union(2,3)`, `union(1,3)`, `union(4,5)` yapılıyor (rütbeye göre, eşitlikte ebeveyn[x] = y). Kaç küme kalır? `find(0)` nedir?
5. Kruskal örneğinde C–J kenarı neden reddedildi?

► Cevapları göster

Alıştırmalar

1. Boş HashSet'e (16 kova, kova = $x \& 15$) 12, 28, 44, 7 eklenir. Kovalar? `add(28)` ne döner?
2. $U = \{0..7\}$: $A = 1011\ 0010_2$, $B = 0111\ 0100_2$. $A \mid B$, $A \& B$, $A \& \sim B$ kümeleri nedir?
3. Quick-Find ile n öğede $n - 1$ union: toplam kaç hücreye bakılır?
4. `ebeveyn = [1, 2, 3, 3, 3]` iken yinelemeli `find(0)` sonrası `ebeveyn` ?
5. Rütbeye göre birleştirmede rütbe en fazla kaç olabilir? Neden?
6. Kenarlar tek tek geliyor; her kenardan sonra "çizge bağlı mı?" Hangi yapı, maliyet?

► Cevaplar / ipuçları

Kaynaklar

- Goodrich, Tamassia ve Goldwasser, *Data Structures and Algorithms in Java*, 6. baskı, 2014.
- Cormen, Leiserson, Rivest ve Stein, *Introduction to Algorithms*, 4. baskı, 2022.
- Sedgewick ve Wayne, *Algorithms*, 4. baskı, 2011.
- Oracle, Java SE 17 `Set` , `HashSet` ve `TreeSet` API belgeleri.

Son

Bölüm 12: Kümeler

Dersin sonu · Veri Yapıları dersinin son bölümü